Successive Control Certificates for Safe Autonomy

by

Rameez Wajid

B.E., National University of Sciences and Technology, 2007

M.S., Air University, 2016

M.S., University of Colorado Boulder, 2023

A thesis submitted to the

Faculty of the Graduate School of the

University of Colorado in partial fulfillment

of the requirements for the degree of

Doctor of Philosophy

Department of Computer Science

2026

Committee Members:

Sriram Sankaranarayanan, Chair

Christoffer Heckman

Morteza Lahijanian

Ashutosh Trivedi

Saber Jafarpour

Wajid, Rameez (Ph.D., Computer Science)

Successive Control Certificates for Safe Autonomy

Thesis directed by Professor Sriram Sankaranarayanan

Autonomous systems operating in safety-critical settings require certified controllers. The controller should always obey the safety / reachability specifications and these guarantees should hold for the actual nonlinear dynamics, without need for linearization. Certificate functions such as control barrier and control Lyapunov functions provide guarantees of this kind, and sum-of-squares (SOS) programming gives a way to compute them for polynomial systems. Three difficulties limit this approach in practice. The joint search for a certificate and a controller is bilinear and therefore nonconvex; a single certificate tends to certify only a small region of the state space; and the exact arithmetic needed to fully trust a certificate scales poorly.

This dissertation addresses these difficulties with one central idea: rather than searching for one large certificate, it constructs a large certified region by composing many small ones. Each small certificate is made easy to compute by fixing the control input to a value from a finite set, which removes the bilinearity and reduces synthesis to a convex SOS feasibility problem. The small certificates are then assembled into a successive hierarchy in which each new certificate drives the system into the region already certified, so that their union covers much more of the state space than any single certificate. A state-dependent switching controller decides which fixed input to apply and switches only finitely often. This yields infinite-horizon safety and finite-time reachability guarantees for the closed-loop system.

The same construction is employed for three specifications. Staying inside a progressively larger safe set is handled with *successive control barrier functions*. Reaching a goal is handled with *successive control Lyapunov functions* that enlarge the certified region of attraction. Avoiding moving obstacles is handled with *repulsive control barrier functions*, together with a translational symmetry that recenters the system on the obstacle. This reduces the moving-obstacle problem to a fixed one

with measurement-induced jumps. These certificates are synthesized using floating-point SDP solvers and may violate their own defining constraints. The dissertation develops a method that repairs each certificate into an exact, rational, diagonally dominant sum-of-squares form whose validity can be checked by a short standalone program or by a theorem prover. The methods are evaluated on a range of nonlinear benchmarks, where they certify larger regions than single certificates, compare favorably with neural-certificate baselines, and scale to systems beyond the reach of grid-based reachability methods.

Dedication

For my parents, across the distance,

and for Sana, who was here for all of it;

and for Izza, Azlan, and Zirwa.

Acknowledgements

I owe my deepest gratitude to my advisor, Sriram Sankaranarayanan, for introducing me to the world of formal methods, for teaching me what good research looks like, for guiding this work and for his patience with my questions and my shortcomings.

Next, I want to thank my dissertation committee members: Christoffer Heckman, Morteza Lahijanian, Ashutosh Trivedi, and Saber Jafarpour, for providing valuable insights, constructive feedback, and challenging me along the way. I also want to thank our collaborators from Toyota Research North America, Georgios Fainekos and Bardh Hoxha, for their suggestions and critical analysis of the ideas explored, and for funding the final stretch of this thesis.

I am thankful to the former and current members of the CUPLV group for discussions, both research and otherwise, and for providing feedback and support. Especially, I thank Vishnu, CK, Mateo, Nick, Felipe, Christian, Lalith, Kirby, Masoumeh, Sara, Guillaume, Kandai, Monal, Emily, Sebastian, Shadi, Shawn, Prasanth, Alireza, Bingzhou, Mahendra, Jasdeep, Serra, Daniel, Daphna, Kunha, Cade, Dakota, Andrew, Amin, Lekai, and Mohammed. I also want to thank Evan and Gowtham for keeping the CUPLV lab spirit. I extend my gratitude to the CS graduate advisor Rajshree Shrestha for her efforts.

I also want to thank our friends from the ICB community in and around Boulder for providing a welcoming environment for my family.

Finally, and most of all, to my family. To my parents and my siblings, whose support reached me across every distance; to Sana, who carried more than her share so that this could exist; and to Izza, Azlan, and Zirwa, for the time this took from them.

Contents

Chapter

Tables

Table

Figures

Figure

"Divide each difficulty into as many parts as is feasible and necessary to resolve it."

— René Descartes, *Discourse on Method*

Chapter 1

Introduction

"I'm sorry, Dave. I'm afraid I can't do that."
— HAL 9000, *2001: A Space Odyssey*

Autonomous systems in safety-critical settings need controllers with mathematical guarantees. But three obstacles limit certificate-based control in practice: the joint search for a certificate and a controller is bilinear and nonconvex, a single certificate covers only a small part of the state space, and the exact arithmetic needed to fully trust a certificate scales poorly. This dissertation answers all three with one move: fix the control input to a finite set so that each certificate is found by a convex sum-of-squares program, then compose many such certificates into a *successive* hierarchy whose union certifies a far larger region, under a switching controller that provably switches finitely often. This chapter motivates the problem, states the thesis, and outlines the contributions.

Autonomous and robotic systems are increasingly expected to operate in environments where safety is a primary requirement. Unlike purely computational systems, robotic systems interact with the physical world, and failures may lead to collisions, loss of stability, or violation of operational constraints. This motivates the development of mathematical methods that can provide guarantees about the behavior of such systems before and during deployment.

This dissertation studies certificate-based methods for safe and goal-directed autonomy in nonlinear robotic systems. In particular, it focuses on how Lyapunov and barrier certificates can be used, extended, and composed to provide formal guarantees for systems operating under constraints. The central theme of the thesis is that single, static certificates are often too conservative for realistic robotic systems, and that successive certificates can provide more flexible guarantees.

1.1 Motivation

Self-driving cars, aerial vehicles, mobile manipulators, and the increasingly capable robots that share our roads, factories, and homes are being entrusted with safety-critical decisions, with the worst-case consequences. Whereas, a controller that produces a suboptimal trajectory is an inconvenience; a controller that occasionally drives a vehicle into an obstacle, or fails to bring a system back to a safe operating point is a hazard. As these systems move from controlled laboratory demonstrations into open, uncertain, and dynamic environments, the question is no longer merely *"does the controller usually work"*, but that of *"can we prove that the closed loop will never do something unsafe, and that it will always make progress toward its objective"*.

This dissertation is concerned with providing such proofs, and with making them *constructive*. We work within the well-established paradigm of *certificate-based control*, in which a property of a closed-loop system is established not by exhaustively simulating its trajectories, but by providing a *certificate*, whose algebraic and differential properties imply the desired behavior for *all* trajectories. Lyapunov functions, which certify stability and convergence to a target, and barrier functions, which certify the forward invariance of a safe set, are the two standard examples [4, 7, 68, 80]. A Lyapunov-like function decreases along trajectories and thereby forces them toward a goal; a

barrier function maintains its sign and thereby keeps trajectories out of an unsafe set. Their *control* counterparts; control Lyapunov functions (CLFs) and control barrier functions (CBFs); additionally encode the existence of an admissible control input that enforces the property, so that a single certificate simultaneously witnesses the property and defines a feedback law to achieve it.

The appeal of certificates is threefold. First, the guarantee is *offline* and *trajectory-independent*, that is, once a valid certificate is in hand, safety or reachability is established for the entire certified region without solving any runtime optimization problem or enumerating initial conditions. Second, certificates are *composable*, meaning that barrier and Lyapunov conditions can be combined to express richer specifications, and (as we will reason throughout this dissertation) families of certificates can be assembled into hierarchies that certify far more than any one of them could alone. Third, for the important class of *polynomial* dynamical systems, the search for a certificate can be posed as a *sum-of-squares* (SOS) program [39, 64], turning a problem of universal quantification over a continuous state space into a finite-dimensional semidefinite feasibility problem. This last point is what makes the certificate-based approach not just a proof technique but a *synthesis* technique.

1.2 Safety Filters and Shielding

A certificate is not only a proof but also an *enforcement* mechanism. This makes certificates well-suited to a problem that has become central as autonomy increasingly relies on learned and otherwise unverified controllers. How to let a high-performance but untrusted nominal policy drive the system most of the time, while *guaranteeing* that the closed loop never does anything unsafe. The architecture that answers this question is the *safety filter* (equivalently, a *shield* or *runtime monitor*); a wrapper that sits between the nominal controller and the plant, passes the nominal input through whenever it is provably safe, and overrides it with a certified fallback whenever safety would otherwise be jeopardized. The design goal is *minimal intervention*, that is, the filter should be *least restrictive*, altering the nominal behavior only when strictly necessary, so that performance is preserved up to the boundary of the safe region [25, 95].

Three largely parallel lines of work have arrived at this architecture from different starting

points, and the certificate is the common object among them. The first is the *control barrier function* (CBF) safety filter. Here, a barrier certificate defines a safe set, and at each instant a quadratic program (the CBF-QP) computes the admissible input closest to the nominal one subject to the barrier's decrease condition, so that forward invariance of the safe set is maintained while the nominal input is followed as closely as possible [4, 5]. The second is the *Hamilton–Jacobi (HJ) reachability* safety filter, in which a reachability value function certifies a maximal safe set and induces a least restrictive switching law that hands control to a safety-preserving policy exactly at the boundary of that set. Fisac et al. [25] make this framework work alongside an arbitrary learning algorithm, intervening only when the computed guarantees require it. The third is *predictive* safety filtering, which certifies safety implicitly through the recursive feasibility of a model-predictive control problem with a safe terminal set, and modifies the learning agent's input only when no safe backup trajectory can be found from the proposed action [91]. These three viewpoints; reachability, barrier, and predictive, are by now understood as facets of a single data-driven safety-filtering paradigm [90].

A fourth, more discrete-flavored line originates in formal methods under the name *"shielding"*. Rather than a continuous-state certificate, a shield is a reactive system synthesized from a temporal logic specification that monitors the nominal controller's proposed actions and corrects them whenever a correct-by-construction safety automaton would be violated [14, 38]. Shielding has been used most prominently to render reinforcement learning safe *during* both training and deployment, by inserting the shield between the agent and the environment so that no unsafe action is ever executed [3]. The conceptual content is identical to the control-theoretic safety filter, which acts as a guardian that overrides an untrusted policy only when necessary. They differ primarily in whether the underlying certificate is a continuous sublevel set or a discrete automaton.

This dissertation is concerned with the certificate that makes such a guardian sound, and with making that certificate *large* so that the guardian intervenes rarely. The recurring difficulty, common to every filter above, is that the certified safe region is typically a conservative inner approximation of the true one. A small certified set forces the filter to override the nominal controller far more often than the actual demand. The successive certificate constructions developed in the technical

chapters attack this conservatism. In particular, Chapter 3 pairs successive control barrier functions with a transit-time-based, chattering-free *sampled-data* runtime monitor that switches among the barriers. The controlled invariant set for the shield is the union of multiple barriers and is therefore substantially larger than any single CBF-QP filter. Chapter 6 develops an *event-triggered* safety filter for moving obstacles, alternating between passive monitoring of the nominal controller and active takeover by a repulsive barrier. Throughout, our emphasis is complementary to the online optimization viewpoint of the CBF-QP and predictive filters. Instead of redesigning the online rule, we enlarge the offline-certified region on which any such rule can allow the nominal controller to operate safely.

1.3 The Synthesis Problem and Its Challenges

The promise of automated certificate synthesis is, unfortunately, hampered by three persistent difficulties that occur across stability and safety, across static and dynamic environments, and across every system we study in this dissertation.

Bilinearity. A control certificate must couple the certificate function V (or B) with a feedback law κ, both of which are unknown at design time. The defining decrease (or invariance) condition involves the Lie derivative $\nabla V(\mathbf{x}) \cdot f(\mathbf{x}, \kappa(\mathbf{x}))$, in which the unknown certificate and the unknown feedback are *multiplied* together. The resulting constraints are bilinear and, in general, non-convex [69, 81]. The standard remedies include bilinear matrix inequalities, alternating "V-K" iteration [87], or restricting attention to control-affine structure [69]. They either inherit the non-convexity, only find local optima, or apply to a narrow class of systems. The bilinearity barrier is the first *obstacle* any practical synthesis method must overcome.

Conservatism. Even when a single valid certificate is found, the region it certifies tends to be a small and conservative inner approximation of the true controllable region. A monolithic Lyapunov function whose sublevel set must satisfy a decrease condition everywhere is pulled toward shapes that the SOS template can represent globally, sacrificing coverage near the curved or input-saturated boundaries of the feasible set. A single barrier function, synthesized for a fixed control

input, marks off as "uncontrollable" many states from which safety is in fact achievable [92]. In a safety filter, this conservatism translates directly into a controller that intervenes on the nominal behavior far more often than necessary.

Scalability. The most accurate alternatives to certificate synthesis come at a steep computational price. Hamilton-Jacobi (HJ) reachability [9, 56] computes the exact backward reachable (and reach-while-stay) set by solving a partial differential equation on a state-space grid. It is optimal but succumbs to the curse of dimensionality beyond four or five state variables. Learning-based certificates like neural Lyapunov and barrier functions verified post hoc with SMT solvers such as those in FOSSIL [19, 23] can be trained quickly, but the verification step scales poorly with dimension and certificate degree, and the certified regions remain modest.

These three difficulties are not independent. The bilinearity barrier is what pushes practitioners toward computationally inexpensive relaxations (fixed feedback, low-degree templates), and those relaxations are precisely what make the resulting certified regions conservative. A method that breaks the bilinearity barrier *cheaply* can afford to be applied many times, and the union of many cheap certificates can recover much of the coverage that a single expensive certificate would have provided.

1.4 Thesis Statement and Central Idea

We advance the following thesis:

Thesis Statement

Hierarchical certificates provide a computationally tractable and progressively expandable route to certified safety and stability for nonlinear autonomous systems; encompassing invariance, reachability, and moving-obstacle avoidance. The hierarchy is built from simple certificates, each synthesized by a convex sum-of-squares feasibility problem under a fixed control input, and combined by a state-dependent switching controller.

The mechanism that makes this possible, and that appears in every technical chapter, has two

ingredients.

Control discretization breaks bilinearity. Rather than searching jointly for a certificate and a feedback law, we fix the feedback to a constant input drawn from a *finite* set $U_{\text{fin}} \subset U$ and search only for a certificate. With the input fixed, the Lie derivative becomes *linear* in the coefficients of the certificate, and the synthesis problem collapses to a convex SOS feasibility problem. The apparent loss of generality is mild. We prove (in a form that reappears in each chapter) that whenever a smooth feedback witnessed by a smooth certificate exists, a sufficiently fine finite control set can reproduce the decrease (or invariance) property up to a constant factor, and that for control-affine dynamics the vertices of the polytopic input set suffice. This single structural fact is what converts an intractable bilinear program into a sequence of tractable convex ones.

Successive hierarchies expand the certified region. A single fixed-input certificate covers only a small region. We therefore build a *hierarchy*. The first level certifies a region from which the target property (reaching the goal, or entering the safe set) holds directly. Each subsequent level then treats the union of all previously certified regions as its *new* target, certifying that trajectories initialized in the new outer region are driven, in finite time, into the already-certified inner region. The cumulative certified region $\Omega^* = \bigcup_i \Omega_i$ grows monotonically with the number of levels, while each level remains a convex problem. A state-dependent switching controller selects, at each state, the innermost level whose certificate is active and applies the associated constant input. We design this controller so that the active level index is non-increasing along trajectories, which yields a finite number of switches and freedom from Zeno behavior. Crucially (in contrast to classical multiple-Lyapunov-function and patchy constructions [15, 30, 41]) the levels need *not* be nested, and a lower level is guaranteed to be reached by construction, so that progress on the certified region is built into the hierarchy rather than hoped for.

Figure 1.1 summarizes the approach taken in this dissertation: a simple idea, applied repeatedly and made sound, addresses all three difficulties across several specifications.

The same ingredients, instantiated for different specifications, give the following technical contributions of this dissertation.

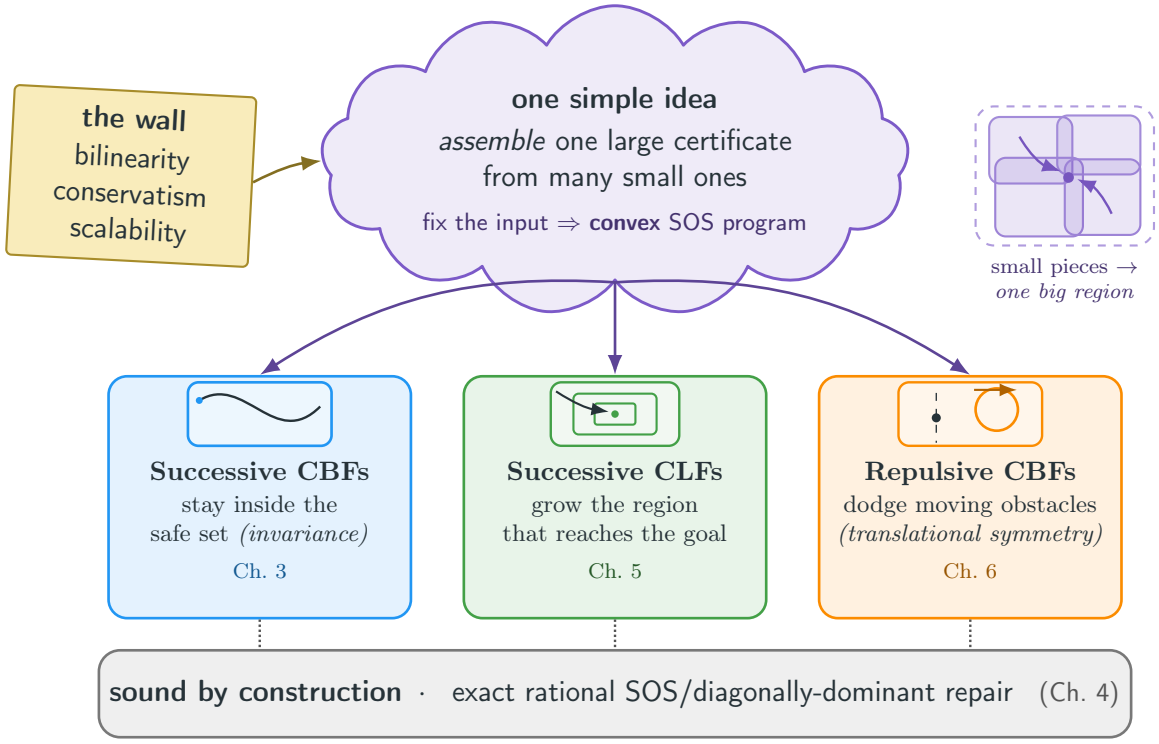


Figure 1.1: A bird's-eye view of the thesis.

1.5 Contributions

The contributions of this dissertation, organized by the chapter that develops them, are as follows.

(1) **Successive control barrier functions for invariance (Chapter 3).** We introduce successive CBFs, a hierarchy of barrier functions in which a later barrier is required to enforce its decrease condition only on the states where all earlier barriers have failed. Each level is synthesized by SOS programming under a fixed control input, and the union of the resulting controlled-invariant sets is substantially larger than any single fixed-input barrier. We define a notion of *transit time*, a lower bound on the time the barrier value needs to cross from a safe threshold to the unsafe boundary, and use it to construct a chattering-free, sampled-data runtime monitor that switches between barriers while preserving an infinite-horizon safety guarantee. The approach scales to benchmarks from two to seven state variables and certifies larger controllable sets than neural CBFs verified with Fossil [23]. This work appeared at HSCC 2025 [92].

(2) **Robustifying SOS proofs (Chapter 4).** SOS certificates are produced by floating-point semidefinite programming solvers, and an apparently feasible solution may in fact violate the certificate conditions because of numerical errors. We develop a post-hoc soundness layer that extracts the Putinar multipliers from the solver, reconstitutes the positivstellensatz residue, and certifies the positive-semidefiniteness of the associated Gram matrices using high-precision arithmetic, building on the validation framework of Roux et al. [75]. This chapter strengthens the guarantees claimed throughout the rest of the dissertation. Every successive certificate we report is accompanied by a numerically robust proof of its defining inequalities.

(3) **Successive control Lyapunov functions for reachability (Chapter 5).** We carry the successive paradigm from safety to stability, building a hierarchy of control Lyapunov-like functions that progressively enlarges the region of attraction toward a target set. Phrased in the language of temporal logic, we decompose the overarching property "Ω^* until X_0" into a chain of

local steps "Ω_i until Ω_{i-1}," each certified by a fixed-input CLF, and we present the accompanying continuous-time state-dependent switching controller with a proof of finite-time reachability, monotone level index, and Zeno-freeness. We demonstrate larger regions of attraction than neural CLF baselines on benchmarks, including Zubov's example, the inverted pendulum, the van der Pol oscillator, and the Caltech ducted fan. This work is being prepared for submission to a journal.

(4) **Repulsive control barrier functions for moving obstacles (Chapter 6).** Finally, we address dynamic environments. For the broad class of systems whose dynamics are invariant under translation in the workspace, we periodically *recenter* the state so that a measured moving obstacle sits at the origin, turning the moving-obstacle problem into a hybrid system with measurement-induced translational jumps. We introduce *repulsive* barrier functions that combine a continuous-time decrease condition with robustness to the worst-case jump, prove an infinite-horizon safety guarantee under nondeterministic obstacle motion between measurements, synthesize the barriers with SOS programming (again expanding the controlled-invariant region via multiple fixed-input barriers), and wrap them in an event-triggered safety filter. This work will be presented at CDC 2026.

Taken together, these chapters show that a single idea, convex per-level synthesis under discretized control, assembled into a switching hierarchy, is broad enough to certify invariance, reachability, and moving-obstacle avoidance, while remaining numerically sound and scaling to systems beyond the reach of grid-based methods.

1.6 Organization of the Dissertation

The remainder of the dissertation is organized as follows. Chapter 2 fixes notation and recalls the background required throughout: polynomial dynamical system models and the robotics benchmarks used, the safety and reachability specifications we certify, the certificate definitions (Lyapunov-like, barrier, control barrier, and control Lyapunov functions), the dual notion of inevitable

collision states, and sum-of-squares programming together with the finite-control-set discretization theorem that underlies every later chapter. Chapter 3 develops successive control barrier functions and the transit-time runtime monitor. Chapter 4 presents the numerical soundness layer for SOS proofs. Chapter 5 develops successive control Lyapunov functions for region-of-attraction expansion. Chapter 6 develops repulsive control barrier functions for moving obstacles via translational symmetry. Finally, Chapter 7 summarizes the contributions and discusses directions for future work, including combined barrier-funnel certificates for simultaneous safety and progress, scalable relaxations such as DSOS/SDSOS, and the integration of these certificates with sampling-based and learning-based planners.

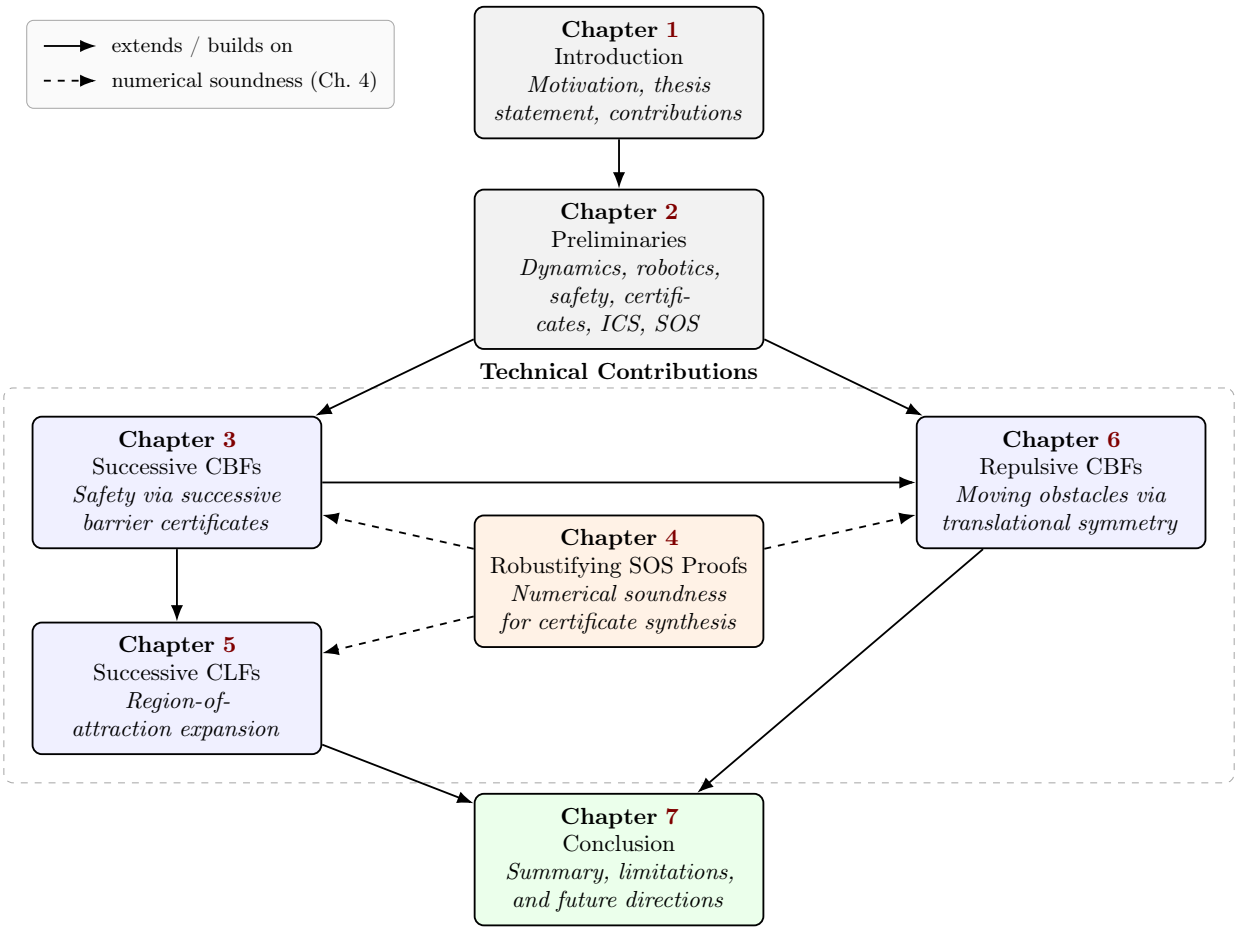


Figure 1.2: Organization of the dissertation. Chapter 2 provides the common mathematical and computational background. Chapters 3-6 form the main technical body of the dissertation, developing successive, robustified, and repulsive certificate methods for safe and goal-directed autonomy.

Chapter 2

Preliminaries

"There is no royal road to geometry."
— Euclid

This chapter collects the background the technical chapters share: nonlinear control-affine dynamics and their polynomial models, the safety and reachability specifications, certificate functions (control barrier and control Lyapunov functions) and their decrease conditions, initialized controlled switched systems as the semantic model for switching controllers, and sum-of-squares programming with the Positivstellensatz as the computational engine that turns certificate conditions into semidefinite programs. Readers familiar with these topics can skim and return as needed.

This chapter collects the definitions, models, and mathematical tools used throughout the dissertation. Section 2.1 introduces the class of polynomial control systems and the robotics benchmarks. Section 2.2 formalizes the safety and reachability specifications we certify. Section 2.3 defines the family of certificates; Lyapunov-like, barrier, control barrier, and control Lyapunov functions, which the later chapters synthesize. Section 2.4 recalls the dual notion of inevitable collision states. Section 2.5 reviews sum-of-squares programming and states the finite-control-set discretization theorem that converts the bilinear synthesis problem into a convex one.

Notation. We write $\mathbb{R}$ for the reals, $\mathbb{N}$ for the naturals, and $\mathbb{R}[\mathbf{x}_1, \dots, \mathbf{x}_n]$ (abbreviated $\mathbb{R}[\mathbf{x}]$) for the ring of multivariate real polynomials in n indeterminates; $\mathbb{R}_d[\mathbf{x}]$ denotes those of total degree at most d. Boldface is reserved for vectors when ambiguity is possible; the ith component of a vector $\mathbf{x}$ is x_i. For a set $A \subseteq \mathbb{R}^n$, $\operatorname{int} A$, $\operatorname{cl} A$, and ∂A denote its interior, closure, and topological boundary. The Euclidean norm is $\|\cdot\|$, and $B_r(\mathbf{x}) = \{\mathbf{y} : \|\mathbf{y} - \mathbf{x}\| \le r\}$ is the closed ball of radius r. A *basic semi-algebraic set* is a set of the form $\{\mathbf{x} \mid p_1(\mathbf{x}) \ge 0, \dots, p_k(\mathbf{x}) \ge 0\}$ for polynomials $p_1, \dots, p_k \in \mathbb{R}[\mathbf{x}]$. We write the Lie derivative of a smooth function V along a vector field g as $\dot{V} = \nabla V \cdot g$.

2.1 Dynamical Systems and Robotics Models

Definition 1 (Polynomial control system) A *polynomial control system* Σ over a state space $X \subseteq \mathbb{R}^n$ is a system of ordinary differential equations

$$\dot{\mathbf{x}}(t) = f\big(\mathbf{x}(t), \mathbf{u}(t)\big), \qquad \mathbf{x} \in X \subseteq \mathbb{R}^n, \quad \mathbf{u} \in U \subseteq \mathbb{R}^m, \tag{2.1}$$

in which each component of f is a polynomial in $(\mathbf{x}, \mathbf{u})$, and the control set U is a compact polytope, in our applications a hyper-rectangle $U = [\mathbf{u}_1^-, \mathbf{u}_1^+] \times \cdots \times [\mathbf{u}_m^-, \mathbf{u}_m^+]$.

For any locally bounded, measurable input signal $\mathbf{u}(\cdot)$, local Lipschitz continuity of f guarantees the existence and forward uniqueness of a trajectory $\mathbf{x} : [0, T_{\max}) \to X$. Given a continuous input signal $\mathbf{u} : [0, T) \to U$, a *trajectory* is a continuously differentiable function $\varphi : [0, T) \to X$ satisfying

$\frac{d\varphi}{dt} = f(\varphi(t), \mathbf{u}(t))$ for all $t \in [0, T)$. We will frequently fix the control to a constant value $\mathbf{u} \equiv v$ and study the resulting autonomous vector field $f(\cdot, v)$, since this is the setting in which each individual certificate is synthesized.

A recurring structural assumption, which we exploit to discretize the control, is that of "*control affine*" systems.

Definition 2 (Control-affine dynamics) The system (2.1) is *control affine* if it can be written $f(\mathbf{x}, \mathbf{u}) = f_0(\mathbf{x}) + f_1(\mathbf{x})\mathbf{u}$ for a polynomial drift f_0 and polynomial input matrix f_1.

Control affine systems with a polytopic input set U implies that the Lie derivative $\nabla V \cdot f(\mathbf{x}, \mathbf{u})$ is affine in $\mathbf{u}$, so its extremum over U is attained at a vertex of U; this is the elementary observation behind choosing $U_{\text{fin}} = \text{vert}(U)$ in Section 2.5.

The benchmark systems used throughout the dissertation are drawn from mobile robotics and aerospace; we describe a few representative ones here for reference.

Example 1 (Dubins / unicycle vehicle) The Dubins vehicle has state (x, y, θ) and a steering input u,

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = u,$$

with constant forward speed v. The right-hand side is independent of the position (x, y), a translation-invariance property we exploit in Chapter 6.

Example 2 (Inverted pendulum) With state $(x_1, x_2) = (\text{angle}, \text{rate})$ and torque u,

$$\dot{x}_1 = x_2, \qquad \dot{x}_2 = \tfrac{g}{l}\left(x_1 - \tfrac{x_1^3}{6}\right) + \tfrac{1}{ml^2}\,u,$$

where the cubic term is the third-order Taylor approximation of sin used to keep the dynamics polynomial.

Example 3 (Kinematic bicycle and wheeled path follower) The kinematic bicycle has state (p_x, p_y, ψ), speed v, steering angle δ, and wheelbase L,

$$\dot{p}_x = v\cos\psi, \qquad \dot{p}_y = v\sin\psi, \qquad \dot{\psi} = \tfrac{v}{L}\tan\delta.$$

Example 4 (Caltech planar ducted fan) The ducted fan in hover mode is a six-state, two-input system

$$\dot{x}_1 = x_4, \quad \dot{x}_4 = (-d\,x_1 + u_1\cos x_3 - (u_2 + mg)\sin x_3)/m,$$

$$\dot{x}_2 = x_5, \quad \dot{x}_5 = (-d\,x_2 + u_1\sin x_3 + (u_2 + mg)\cos x_3)/m,$$

$$\dot{x}_3 = x_6, \quad \dot{x}_6 = r\,u_1/J,$$

which serves as our highest-dimensional benchmark and stresses the scalability of the SOS synthesis.

2.2 Specifications for Safe Autonomy

We certify three families of properties, which together cover the behaviors a safe autonomous system must exhibit. We state them informally here and formalize the relevant certificate for each in Section 2.3. It is convenient to phrase them in the notation of signal temporal logic (STL) [51], writing "Ω **until** X_0" for the property that a trajectory remains in Ω until it reaches X_0.

Invariance (safety). Given an unsafe set $\mathcal{O} \subseteq X$, safety requires that the closed-loop trajectory never enters $\mathcal{O}$. We establish this by exhibiting a controlled-invariant set [8, 12], Ω with $\Omega \cap \mathcal{O} = \emptyset$ and a control strategy that renders Ω forward invariant. This is the specification of Chapters 3 and 6.

Reachability. Given a target set $X_0 \subseteq X$ with an equilibrium $\mathbf{x}^* \in X_0$, reachability requires that the trajectory reach X_0 in finite time while remaining within a certified region $\Omega^* \supseteq X_0$. In temporal-logic terms we seek a control strategy enforcing "Ω^* **until** X_0," and we wish Ω^*, the region of attraction, to be as large as possible. This is the specification of Chapter 5.

Two structural features of these specifications appear in the technical chapters. First, the executions we analyze are sometimes genuinely *hybrid*. In Chapter 6 the state undergoes discrete

jumps each time the obstacle position is measured and the workspace is recentered, so a safety certificate must be robust to both the continuous flow and the measurement-induced resets, in the spirit of hybrid dynamical systems [31]. Second, several of our guarantees are based on *sampled-data* control paradigm. A runtime monitor evaluates the certificate only at discrete times $t_k = k\tau$ and must ensure that the system cannot transit from safe to unsafe within one sampling interval, which is what the transit-time analysis of Chapter 3 provides.

2.3 Certificates

We now define the certificates synthesized in the dissertation. Each is a smooth function whose sign and Lie-derivative conditions imply a property of the closed loop. Throughout, we use the convention in which a barrier function is *positive* on the unsafe set and the controlled-invariant region is the sublevel set $\{\mathbf{x} \mid B(\mathbf{x}) \leq 0\}$, while a Lyapunov-like function is *nonnegative* and *decreasing* toward the target.

2.3.1 Lyapunov-like functions and the region of attraction

Because our objective is reaching a target set X_0 rather than asymptotic stability of a point, we use the slightly more general notion of a *Lyapunov-like* function, which is required to decrease only outside the target.

Definition 3 (Lyapunov-like function) Let $\mathbf{x}^* \in X_0$ and let $D = \{\mathbf{x} \mid \|\mathbf{x} - \mathbf{x}^*\| \leq R\}$ be a ball with $D \supset X_0$. A smooth function $V : X \to \mathbb{R}$ is a *Lyapunov-like function* for the closed-loop system $\dot{\mathbf{x}} = f(\mathbf{x}, \kappa(\mathbf{x}))$ over D if

(i) (**positive definiteness**) $V(\mathbf{x}) \geq \epsilon \|\mathbf{x} - \mathbf{x}^*\|^2$ for some $\epsilon > 0$ and all $\mathbf{x} \in D$, with $V(\mathbf{x}^*) = 0$;

(ii) (**strict decrease**) there is $\xi > 0$ with $\dot{V}(\mathbf{x}) = \nabla V(\mathbf{x}) \cdot f(\mathbf{x}, \kappa(\mathbf{x})) \leq -\xi$ for all $\mathbf{x} \in D \setminus X_0$.

Define $d_V := \inf_{\mathbf{x} \in \partial D} V(\mathbf{x})$, which exists and satisfies $d_V \geq \epsilon R^2$ by compactness of D and positive definiteness of V; choosing a level $\gamma_V < d_V$, the sublevel set $\Omega_V := \{\mathbf{x} \mid V(\mathbf{x}) \leq \gamma_V\}$ is

then a controlled-invariant region of attraction: it is compact, contains $\mathbf{x}^*$, is disjoint from ∂D, and every trajectory initialized in Ω_V reaches X_0 in finite time while remaining in Ω_V. This is described appropriately in Chapter 5.

2.3.2 Barrier and control barrier functions

A *barrier function* certifies safety for a system without control inputs.

Definition 4 (Barrier function [68]) Let $\dot{\mathbf{x}} = f(\mathbf{x})$ be Lipschitz and let $\mathcal{O}$ be the unsafe set. A differentiable $B : \mathbb{R}^n \to \mathbb{R}$ is a *barrier function* if there exist $\epsilon > 0$ and λ such that

$$\forall \mathbf{x} \in \mathcal{O},\ B(\mathbf{x}) \geq \epsilon, \qquad \text{and} \qquad \forall \mathbf{x} \in X,\ \nabla B(\mathbf{x}) \cdot f(\mathbf{x}) \leq -\lambda B(\mathbf{x}).$$

The exponential-decrease form $\dot{B} \leq -\lambda B$ has the property that $e^{\lambda t} B(\varphi(t))$ is non-increasing, so $B(\varphi(0)) \leq 0$ implies $B(\varphi(t)) \leq 0$ for all t, and since $B \geq \epsilon > 0$ on $\mathcal{O}$ the trajectory never enters the unsafe set. A *control* barrier function extends this to systems with inputs.

Definition 5 (Control barrier function) For the system (2.1) with unsafe set $\mathcal{O}$, a differentiable $B : X \to \mathbb{R}$ is a *control barrier function* (CBF) if there exist $\epsilon > 0$ and λ such that

$$\forall \mathbf{x} \in \mathcal{O},\ B(\mathbf{x}) \geq \epsilon, \qquad \text{and} \qquad \forall \mathbf{x} \in X,\ \exists \mathbf{u} \in U :\ \nabla B(\mathbf{x}) \cdot f(\mathbf{x}, \mathbf{u}) \leq -\lambda B(\mathbf{x}).$$

The existential quantifier over $\mathbf{u}$ in the decrease condition is the source of the $\forall\exists$ quantifier alternation that makes CBF synthesis bilinear and non-convex; the finite-control-set theorem of Section 2.5 is what lets us replace the existential by a finite disjunction over fixed inputs and thereby decouple the search. Combining barriers synthesized for different fixed inputs into a single nonsmooth function via the pointwise minimum, $B(\mathbf{x}) = \min_i B_i(\mathbf{x})$, enlarges the controlled-invariant region but reintroduces nonsmoothness [28]; Chapter 3 handles this with transit-time bounds and a sampled-data monitor rather than nonsmooth analysis.

For dynamic environments, we will need a barrier that is additionally robust to discrete state jumps. The following *repulsive* variant, developed in Chapter 6, augments the decrease condition (with the *repulsive* convention $\lambda > 0$, so the barrier is driven downward) with a bounded-jump condition.

Definition 6 (Repulsive barrier function, informal) Partition the state as $(\mathbf{y}, \mathbf{z})$ with workspace variables $\mathbf{y}$ and non-workspace variables $\mathbf{z}$. A smooth $B(\mathbf{y}, \mathbf{z})$ is a *repulsive barrier function* with respect to an unsafe set $\mathcal{O}$ and translational jumps if (i) $B(\mathbf{y}, \mathbf{z}) \geq \epsilon$ on the unsafe set; (ii) for all $(\mathbf{y}, \mathbf{z})$ there exists $\mathbf{u} \in U$ with $\nabla B \cdot f \leq \lambda B$ for a fixed $\lambda > 0$; and (iii) there is a constant $K > 0$ such that $B(\mathbf{y} - \mathbf{y}_o, \mathbf{z}) - B(\mathbf{y}, \mathbf{z}) \leq K$ for every admissible measured obstacle offset $\mathbf{y}_o$.

2.3.3 Control Lyapunov functions

The control analogue of a Lyapunov-like function encodes the existence of an admissible input enforcing decrease.

Definition 7 (Control Lyapunov function [7, 80]) A smooth $V : X \to \mathbb{R}$ is a *control Lyapunov function* (CLF) for the system (2.1) with target X_0 over a domain D if it is positive definite as in Definition 3 and satisfies, for some $\xi > 0$,

$$\forall \mathbf{x} \in D \setminus X_0,\ \exists \mathbf{u} \in U :\ \nabla V(\mathbf{x}) \cdot f(\mathbf{x}, \mathbf{u}) \leq -\xi.$$

As with CBFs, the existential over $\mathbf{u}$ is what we discretize. A finite control set turns the CLF condition into a finite collection of fixed-input Lyapunov-like conditions, each of which is a convex SOS feasibility problem.

2.4 Inevitable Collision States

The notion of a controlled-invariant set has an instructive dual in the concept of an *inevitable collision state* (ICS) [26]. A state is an ICS if, no matter what control strategy is applied, a collision

with an obstacle will eventually occur. Where a controlled-invariant set guarantees that *some* control strategy keeps the system safe forever, the ICS set characterizes the states from which *no* strategy can; the complement of the ICS set is the maximal controlled-invariant region.

Determining whether a state is an ICS requires reasoning over all possible future trajectories and is, in general, intractable [53]; even approximating the ICS set for a few trajectories of a simple system is computationally intensive. Existing approaches approximate the ICS set with generic checkers, sampling-based planners, or backward reachability. A barrier function offers a complementary, *guaranteed* over-approximation of the ICS set: the complement of the controlled-invariant region $\{B \leq 0\}$ is an over-approximation of the ICS set, and successive barriers (Chapter 3) tighten this over-approximation by shrinking the set of states marked uncontrollable. We make this link between ICS and control barrier functions explicit and exploit it to certify safe planning.

2.5 Sum-of-Squares Programming

All certificates in this dissertation are synthesized by sum-of-squares programming, which provides a tractable, convex relaxation of the nonnegativity conditions that define the certificates.

2.5.1 The SOS cone and Putinar's positivstellensatz

A polynomial $\sigma \in \mathbb{R}[\mathbf{x}]$ is a *sum of squares* (SOS) if $\sigma = \sum_{i=1}^{k} p_i^2$ for some $p_i \in \mathbb{R}[\mathbf{x}]$. Every SOS polynomial is nonnegative on $\mathbb{R}^n$, though the converse fails in general. The set $\mathsf{SOS}_{2d}^n \subset \mathbb{R}_{2d}[\mathbf{x}]$ of SOS polynomials of degree at most $2d$ is a convex cone whose membership is decidable by a semidefinite program: writing $\sigma(\mathbf{x}) = m(\mathbf{x})^\top Q\, m(\mathbf{x})$ for a vector m of monomials of degree at most d, σ is SOS if and only if there is a positive semidefinite Gram matrix $Q \succeq 0$ representing it [64, 65].

Set-constrained inequalities on basic semi-algebraic sets are handled through Putinar's positivstellensatz [39, 70]. Under an Archimedean condition [52] on $\{g_1 \geq 0, \ldots, g_\ell \geq 0\}$, any polynomial that is strictly positive on this set admits a representation

$$\sigma_0 + \sum_{i=1}^{\ell} \sigma_i\, g_i, \qquad \sigma_0, \sigma_1, \ldots, \sigma_\ell \in \mathsf{SOS},$$

with SOS multipliers σ_i. We encode each certificate condition in this form. For example, the barrier conditions of Definition 4 become

$$B - \epsilon = \alpha_0 + \textstyle\sum_i \alpha_i p_i, \qquad -\nabla B \cdot f - \lambda B = \beta_0 + \textstyle\sum_i \beta_i r_i \qquad \alpha_i,\ \beta_i \in \mathsf{SOS},$$

and the Lyapunov-like conditions of Definition 3 likewise become an SOS feasibility problem. Writing the unknown certificate as a linear combination $V = \sum_k c_k \phi_k$ over a fixed monomial basis $\{\phi_k\}$, the coefficients c_k, the level γ, and the multiplier coefficients enter the constraints *linearly* once the control input is fixed, so the problem is a convex semidefinite feasibility problem. After a feasible certificate is found, the largest sublevel set is obtained by maximizing γ [81, 87].

2.5.2 Breaking bilinearity: finite control sets

The linearity exploited above holds only when the control input is fixed. The following theorem, which appears in a tailored form in each technical chapter, justifies replacing the search over all admissible feedbacks by a search over a finite set of fixed inputs, at the cost of at most a factor of two in the decrease rate. It is the structural fact that breaks the bilinearity barrier.

2.5.3 Numerical conditioning and soundness

Two practical caveats accompany SOS synthesis and motivate techniques developed later. First, large or poorly scaled domains make the underlying semidefinite program ill-conditioned [44]; we mitigate this with a *domain-expansion* scheme that begins with a conservative sub-domain and grows it geometrically across levels, certifying handoff into the previously-certified region at each step. Second, the semidefinite program is solved in floating point, so a reported solution may violate the certificate conditions because of numerical error. Chapter 4 addresses this by extracting the Putinar multipliers, recomputing the positivstellensatz residue, and verifying the positive-semidefiniteness of the Gram matrices in high-precision arithmetic, along the lines of the validation framework of Roux et al. [75]. Where greater scalability is needed than full SOS affords,

the diagonally-dominant relaxations DSOS/SDSOS [1] provide a more tractable, if more conservative, alternative. All implementations in this dissertation use the `SumOfSquares.jl`/JuMP toolchain with the MOSEK solver [40].

With these preliminaries in place, the next chapter develops the first instantiation of the successive paradigm, successive control barrier functions for invariance.

Chapter 3

Successive Control Barrier Functions

"You shall not pass!"
— Gandalf, *The Lord of the Rings*

A single control barrier function certifies a controlled-invariant set that is typically a small inner approximation of the maximal one, and searching for a larger monolithic certificate is bilinear. This chapter fixes the control input to a finite set, so each fixed-input barrier is the solution of a convex SOS feasibility problem, and composes the resulting barriers into a successive hierarchy whose controlled-invariant region is the union of the individual ones. A transit-time bound converts the nonsmooth combined certificate into a sampled-data runtime monitor that switches finitely often and guarantees safety over an infinite horizon. On benchmarks from two to seven state variables, the construction certifies substantially larger controllable sets than single barriers or neural alternatives.

3.1 Introduction

This chapter develops the first instantiation of the *successive certificates* paradigm; a hierarchy of control barrier functions for the *safety* specification of Section 2.2.

The overall goal is to address the sources of conservatism already examined in Chapter 1. A single control barrier function (Definition 5) certifies a controlled-invariant set that is typically a small inner approximation of the maximal control invariant region. The bilinear constraints (Section 1.3) make a larger monolithic certificate hard to synthesize. As discussed in Section 1.2, a small certified set forces a safety filter to override the nominal controller more often than the physics require. We attack this on two fronts at once. We fix the control input to a finite set so that, by Theorem 2, each barrier is the solution of a convex SOS feasibility problem. We then combine the resulting fixed-input barriers into a *successive* hierarchy whose controlled-invariant region is the union of the individual ones.

The key idea in the notion of *successive control barrier functions* is to derive these functions by combining simple barrier functions. We first use a very simple approach for synthesizing CBFs by combining barrier functions. Each such barrier function is designed for a fixed control input $\mathbf{u} = \mathbf{u}_i$ and defines a set of states from which the application of the fixed control input $\mathbf{u}_i$ will avoid visiting the unsafe set. We define the notion of *transit time*: the time taken for the barrier function to go from some fixed threshold value inside the control invariant set to the boundary between controlled (safe) and potentially uncontrolled (unsafe) sets wherein the CBF changes sign. We show how to provide a lower bound for transit time and how such a bound can lead to a periodically sampled computation for monitoring the barrier and modifying the nominal controller to enforce safety. However, such CBFs turn out to be quite conservative, in practice: they produce relatively small control invariant sets as a result of this conservatism. In turn, this results in the runtime enforcement intervening more often to modify the nominal controller unnecessarily.

To alleviate this conservatism, we define successive control barrier functions which define a sequence of CBFs $B_0, \ldots, B_k$ wherein a later CBF satisfies its decrease condition only if the earlier

ones are all violated. The key idea here is that when a trajectory violates the CBF B_i, we allow such a violation to happen provided a later B_j for $j > i$ can be used to maintain safety. We demonstrate how to construct such successive CBFs. We then present an approach to use the successive CBFs to maintain control invariance. We use the notion of a transit time to allow for a sampled-data approach which involves periodic sampling using the minimum transit time.

Finally, we demonstrate our approach using a set of small but challenging benchmark examples of nonlinear systems ranging from 2 to 7 state variables. We show how the use of successive barrier functions can help add to the control invariant set. We compare our result to a recent approach that uses neural networks as CBFs and has been implemented in the tool FOSSIL [23]. The comparison with FOSSIL demonstrates that our approach, although slower on small examples, can still scale to larger dimensional benchmarks and can obtain control invariant sets that add more controllable states when compared to those included in the CBFs discovered by FOSSIL.

Contributions of this chapter.

- A method to combine *fixed-input* barrier functions, synthesized by SOS programming, into a single (nonsmooth) control barrier function;
- A *transit-time* analysis yielding a lower bound on the time a trajectory needs to cross the safety margin, enabling a chattering-free sampled-data monitor;
- The notion of *successive* barrier functions and an iterative synthesis procedure that progressively enlarges the controlled-invariant set;
- A runtime safety shield over the hierarchy with a finite-switch, Zeno-free guarantee; and
- An empirical study on benchmarks from two to seven state variables, with a comparison against neural certificates.

3.1.1 Related Work

Although CBFs are being widely used to enforce safety properties, computing a CBF for a given nonlinear dynamics is quite challenging. A significant challenge in computing control barrier functions arises due to the relative degree of the function. Using exponential control barrier functions addresses these challenges [60], and higher-order control barrier functions generalize these approaches further [82, 96]. The higher-order approach keeps differentiating a high relative degree constraint function until a control input is found. The CBF conditions are then enforced over the highest-order derivative. However, in practice, many dynamical systems may not obey the relative degree condition. Also, since the approach often starts from the specification of the safety property, it can lead to infeasibility during runtime wherein we may fail to find a control input that can be applied to maintain safety.

Our approach relies on computing multiple barrier functions. Computing more than one barrier function has recently become a prominent approach for safe planning in challenging environments. The need for multiple barriers primarily arose to counter scenarios where a single barrier function may not be sufficient. Many of these approaches consider environments with multiple obstacles/constraints by synthesizing multiple barriers, wherein each of these barriers enforcing a subset of constraints [29, 43]. One way of computing these viability domains is applying all barriers in a single quadratic program [71, 97], while some approaches decouple the design to ensure joint feasibility under more conservative assumptions [16]. Our approach considers a single unsafe set X_u unlike the other approaches mentioned above. The motivation for considering multiple CBFs lies in improving the control invariant set for a single unsafe set specification.

When dealing with multiple CBFs, a valid concern is how often one needs to switch between barriers. A continuous time monitoring strategy may lead to Zeno behavior. There have been attempts to alleviate this problem by incorporating dwell time constraints when synthesizing barriers [17, 18], where the controllers can only intervene once a minimum dwell time has elapsed. The dwell times are present to compute impulsive timed CBFs. Our work uses a different approach

that consists of computing minimum transit times using a notion introduced in this paper, that we call the "slow transit property". The transit property allows our approach to effectively work in a sampled-data manner; i.e., by periodically sampling the state of the system and making a decision on how to modify the nominal control input for the subsequent time interval. Our approach is set up so as to guarantee that the system cannot transit from safe to unsafe within this time interval.

The concept of a controlled invariant set is dual to another interesting concept of Inevitable Collision States (ICS). The ICS concept was first formulated in [26]. A state is an ICS where, no matter what trajectory is executed by the system, a collision with an obstacle will eventually occur [54]. Whereas, the controlled invariant set guarantees that there always exists a control strategy that will ensure collisions with obstacles are not possible. Determining whether a state is ICS for all possible trajectories is intractable [53]. Even for a simple dynamical system, computing ICS for only a few trajectories is computationally intensive [66].

There have been few notable attempts to approximate ICS. One of the earliest attempts was a generic ICS checker implemented in [54, 55]. Bekris et al. [10, 11] introduced sampling-based planners for avoiding ICS efficiently. Another approach was to employ backward reachability for ICS approximation [66]. Lim et al. [42] use precomputed offsets for safe navigation for fixed-wing aerial vehicles. In this paper, we present a new guaranteed over-approximation technique that uses sum of squares optimization and exploits the link between ICS and control barrier functions.

3.2 Successive Control Barrier Functions

In this section, we describe the overall successive control barrier function approach of this paper.

(a) First, we show that fixing a finite set of control inputs can be used to synthesize barrier functions that can be combined into a control barrier function. Section 3.2.1 describes this approach.

(b) Unfortunately, our approach produces a non-smooth control barrier function which requires

theoretical ideas from non-smooth analysis and in practice can lead to zenoness/chattering. We provide an approach for analyzing "transit times" that will allow us to work with discrete time monitors.

(c) The combination of using SOS programming and enforcing a transit time condition make the resulting control barrier functions too *conservative*; i.e., their controlled invariant region marks off many states as "uncontrollable". We will address this by presenting the idea of successive control barriers.

3.2.1 From Barriers to Control Barriers

We recall from Section 2.3.2 the (control) barrier function of Definitions 4 and 5, and the convention that the controlled-invariant region is the sublevel set $\{\mathbf{x} : B(\mathbf{x}) \leq 0\}$ while $B \geq \epsilon$ on the unsafe set $\mathcal{O}$.

Theorem 1 Let B be a barrier function following Def. 4 and $\mathbf{x}_0 \in \mathbb{R}^n$ be such that $B(\mathbf{x}_0) \leq 0$. Any trajectory $\varphi : [0, T) \to \mathbb{R}^n$ with $\varphi(0) = \mathbf{x}_0$ will not intersect the unsafe set X_u.

Proof. Let $\varphi : [0, T) \to \mathbb{R}^n$ be any trajectory. Since f is continuous, it follows that φ is differentiable. Consider the function $e^{\lambda t}B(\varphi(t))$. For all $t \in [0, T)$, $\frac{d}{dt}\left(e^{\lambda t}B(\varphi(t))\right) = \lambda e^{\lambda t}B(\varphi(t)) + e^{\lambda t}\frac{d}{dt}B \leq \lambda e^{\lambda t}B(\varphi(t)) - \lambda e^{\lambda t}B(\varphi(t)) \leq 0$. Therefore, $e^{\lambda t}B(\varphi(t)) \leq B(\varphi(0))$. Thus, for all $t \geq 0$, $B(\varphi(t)) \leq 0$ if $B(\varphi(0)) \leq 0$. Therefore, $\varphi(t) \notin X_u$ since $B(\mathbf{x}) \geq \epsilon > 0$ for all $\mathbf{x} \in X_u$. □

Let $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ be the dynamical model with a compact set of control inputs $\mathbf{u} \in U$ and X_u be the set of unsafe states. Let U_{fin} be a finite subset of U. Our control barrier function will consider controls only from this set U_{fin}.

Remark 1 If the dynamics $f(\mathbf{x}, \mathbf{u})$ are control affine; i.e., $f(\mathbf{x}, \mathbf{u}) = f_1(\mathbf{x}) + f_2(\mathbf{x})\mathbf{u}$ and U is represented by a (compact) convex polyhedron, then U_{fin} can be taken to be the vertices of U. This is because whenever for a state $\mathbf{x}$ there exists a $\mathbf{u} \in U$, such that $(\nabla B(\mathbf{x})) \cdot (f_1(\mathbf{x}) + f_2(\mathbf{x})\mathbf{u}) \leq -\lambda B(\mathbf{x})$, we can show that one of the vertices $\mathbf{u} \in U_{\text{fin}}$ will satisfy the decrease condition since it is affine in $\mathbf{u}$.

Theorem 2 (Finite-control-set sufficiency) Let $V \in C^1$ and let $\kappa : X \to U$ be a continuous feedback satisfying $\nabla V(\mathbf{x}) \cdot f(\mathbf{x}, \kappa(\mathbf{x})) \leq -\xi$ for all $\mathbf{x}$ in a compact region. Then there exists a finite set $U_{\text{fin}} \subset U$ such that for every such $\mathbf{x}$ there is $\bar{\mathbf{u}} \in U_{\text{fin}}$ with $\nabla V(\mathbf{x}) \cdot f(\mathbf{x}, \bar{\mathbf{u}}) \leq -\xi/2$. If the dynamics are control affine and U is a polytope, one may take $U_{\text{fin}} = \text{vert}(U)$.

Proof. By compactness and C^1-regularity of V and f, the map $\mathbf{u} \mapsto \nabla V(\mathbf{x}) \cdot f(\mathbf{x}, \mathbf{u})$ is Lipschitz in $\mathbf{u}$ uniformly in $\mathbf{x}$, with some constant $L > 0$. Take U_{fin} to be a $\xi/(2L)$-net of U (which exists since U is totally bounded). For each $\mathbf{x}$, choose $\bar{\mathbf{u}}$ within $\xi/(2L)$ of $\kappa(\mathbf{x})$; then $\nabla V \cdot f(\mathbf{x}, \bar{\mathbf{u}}) \leq \nabla V \cdot f(\mathbf{x}, \kappa(\mathbf{x})) + L \cdot \xi/(2L) \leq -\xi + \xi/2 = -\xi/2$. For control-affine dynamics, $\nabla V \cdot f$ is affine in $\mathbf{u}$, so its minimum over the polytope U is attained at a vertex, and the finite vertex set suffices. □

With $\mathbf{u}$ restricted to U_{fin}, each level of the successive hierarchy is a convex SOS feasibility problem; the apparent loss of generality is bounded by Theorem 2, and in practice a handful to a few dozen candidate inputs suffice. In the control-affine case the finite set is exactly the vertices of the input polytope.

Example 5 Consider the example dynamics (Ex. 1) wherein we fix the external inputs $u_1 = 0.05$. Let $X_u = \{(x, y) | x^2 + y^2 \leq 0.04\}$. By fixing a maximum degree of 4 for the barrier function, we obtain

$$B_1 = \begin{pmatrix} 10 - 10\theta + 1.95y - 5.5x + 10\theta^2 - 10y\theta + 1.7y^2 + 1.3x\theta - 2.4xy + x^2 \\ + \cdots + \\ 0.05x^2\theta^2 + 0.075x^2y\theta + 0.0024x^2y^2 - 0.017x^3\theta - 0.0027x^3y + 0.001x^4 \end{pmatrix}.$$

This barrier function and the function B_0 was synthesized using the `SumOfSquares.jl` package in Julia [40, 94].

Algorithm 1 computes a set of barrier functions and associated controls of the form $\mathcal{B} = \{(B_1, \mathbf{u}_1), \ldots, (B_k, \mathbf{u}_k)\}$, wherein each B_i is a barrier function for the dynamics $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}_i)$ with

Algorithm 1: Algorithm for multiple control barrier computation

Data: System Σ, unsafe set X_u, finite control set U_{fin}, constants $\epsilon > 0$ and λ, degree bound D, test set S.

Result: Set of barrier functions and associated controls $\mathcal{B}$.

1 $\mathcal{B} = \emptyset$ `/* Initialize to empty set */`
2 **for** $\mathbf{u}_t \in U_{fin}$ **do**
3 **for** $\mathbf{x}_t \in S$ **do**
4 find polynomial B of degree D
5 s.t. $(\forall \mathbf{x} \in X_u)\ B(\mathbf{x}) \geq \epsilon$
6 $(\forall \mathbf{x})\ \nabla B \cdot (f(\mathbf{x}, \mathbf{u}_t)) \leq -\lambda B$
7 $B(\mathbf{x}_t) \leq 0$
8 **if** B *found* **then**
9 $\mathcal{B} = \mathcal{B} \cup \{(B, \mathbf{u}_t)\}$
10 $S = S \setminus \{\mathbf{x} \in S \mid B(\mathbf{x}) \leq 0\}$ `/* remove already eliminated test point */`
11 **return** $\mathcal{B}$

$\mathbf{u} = \mathbf{u}_i$. To facilitate multiple barrier computation, we fix a finite set S of "test" states such that $S \cap X_u = \emptyset$. Our goal is to attempt to find a barrier for each test state $\mathbf{x}_t \in S$ that shows that it belongs to the control invariant region when the control is fixed to some $\mathbf{u}_i \in U_{\text{fin}}$. Lines 5- 8 present the SOS formulation. In particular, line 8 adds a linear constraint in the coefficients of the unknown polynomial B that forces the polynomial to be non-positive at the test point $\mathbf{x}_t$. If a barrier is found for a test point $\mathbf{x}_t$ and some control $\mathbf{u}_j \in U_{\text{fin}}$, the test point is eliminated from the set S. In practice, S consists of randomly generated states that lie outside X_u.

We wish to combine the set of barrier functions obtained from Algorithm 1 into a single *control barrier function*. For instance, we can consider the piecewise minimum: $B(\mathbf{x}) = \min_{(B_i, \mathbf{u}_i) \in \mathcal{B}} B_i(\mathbf{x})$, as the minimum over all the barriers considered. However, such a function is *non-smooth*. Although the theory of nonsmooth barrier functions [28, 29] have been studied using techniques from nonsmooth analysis [22], the results of Glotfelter et al. are not directly applicable in our case, where the barrier functions $B_1, \ldots, B_m$ are obtained by altering the control inputs and thus technically, *pertain to different dynamics*. We now study a simpler but conservative scheme that imposes an additional transit time condition; imposing the transit time condition on the barriers computed using Algorithm 1 allows us to effectively combine them into a single control barrier function and enforce control invariance.

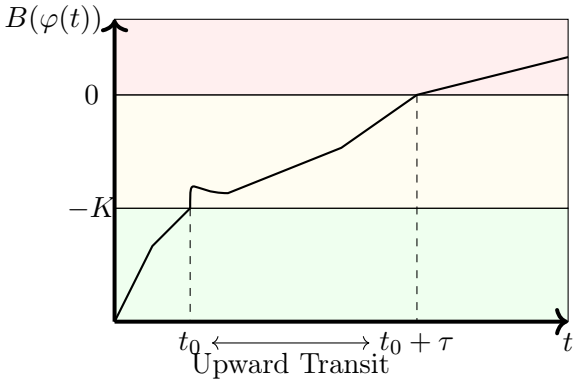


Figure 3.1: Illustration of a trajectory transiting from $B(\varphi(t_0)) \leq -K$ to $B(\varphi(t_0 + \tau)) \geq 0$.

3.2.2 Transit Time Bounds for a Barrier Function

Let $B(\mathbf{x})$ be a barrier function for a fixed control input $\mathbf{u}_r \in U$. The set $\{\mathbf{x} \mid B(\mathbf{x}) \leq 0\}$ is the controlled invariant (CI) corresponding to $B(\mathbf{x})$ using the control $\mathbf{u} = \mathbf{u}_r$. Let $K > 0$ be a fixed threshold. Let $\mathbf{u}(t) \in U$ be an arbitrary control signal. Let $\varphi(t)$ denote the resulting trajectory. We say that φ "transits" from a state $\varphi(t_0)$ wherein $B(\varphi(t_0)) = -K$ to a state $B(\varphi(t_0 + \tau)) = 0$ for transit time $\tau > 0$ if for all time $t \in (t_0, t_0 + \tau)$, $B(\varphi(t)) \in (-K, 0)$ (See Fig. 3.1). Since we have assumed that the dynamics are time invariant, we can set $t_0 = 0$ without loss of generality.

Definition 8 (Minimum "Upward" Transit Time) The minimum (upward) transit time given a barrier function $B(\mathbf{x})$ and threshold $K > 0$ is defined as the infimum over the transit times of all possible trajectories $\varphi : [0, T) \to X$ of the system $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ corresponding to a continuous control signal $\mathbf{u} : [0, T) \to U$.

$$\tau^* = \inf \left\{ \tau \;\middle|\; \begin{array}{c} \varphi \text{ is a trajectory for continuous } \mathbf{u} : [0, T) \to U, \\ 0 < \tau \leq T,\ B(\varphi(0)) = -K, B(\varphi(\tau)) = 0, \text{and} \\ \forall t \in (0, \tau)\ B(\varphi(t)) \in (-K, 0) \end{array} \right\}.$$

We will now augment the requirements for a barrier function to be able to provide a bound on its transit time given a threshold K.

Definition 9 (Upward Slow Transit Property) We say that a control barrier function has the upward *slow transit property* (STP) iff there exist $\delta, \eta \in \mathbb{R}$ such that

$$\forall\ \mathbf{x} \in X, \forall \mathbf{u} \in U,\ B(\mathbf{x}) \in [-K, 0]\ \Rightarrow\ \nabla B \cdot f(\mathbf{x}, \mathbf{u}) \leq \eta B(\mathbf{x}) + \delta\,. \tag{3.1}$$

Theorem 3 If a barrier function $B(\mathbf{x})$ satisfies (3.1) then (a) if $\max(\delta, -\eta K + \delta) > 0$, then $\tau^* \geq \frac{K}{\max(\delta, -\eta K + \delta)}$; and (b) if $\max(\delta, -\eta K + \delta) \leq 0$, then no transits are possible: i.e., $\tau^* = \infty$.

Proof. Let $\varphi : [0, T) \to X$ be any trajectory corresponding to some continuous control signal $\mathbf{u} : [0, T) \to U$ and $\tau > 0$ be a time such that the conditions for transit hold: $B(\varphi(0)) = -K, B(\varphi(\tau)) = 0,$ and $\forall t \in (0, \tau)\ -K \leq B(\varphi(t)) \leq 0$. Since $\mathbf{u}$ is continuous, φ is continuous and differentiable. Applying the Lagrange mean value theorem, we obtain,

$$B(\varphi(\tau)) = B(\varphi(0)) + \tau \frac{d}{dt} B(\varphi(t))\,, \text{where } t \in (0, \tau)\,.$$

Next, we observe that $\frac{d}{dt} B(\varphi(t)) = \nabla B(\varphi(t)) \cdot f(\varphi(t)) \leq \eta B(\varphi(t)) + \delta$, since $B(\varphi(t)) \in [-K, 0]$. We have

$$0 = B(\varphi(\tau)) = B(\varphi(0)) + \tau \frac{d}{dt} B(\varphi(t)) \leq -K + \tau(\delta + \eta B(\varphi(t)))\,.$$

Since $B(\varphi(t)) \in [-K, 0]$, $\delta + \eta B(\varphi(t)) \leq \begin{cases} \delta & \eta \geq 0 \\ \delta - \eta K & \eta < 0 \end{cases}$.

Therefore, $\delta + \eta B(\varphi(t)) \leq \max(\delta, \delta - \eta K)$. Suppose $\max(\delta, \delta - \eta K) > 0$, then $\tau^* \geq \frac{K}{\max(\delta, -\eta K + \delta)}$. This proves (a).

Otherwise, if $\max(\delta, \delta - \eta K) \leq 0$, we have $\delta - \eta K \leq 0$ and $\delta \leq 0$. As a result, we conclude that whenever $B(\mathbf{x}) \in [-K, 0]$, $\frac{d}{dt} B(\mathbf{x}) = \nabla B(\mathbf{x}) \cdot f(\mathbf{x}, \mathbf{u}) \leq \eta B(\mathbf{x}) + \delta \leq 0$. Therefore, applying mean value theorem, $B(\varphi(\tau)) = B(\varphi(0)) + \tau \frac{d}{dt} B(\varphi(t)) \leq B(\varphi(0)) \leq -K$. Hence, no transit is possible. □

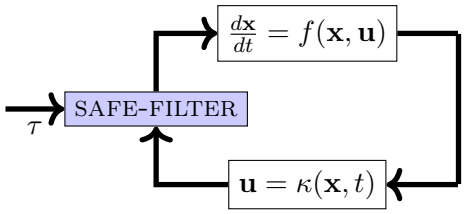


Figure 3.2: Safety enforcement for multiple barrier functions through a safety filter.

Example 6 Analyzing the barrier function in our running example 5 using SOS programming while fixing $K = 1$ yields a bound on the minimum transit time of $\tau^* \geq 1.86$.

3.2.3 Safety Enforcement using Multiple Barriers

Let $B_1, \ldots, B_m$ be barrier functions for unsafe set X_u obtained by fixing the control inputs to $\mathbf{u}_1, \ldots, \mathbf{u}_m$, respectively. We assume that all B_i satisfy the upward STP (Def. 9). This is ensured by modifying Algorithm 1 to test that each barrier function discovered by the algorithm satisfies (3.1) and discarding ones that do not. Let $\tau_1, \ldots, \tau_m$ be lower bounds on the transit time for these barriers using the approach described in the previous section for a fixed threshold $K > 0$. Let $\tau = \min(\tau_1, \ldots, \tau_m)$. Let $\mathbf{u} = \kappa(\mathbf{x}, t) \in U$ be a nominal control law that is continuous in $\mathbf{x}$ and t.[1]

Figure 3.2 shows the setup of the safety filter corresponding to multiple barriers $B_1, \ldots, B_m$. The SAFE-FILTER module is invoked every τ time units and can issue one of two decrees: (a) PASS: allow the feedback law κ to operate for the next τ time units; or (b) OVERRIDE($\mathbf{u}_j$): keep a constant control input $\mathbf{u} = \mathbf{u}_j$ for the next τ time units.

Let $B_{min}(\mathbf{x}) = \min(B_1(\mathbf{x}), \ldots, B_m(\mathbf{x}))$ be the minimum over all the barrier functions. We will let $\mathsf{ind}(\mathbf{x}) = \min\{j \mid B_j(\mathbf{x}) = B_{min}(\mathbf{x})\}$ be the least index j for which $B_j(\mathbf{x}) = B_{min}(\mathbf{x})$ and $\mathbf{u}_{min}(\mathbf{x}) = \mathbf{u}_{\mathsf{ind}(\mathbf{x})}$ be the control input corresponding to the minimum index. The safety filter is

[1]The dependence on time allows κ to be a feed-forward control input: $\mathbf{u} = \kappa(t)$.

defined as follows:

$$\textsc{safe-filter}(\mathbf{x}; B_{min}) = \begin{cases} \textsc{Pass} & \text{if } B_{min}(\mathbf{x}) < -K \\ \textsc{Override}(\mathbf{u}_{min}(\mathbf{x})) & \text{otherwise} \end{cases}$$

In other words, the safety filter allows the feedback κ to be applied for the next τ time units if at least one of the functions $B_1, \ldots, B_m$ is below the threshold $-K$. Otherwise, it applies the control $\mathbf{u}_j$ corresponding to that barrier function B_j whose value is minimal among $B_1, \ldots, B_m$.

Let $\varphi : [0, T) \to X$ be a trajectory of the closed loop system with the safety filter applied (see Fig. 3.2). If $B_{min}(\varphi(0)) < 0$ then $B_{min}(\varphi(t)) < 0$ for all $t \in [0, T)$. We will assume that the safety filter is evaluated at times $t = 0, \tau, 2\tau, \ldots$. Furthermore, we assume that the execution of κ and the safety filter are instantaneous. First, we establish a lemma for the safety filter. Let $\mathbf{x}_k = \varphi(k\tau)$ for $k \in \mathbb{N}$.

Lemma 1 If $B_{min}(\mathbf{x}_k) \in [-K, 0)$, then applying the overriding feedback $\mathbf{u}_j$ provided by $\textsc{safe-filter}(\varphi(k\tau); B_{min})$ ensures that $B_{min}(\varphi(t)) < 0$, for all $t \in [k\tau, (k+1)\tau]$.

Proof. Let $j = \mathsf{ind}(\mathbf{x}_k)$. Applying the control $\mathbf{u}_j$ ensures that for $t \in [k\tau, (k+1)\tau]$, we have $\frac{d}{dt} B_j(\varphi(t))|_{\mathbf{u}=\mathbf{u}_j} \leq -\lambda B_j(\varphi(t))$. Since, $B_{min}(\mathbf{x}_k) = B_j(\mathbf{x}_k) < 0$. Applying theorem 1, yields $B_{min}(\varphi(t)) \leq B_j(\varphi(t)) < 0$ for $t \in [k\tau, (k+1)\tau]$. □

Lemma 2 If $B_{min}(\mathbf{x}_k) < -K$ then $B_{min}(\varphi(t)) < 0 \ \forall \ t \in [k\tau, (k+1)\tau]$.

Proof. Suppose we have $B_{min}(\varphi(t)) \geq 0$ for some $t \in [k\tau, (k+1)\tau]$, by the continuity of $B(\varphi(t))$, there exist time intervals t_1, t_2 such that (a) $B(\varphi(t_1)) = -K$, (b) $B(\varphi(t_2)) = 0$ and $B(\varphi(s)) \in (-K, 0)$ for $s \in (t_1, t_2)$. The trajectory transits during the time interval $[t_1, t_2]$. By definition of τ, we have $\tau \leq t_2 - t_1$. Therefore, t_1, t_2 cannot both be in the time interval $[k\tau, (k+1)\tau]$. This yields a contradiction. Thus $B_{min}(\varphi(t)) < 0$ for all $t \in [k\tau, (k+1)\tau]$. □

Theorem 4 Let $\varphi : [0, T) \to X$ be any closed-loop trajectory with the safety filter applied at times $t_k = k\tau$ for $k \in \mathbb{N}$. If $B_{min}(\varphi(0)) < 0$ then $B_{min}(\varphi(t)) < 0$ for all times $t \in [0, T)$.

Proof. The proof follows directly from Lemmas 1 and 2. □

Thus, the time-triggered application of the SAFE-FILTER rule ensures that the set $\{\mathbf{x} \in X \mid B_1(\mathbf{x}) \leq 0 \text{ or } \cdots \text{or } B_m(\mathbf{x}) \leq 0\}$ can be maintained as a controlled invariant set. Our goal, however, is to find as large a control invariant set as possible so that we can apply the nominal controller without overriding too often. We will present the idea of a successive barrier function to add more states that can be added to the control invariant region.

3.2.4 Successive Barriers

Suppose we have synthesized multiple barrier functions $B_1, \ldots, B_m$ with associated controls $\mathbf{u}_1, \ldots, \mathbf{u}_k$. We can establish control invariance using the function $B^{(0)}(\mathbf{x}) = \min_{j=1}^{m} B_j(\mathbf{x})$ for states $\mathbf{x}$ wherein $B^{(0)}(\mathbf{x}) \leq 0$, using the upward slow transit property. We will focus on synthesizing *successive* barrier functions that "work" only when $B^{(0)}(\mathbf{x}) \geq 0$. The goal is that these functions together add to the set of states from which we can apply controls to successfully avoid reaching the unsafe states. Let $U_{\text{fin}} = \{\mathbf{u}_1, \ldots, \mathbf{u}_N\} \subseteq U$ be a fixed finite set of control inputs.

Definition 10 A function $B_i(\mathbf{x})$ is a successive barrier function w.r.t a previously established multiple control barrier function $B^{(0)}(\mathbf{x})$ for a fixed control input $\mathbf{u}_i \in U_{fin}$ iff

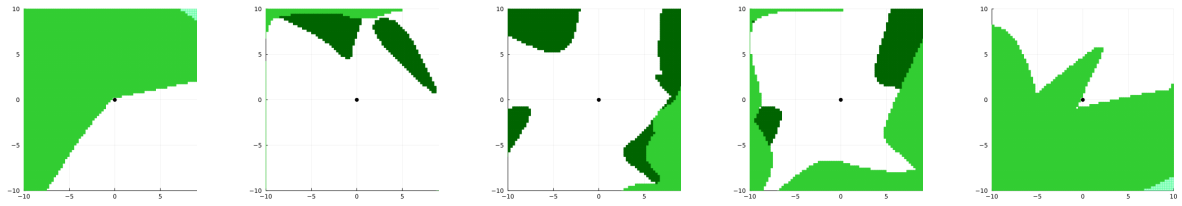
(1) $B_i(\mathbf{x}) \geq \epsilon$ for all $\mathbf{x} \in X_u$,

(2) for all $\mathbf{x}$ such that $B^{(0)}(\mathbf{x}) \geq 0$, $\nabla B_i \cdot f(\mathbf{x}, \mathbf{u}_i) \leq -\lambda B_i(\mathbf{x})$. In other words, B_i satisfies the derivative condition only for those states wherein $B^{(0)}(\mathbf{x}) \geq 0$.

(3) B_i has its upward transit time bounded from below by τ_i.

Let $B_1(\mathbf{x}), \ldots, B_M(\mathbf{x})$ be successive barrier functions for control inputs $\mathbf{u}_1, \ldots, \mathbf{u}_M$, respectively and previously established control barrier function $B^{(0)}(\mathbf{x})$. Let $B^{(1)}(\mathbf{x}) = \min(B_1(\mathbf{x}), \ldots, B_M(\mathbf{x}))$ and τ be the minimum transit time among all barriers that have been synthesized thus far.

Note that the concept of successive barriers can be iterated. Let $B^{(0)}$ be our initial control barrier function. We can compute a successive barrier $B^{(1)}$ which satisfies the derivative conditions



$\theta = -\pi$ $\theta = -\frac{\pi}{2}$ $\theta = 0$ $\theta = \frac{\pi}{2}$ $\theta = \pi$

Figure 3.3: Successive barrier functions extend the overall control invariant region. The earlier approximations of the control invariant set are shown in light green. The minimum transit time was computed to be 0.02.

only if $B^{(0)}(\mathbf{x}) \geq 0$. In turn, we can use this to derive another function $B^{(2)}$ as the minimum over multiple polynomials each of whose derivative conditions holds only if $B^{(0)}(\mathbf{x}) \geq 0$ and $B^{(1)}(\mathbf{x}) \geq 0$ and so on. This yields a sequence of successive barrier functions. Figure 3.3 shows how successive barriers can add to the set of control invariant states.

We will now outline a scheme to synthesize successive barrier functions iteratively. Algorithm 2 shows the overall algorithm. The approach maintains a finite set S of sample *test states* and a family of sets of successive barrier functions: $\mathcal{B} = \langle B^{(0)}, B^{(1)}, \ldots, B^{(k)} \rangle$, wherein each $B^{(i)}$ is of the form $\min(B_{i,1}, \ldots, B_{i,j})$ for some polynomial functions $B_{i,1}, \ldots, B_{i,j}$. Note that the constraint $B^{(i)} \geq 0$ is equivalent to $\bigwedge_{k=1}^{j} B_{i,k} \geq 0$. We initialize $\mathcal{B}$ to be the "ancestors" (set of previous successive barriers) and the test states to be a randomly sampled set of some fixed size N.

We iterate over the control inputs $\mathbf{u}_i \in U_{fin}$ and test states $\mathbf{x}_t \in S$. We attempt to discover a barrier function B of degree at most D such that (a) $\forall \mathbf{x} \in X_u, B(\mathbf{x}) \geq \epsilon$; (b) the derivative condition; $\bigwedge_{l=1}^{k} B^{(l)} \geq 0 \;\Rightarrow\; \nabla B \cdot f(\mathbf{x}, \mathbf{u}) \leq -\lambda B$, and finally (c) we insist that $B(\mathbf{x}_t) \leq 0$, i.e., the barrier ensures safety at the test point $\mathbf{x}_t$. We use sum of squares (SOS) programming to find such a barrier function (Lines 3-7). If it succeeds, then we add the new barrier to our list (Line 9), eliminating $\mathbf{x}_t$ and other test points already excluded from the unsafe region (Line 10).

Algorithm 2: Algorithm for successive barrier computation

Data: System Σ, unsafe set X_u, finite control set U_{fin}, previous successive barriers $\mathcal{B} = \langle B^{(0)}, B^{(1)}, \ldots, B^{(k)} \rangle$, constants $\epsilon > 0$, η, δ and λ, degree bound D, test set S.

Result: $k+1$ level successive barrier function $B^{(k+1)}$.

$B^{(k+1)} \leftarrow \infty$ `/* Initialize to trivial function */`

for $\mathbf{u} \in U_{fin}$ **do**

  for $\mathbf{x}_t \in S$ **do**

    find polynomial B of degree D

      s.t. $\mathbf{x} \in X_u \Rightarrow B(\mathbf{x}) \geq \epsilon$

    $\bigwedge_{l=1}^{k} B^{(l)} \geq 0 \Rightarrow \nabla B \cdot f(\mathbf{x}, \mathbf{u}) \leq -\lambda B$

      $B(\mathbf{x}_t) \leq 0$

    if *B found and B has bounded transit time τ* **then**

      $B^{(k+1)} = \min(B^{(k+1)}, B)$

      $S = S \setminus \{\mathbf{x} \in S \mid B(\mathbf{x}) \leq 0\}$ `/* remove already eliminated test points from` S `*/`

return $B^{(k+1)}$

3.2.5 Barriers as ICS Overapproximations

Recall the notion of an inevitable collision state (ICS) from Section 2.4: a state from which no control strategy can avoid an eventual collision. The complement of a controlled-invariant region $\{\mathbf{x} : B(\mathbf{x}) \leq 0\}$ is a guaranteed over-approximation of the ICS set, and each successive barrier, by reclaiming states previously marked uncontrollable, *tightens* this over-approximation.

3.3 Runtime Safety using Successive Barriers

Suppose we have synthesized a sequence of successive barrier functions $B^{(0)}, B^{(1)}, \ldots, B^{(k)}$, wherein each $B^{(i)} = \min(B_{i,1}, \ldots, B_{i,n(i)})$ is obtained as a combination of multiple barrier functions with associated inputs $\mathbf{u}_{i,1}, \ldots, \mathbf{u}_{i,n(i)}$. Let $\tau_1, \ldots, \tau_k > 0$ be bounds on the upward transit times for $B^{(1)}, \ldots, B^{(k)}$, respectively. We define $\tau = \min(\tau_1, \ldots, \tau_k)$. Our goal is to combine the safety monitors for each of the individual control barriers, as explained in Section 3.2.3 to yield a composite safety monitor for the overall sequence of successive barrier functions.

Let $\mathbf{x}$ be the state at any time t. We say that the control barrier function $B^{(i)}$ is *green* if $B^{(i)} < -K$, *yellow* if $B^{(i)} \in [-K, 0)$ and *red* if $B^{(i)} \geq 0$. Note that if the control barrier is green or yellow, then the state $\mathbf{x}$ cannot be in the unsafe set X_u by construction. A color indicator function for a given state $\mathbf{x}$ and control barrier $B^{(i)}$ can be defined as follows:

$$\mathcal{C}_i(\mathbf{x}) = \begin{cases} G & \text{if } B^{(i)}(\mathbf{x}) < -K \\ Y & \text{if } -K \leq B^{(i)}(\mathbf{x}) < 0 \\ R & \text{otherwise} \end{cases}$$

The concept is illustrated in Figure 3.4. Now we can synthesize the runtime safety shield for the successive barriers. Assume that a nominal control law $\mathbf{u} = \kappa(t, \mathbf{x})$ is used to decide on the control input for a given state $\mathbf{x}$. For each of the barriers $B^{(i)}$, we will run the safety filter defined in section 3.2.3 using τ as the time period. If $\mathcal{C}_i(\mathbf{x}) = G$, then the judgement is PASS, whereas if $\mathcal{C}_i(\mathbf{x}) = Y$, the judgement is OVERRIDE($\mathbf{u}_i$) for some $\mathbf{u}_i \in U_{\text{fin}}$. Let $\varphi(t)$ be some trajectory and

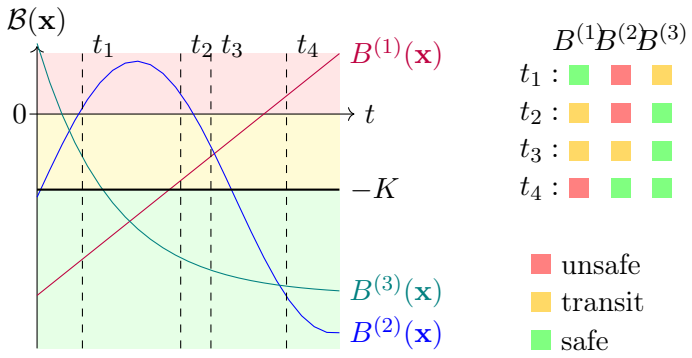


Figure 3.4: Barrier function evolution and the corresponding states of the runtime monitor.

$\mathbf{x}_l = \varphi(l\tau)$ for some natural number l. We will now define the safety filter function.

Definition 11 Given successive barrier functions $B^{(1)}, \ldots, B^{(k)}$, the function SAFE-FILTER$(\mathbf{x}_l; B^{(1)}, \ldots, B^{(k)})$ operates as follows:

rule-1: Suppose there is an index $j \in [1, k]$ such that $\mathcal{C}_j(\mathbf{x}_l) = G$, then SAFE-FILTER$(\mathbf{x}_l; B^{(1)}, \ldots, B^{(k)})$ is set to PASS, or equivalently, allows the nominal control law to be applied for the next τ time units.

rule-2: Otherwise, let $j \in [1, k]$ be the least yellow index: i.e., (a) $\mathcal{C}_j(\mathbf{x}_l) = Y$, (b) for all $i < j$, $\mathcal{C}_i(\mathbf{x}_l) = R$ and (c) for all $i > j$, $\mathcal{C}_i(\mathbf{x}_l) \in \{R, Y\}$. We define SAFE-FILTER$(\mathbf{x}_l; B^{(1)}, \ldots, B^{(k)}) =$ SAFE-FILTER$(\mathbf{x}_l; B^{(j)}) =$ OVERRIDE$(\mathbf{u}_j)$, for some $\mathbf{u}_j \in U_{\text{fin}}$.

rule-3: Otherwise, all of the control barriers are in the red state. This state should never be reached since the trajectory may be potentially unsafe. SAFE-FILTER is undefined for this case.

Let $\varphi(t)$ be a trajectory obtained by composing nominal controller $\mathbf{u} = \kappa(t, \mathbf{x})$ and the SAFETY-FILTER$(\mathbf{x}; B^{(1)}, \ldots, B^{(k)})$ executed with time period τ. We will first prove that if we are in a state $\mathbf{x}_l$ at time $t = l\tau$ such that $\mathcal{C}_j(\mathbf{x}_l) \in \{G, Y\}$ for at least one index $j \in [1, k]$ (i.e., not all control barriers are red) then $\mathcal{C}_i(\mathbf{x}_{l+1}) \in \{G, Y\}$ for some index i.

Lemma 3 Let $j \in [1, k]$ such that $\mathcal{C}_j(\mathbf{x}_l) = G$. It follows that $\mathcal{C}_j(\varphi(t)) \in \{G, Y\}$ for all $t \in [l\tau, (l+1)\tau]$.

Proof. This follows directly from Lemma 2. □

Lemma 4 If $\mathcal{C}_j(\mathbf{x}_l) = Y$ and $\mathcal{C}_i(\mathbf{x}_l) = R$ for all $i \in [1, j)$ then for all $t \in [l\tau, (l+1)\tau]$, there exists $i \leq j$ such that $\mathcal{C}_i(\varphi(t)) \in \{G, Y\}$.

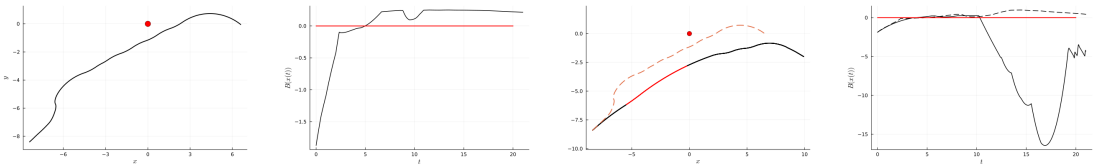
Proof. Consider two cases: (**case-1**) all lower indices remain "red" or "yellow" for the entire interval, i.e., $\forall\ t \in [l\tau, (l+1)\tau]$ and for all $1 \leq i < j$, we have $\mathcal{C}_i(\varphi(t)) \in \{R, Y\}$, or (**case-2**) there exists a time instant $t \in [l\tau, (l+1)\tau]$ such that $\mathcal{C}_i(\varphi(t)) = G$ for some $1 \leq i < j$. Note that if $j = 1$, then **case-1** must hold trivially since there are no lower indices.

(**case-1**:) Let SAFETY-FILTER$(\mathbf{x}_l, B^{(j)})$ = OVERRIDE$(\mathbf{u}_j)$ for $\mathbf{u}_j \in U_{\text{fin}}$. Suppose **case-1** holds, then we have that for some barrier $B_{i,j}$ satisfies the derivative condition $\frac{dB}{dt} \leq -\lambda B$ with control inputs fixed to $\mathbf{u}_j$ throughout the time interval $[l\tau, (l+1)\tau]$. Therefore, since $B_{i,j}(\mathbf{x}_l) < 0$, we have $B_{i,j}(\varphi(l\tau + t)) \leq e^{-\lambda t} B_{i,j}(\mathbf{x}_l) < 0$. Hence, $\mathcal{C}_j(\varphi(t)) = Y$ for all $t \in [l\tau, (l+1)\tau]$ This establishes the result for **case**-1.

(**case-2**:) Let $B^{(i)}$ be such that $\mathcal{C}_i(\varphi(t)) = G$ for some t in the time interval. There must exist a time instant $t^* \in [l\tau, (l+1)\tau]$ such that $B^{(i)}(\varphi(t^*)) = -K$ for some $i < j$ and $B^{(i)}((\varphi(t^* + \delta)) < -K$ for some $\delta > 0$. Let t^* be the infimum among all such time instants satisfying the property. Using the upward transit time bound, we note that $\mathcal{C}_i(\varphi(t^* + \delta)) = G$ and the control input during the time interval $[l\tau, (l+1)\tau]$ is fixed to $\mathbf{u} = \mathbf{u}_j$. Therefore, the control input is continuous. Hence, we conclude that $\mathcal{C}_i(\varphi(t)) \in \{G, Y\}$ for $t \in [t^*, (l+1)\tau]$ because $(l+1)\tau - t^* < \tau$ and $\mathcal{C}_j(\varphi(t)) \in \{G, Y\}$ for $t \in [l\tau, t^*)$. □

Theorem 5 Starting from a state $\mathbf{x}_0$, wherein, $\mathcal{C}_j(\mathbf{x}_0) \neq R$ for all $j \in [1, k]$ the system will never reach an unsafe state $x \in X_u$.

Proof. Using Lemma 3 and 4 we conclude that the state where all $\mathcal{C}_j(\varphi(t)) \neq R$ for all $j \in [1, k]$ cannot be reached in any trajectory φ starting from $\mathbf{x}_0$. Since for some j, $\mathcal{C}_j(\varphi(t)) \in \{G, Y\}$, it follows that $\varphi(t) \notin X_u$. □



$(x(t), y(t))$ t vs $\mathcal{B}(\mathbf{x}(t))$ $(x(t), y(t))$ t vs $\mathcal{B}(\mathbf{x}(t))$

Figure 3.5: (**Left**) Nominal trajectory for a fixed (feedforward) control signal $u(t)$ for dynamics in Ex. 1. ((**Right**) Effect of implementing the safety filter from Figure 3.2. Dashed lines show the original behavior.

Example 7 Figure 3.5 shows a nominal trajectory $(x(t), y(t))$ for a fixed control input signal $u(t)$ along with a plot of the function $\mathcal{B}(\mathbf{x}(t))$ over time, using the dynamics from Ex. 1. The portion of the trajectory that enters the unsafe region overapproximation is shaded red. This also corresponds to the part where $\mathcal{B}(\mathbf{x}) \geq -K$. On the other hand, we implement the safety filter. We periodically sampled states with a period $\tau = 0.02$. Notice that the override happens with $\mathcal{B}(\mathbf{x}) \geq -K$ where K is set to 1 in our implementation. After the override, the value of $\mathcal{B}(\mathbf{x})$ falls under the action of the control input corresponding to the control barrier function.

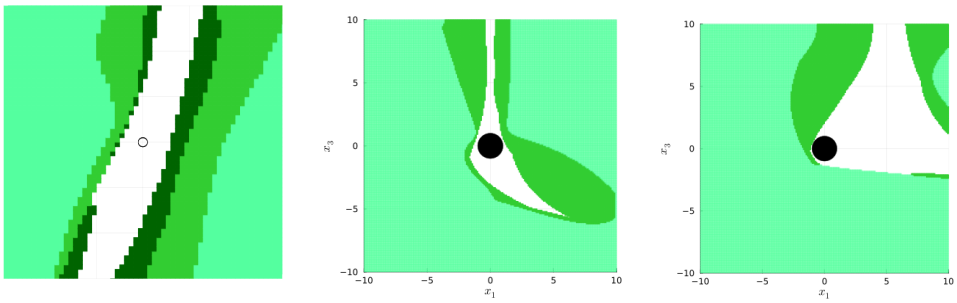
3.4 Empirical Evaluation

In this section, we will illustrate our approach with a few numerical examples. We demonstrate that the approach of computing successive barrier functions yields increasingly tighter approximations of the unsafe region.[2]

3.4.1 2D Nonlinear Dynamics

Consider a nonlinear system with two state variables (x_1, x_2) and two control inputs $(u_1, u_2) \in [-0.05, 0.05]^2$.

[2]The implementation and resulting barrier functions are available at https://github.com/rameezw/SuccessiveBarriers.



(a) **(b)** **(c)**

Figure 3.6: Improvement in approximation of the control invariant set through successive barrier functions. **(a)** approximation for the 2D example from section 3.4.1 with three iterations of successive barrier functions; **(b)** two iterations of successive barrier functions for 4D example from Section 3.4.2 with $x_2 = x_4 = 0$; **(c)** $x_2 = x_4 = -5$.

$$\dot{x}_1 = \frac{1}{2}x_1 - \frac{1}{5}x_2 - \frac{1}{100}x_1x_2 - \frac{1}{2}u_1 + \frac{1}{2}u_2,$$
$$\dot{x}_2 = x_1 - \frac{2}{5}x_2 - \frac{1}{20}x_2^2 - \frac{7}{10}u_2 + \frac{1}{10}u_1,$$

We take $U_{fin} = \{(u_1, u_2) \mid u_1 = \pm 0.05, u_2 = \pm 0.05\}$ and generate 500 randomly generated test points. The unsafe set is taken to be $X_u = \{(x_1, x_2) | x_1^2 + x_2^2 \leq 0.1\}$. We computed three levels of successive barrier functions by applying Algorithm 2. The approach managed to eliminate 490 test points from the unsafe region. $B^{(1)}$ is obtained as the minimum of two polynomial barrier functions, $B^{(2)}$ is obtained as a minimum of 9 functions and $B^{(3)}$ consists of 5 functions. Figure 3.6(a) shows the overall control invariant region obtained by our approach. Synthesizing the first level barriers required 160 seconds of CPU time, the second level barrier functions required 256 seconds of CPU time and the third level required 344 seconds. All timings were measured on a MAC OSX laptop running 3.1 GHz Quad-Core Intel Core i7 and 16GB RAM.

3.4.2 4D Nonlinear Dynamics

We consider an example of a polynomial system with 4 state variables and 2 control inputs $u_1, u_2 \in [-0.1, 0.1]^2$.

$$\dot{x}_1 = x_2, \quad \dot{x}_2 = \frac{1}{2}x_1 - \frac{1}{5}x_2 + \frac{1}{20}x_3x_1 - \frac{1}{100}x_1x_2 - \frac{1}{2}u_1,$$
$$\dot{x}_3 = x_4, \quad \dot{x}_4 = -\frac{2}{5}x_4 + \frac{1}{5}x_1 - \frac{1}{20}x_3^2 - \frac{7}{10}u_2,$$

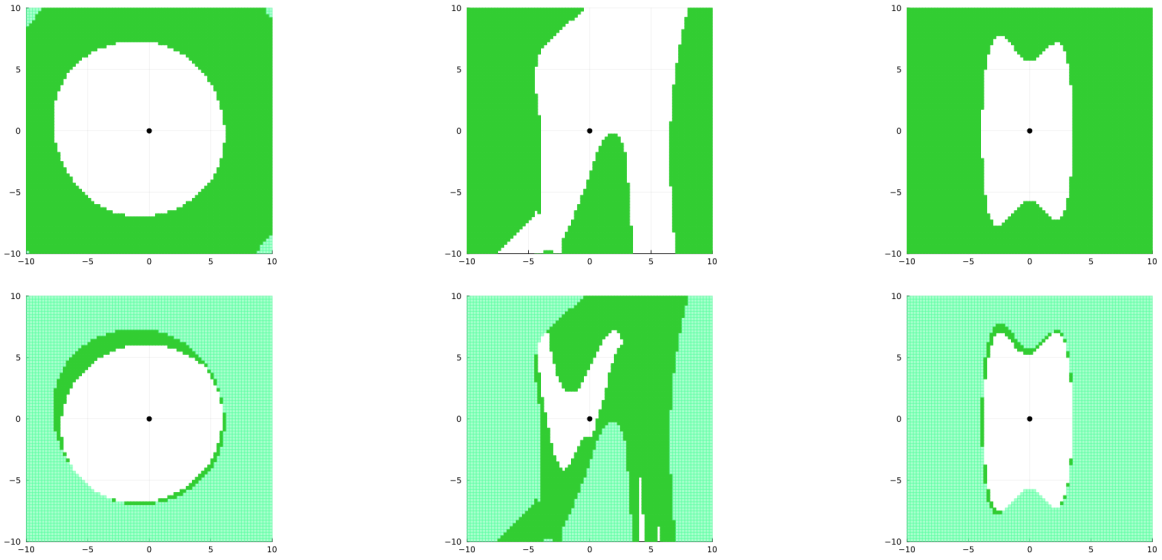
The unsafe set is taken to be $X_u = \{(x_1, x_2)|x_1^2 + x_2^2 \leq 1.0\}$. The set U_{fin} was set to the vertices of $U : [-0.1, 0.1]^2$ and we chose 200 randomly selected test states. We computed two levels of successive barrier functions by applying Algorithm 2. The barrier functions synthesized eliminated 190 test points from the unsafe set. $B^{(1)}$ is obtained as the minimum of 4, and $B^{(2)}$ as a minimum of 34 polynomial functions. The minimum transit time was 1.76. Figure 3.6(b,c) show the control invariant set thus obtained. Synthesizing the first level barriers required 170 seconds of CPU time, and the second level barrier functions required 1010 seconds of CPU time.

3.4.3 5D Dynamics (Coordinated Turn Model)

We now consider a coordinated turn model with 5 state variables and 2 control inputs $u_1, u_2 \in [-5, 5]^2$.

$$\begin{aligned} \dot{x}_1 &= x_3 \cos x_4, & \dot{x}_4 &= x_5, \\ \dot{x}_2 &= x_3 \sin x_4, & \dot{x}_5 &= u_2, \\ \dot{x}_3 &= u_1, \end{aligned}$$

The unsafe set is taken to be $X_u = \{(x_1, x_2)|x_1^2 + x_2^2 \leq 0.1\}$. The set U_{fin} was set to the vertices of $U : [-5, 5]^2$ and we chose 500 randomly selected test states. We computed three levels of successive barrier functions by applying Algorithm 2. $B^{(1)}$ is obtained as the minimum of four polynomial barrier functions, $B^{(2)}$ is obtained as a minimum of 15, and $B^{(3)}$ is obtained as a minimum of 12 polynomial functions. Figure 3.8(a,b,c) show the resulting control invariant sets obtained by

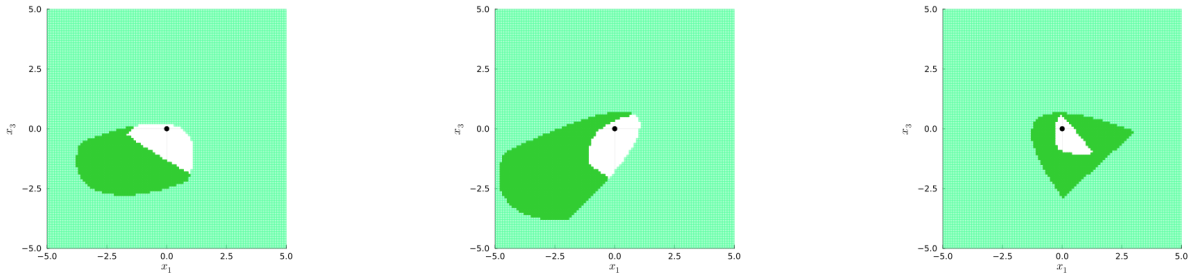


(a) **poly1**

(b) **van der Pol**

(c) **inv pendulum**

Figure 3.7: Extension of the controlled invariant set through successive barrier functions for different dynamical systems.



(a) **(b)** **(c)**

Figure 3.8: Control invariant set improvement through three iterations of successive barrier functions for 5D coordinated turn model from Section 3.4.3 with **(a)** $x_3 = 4, x_4 = 0, x_5 = 3$; **(b)** $x_3 = 4, x_4 = 1, x_5 = 3$; **(c)** $x_2 = x_4 = -5$.

our approach.

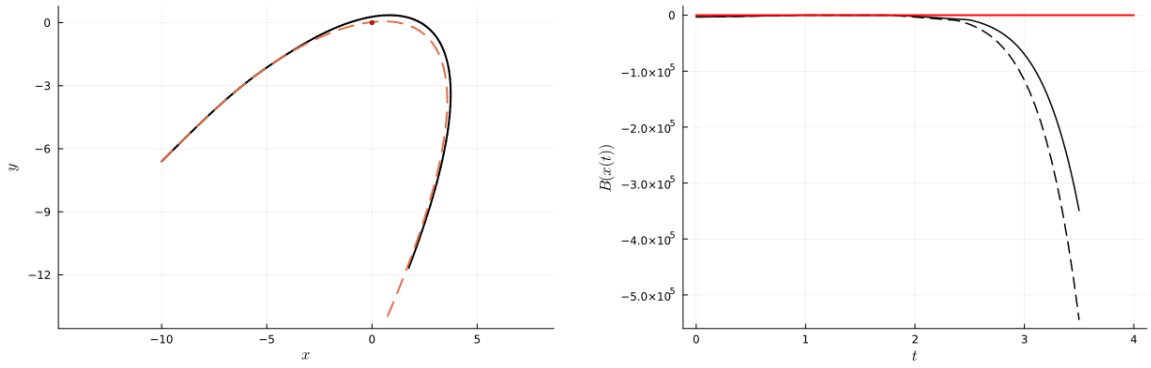
We use a polynomial approximation of trigonometric functions. We also model the error between the piecewise affine approximation and the trigonometric function as a disturbance input whose values are appropriately bounded. Details of the polynomial functions and error bounds are available as part of the supplementary material.

3.4.4 6D Nonlinear Dynamics (Planar Multirotor)

We now consider a planar multirotor model with 6 state variables and 2 control inputs $u_1, u_2 \in [0, 2]^2$.

$$\begin{aligned} \dot{x}_1 &= x_3, & \dot{x}_4 &= (u_1 + u_2)\cos x_5 - 2, \\ \dot{x}_2 &= x_4, & \dot{x}_5 &= x_6, \\ \dot{x}_3 &= (u_1 + u_2)\sin x_5, & \dot{x}_6 &= u_1 - u_2, \end{aligned}$$

The unsafe set is taken to be $X_u = \{(x_1, x_2) | x_1^2 + x_2^2 \leq 0.1\}$. The set U_{fin} was set to the vertices of $U : [0, 2]^2$ and we chose 900 randomly selected test states. We computed three levels of successive barrier functions by applying Algorithm 2. $B^{(1)}$ is obtained as the minimum of four polynomial barrier functions, $B^{(2)}$ is obtained as a minimum of 19 polynomial functions, and $B^{(3)}$ is



$(x(t), y(t))$ t vs $\mathcal{B}(\mathbf{x}(t))$

Figure 3.9: Effect of implementing the safety filter from Figure 3.2 for a fixed (feedforward) control signal $u(t)$ for dynamics in Ex. 3.4.4. Dashed lines show the original behavior.

obtained as a minimum of 33 polynomial functions.

3.4.5 Certifying Sum-Of-Square (SOS) Programs

The barriers above are produced by a floating-point SDP solver, so the SOS certificates must themselves be validated before the safety guarantees can be claimed. We defer this verification layer of extracting the Putinar multipliers and checking the positivstellensatz residue in high-precision arithmetic [75] to Chapter 4, where it is developed uniformly for all certificate types in the dissertation.

3.4.6 Comparison with FOSSIL

FOSSIL[23] is a tool for computing verified neural certificates. A comparison showing the effectiveness of our approach for barrier synthesis was conducted for multiple dynamical systems. The results are displayed in Table 3.1. The neural certificate was trained on a 2-layer network having 10 neurons each, with sigmoid and square activations, respectively; and was verified using dReal. In these evaluations, a timeout limit was set at 10^4 seconds, beyond which the experiment was considered a failure. The comparisons show that although FOSSIL swiftly computes barriers for low-dimensional systems, it does not terminate for high dimensions. On the contrary, our SOS-based successive barriers approach, although takes some time is successful in computing the

Table 3.1: Barrier synthesis comparison

			FOSSIL		Ours					
system	dim	inputs	success	time(s)	success	time(s):$B^{(1)}$	# barriers	max degree	time(s):$B^{(2)}$	# barriers
poly1	2	1	✓	3.2	✓	**0.6**	2	2	34.1	1
poly2	2	1	✓	1.9	✓	**1.0**	2	2	36.9	3
van der Pol	2	1	✓	4.8	✓	**2.0**	2	2	247.4	7
inv pendulum	2	1	✓	3.2	✓	**2.0**	2	2	207.3	1
poly3	2	2	✓	**1.3**	✓	7.6	4	4	194.1	12
poly4	3	2	✓	74.8	✓	**6.8**	4	4	281.2	8
poly5	4	2	✗	-	✓	**36.4**	4	4	587.2	27
coord turn	5	2	✗	-	✓	**108.4**	4	4	2290.8	15
planar multirotor	6	2	✗	-	✓	**131.6**	4	4	3305.5	9

barrier functions for multiple levels of the hierarchy, even for higher dimensions.

Another comparison was the size of the controlled invariant set computed using the two approaches. The results are shown in Table 3.2. The comparison was done over 1000 test points sampled using a fixed seed over the same domains. It turns out that the successive barriers approach improves the size of the safe sets defined by the neural CBFs computed using FOSSIL. Hence, using successive barrier functions results in less conservative approximations of the controlled invariant set.

3.5 Discussion and Limitations

We develop the concept of successive control barrier functions for tight over-approximations of unsafe sets for nonlinear systems. A runtime monitor is also presented which utilizes a shield based on the successive barriers to ensure collision avoidance. The switching strategy between the successive barriers respects minimum transit time constraints.

This construction reduces, but does not eliminate, the conservatism of a single barrier; the certified region grows with each level, and Chapter 5 shows the same mechanism enlarging a region of attraction for the reachability specification, while Chapter 6 reuses the multiple-barrier idea for moving obstacles. We return to the broader implications, and to the limitations shared across the technical chapters, in Chapter 7.

Table 3.2: Controlled invariant set comparison for 1000 test points. S denotes points in the safe region(inside the control invariant set) and U denotes points in the unsafe region(outside the control invariant set)

system	dimensions	inputs	FOSSIL(S)	Ours(S)	FOSSIL(U) & Ours(S)	Ours(U) & FOSSIL(S)
poly1	2	1	417	**649**	**375**	143
poly2	2	1	716	**922**	**215**	9
van der Pol	2	1	507	**649**	**322**	180
inv pendulum	2	1	683	**798**	**193**	78
poly3	2	2	556	**977**	**421**	0

Chapter 4

Robustifying SOS proofs

" $0.1 + 0.2 = 0.30000000000000004$ "
— IEEE 754 floating-point arithmetic

The certificates in this dissertation are produced by floating-point semidefinite solvers, so the returned polynomial identities hold only up to a numerical residual: acceptable for design iteration, but not an exact proof. This chapter closes the gap by rationalizing the solver output and refitting the Gram matrices inside the diagonally dominant cone by an exact rational linear program, which yields an explicit rational weighted-SOS decomposition requiring no square roots. The repaired certificate satisfies its defining identities exactly and can be verified independently by a short standalone checker or by the Lean 4 theorem prover, turning numerical evidence into a machine-checkable proof.

This chapter addresses a verification gap in the SOS-based certificates used in the dissertation. The SOS programs used to synthesize barriers and multipliers are solved by floating-point SDP solvers, so the returned Gram matrices and polynomial identities are only approximate. A numerical residual of order 10^{-6} or 10^{-8} may be acceptable for design iteration, but it is not an exact mathematical proof.

The goal here is narrower and more concrete: given a numerical SOS certificate, produce a rational certificate whose polynomial identities can be checked exactly and whose nonnegativity follows from an explicit rational sum-of-squares decomposition. Section 4.1 recalls the current high-precision check. Sections 4.2-4.3 describe rationalization and a diagonally-dominant refitting step. Section 4.4 states the resulting soundness guarantee, Section 4.5 explains how the certificate can be machine-checked, and Sections 4.6-4.8 give examples and limitations.

4.1 Certifying Sum-Of-Square (SOS) Programs

SOS programming uses semi-definite programming solvers to synthesize the barrier functions $B(\mathbf{x})$. In order to certify that B satisfies the conditions for being a barrier/successive barrier, we need to verify that the results are not invalidated due to numerical issues [75]. Note that each barrier consists of multiple entailments of the form:

$$p_1(\mathbf{x}) \geq 0, \ldots, p_m(\mathbf{x}) \geq 0 \;\models\; p \geq 0\,.$$

SOS programming certifies this through a Putinar positivstellensatz proof that states that

$$\exists\, \sigma_1, \ldots, \sigma_m \in \mathsf{SOS}_d[\mathbf{x}]\; p - \sigma_1 p_1 - \cdots - \sigma_m p_m \in \mathsf{SOS}_d[\mathbf{x}]\,,$$

wherein $\mathsf{SOS}_d[\mathbf{x}]$ represents the set of all SOS polynomials over $\mathbf{x}$ of degree at most d, and d is a parameter chosen by the user/SOS modeling package [40]. We write $\sigma_0 := p - \sigma_1 p_1 - \cdots - \sigma_m p_m$ for the remainder SOS term, so the proof consists of the $m+1$ sum-of-squares polynomials $\sigma_0, \sigma_1, \ldots, \sigma_m$

together with the single identity that ties them to p and the p_i.

Remark 2 (Putinar versus Schmüdgen) Putinar's is not the only Positivstellensatz. Schmüdgen's theorem [77] states that on a compact semi-algebraic set every strictly positive polynomial has a representation $p = \sum_{\alpha \in \{0,1\}^m} \sigma_\alpha \, p_1^{\alpha_1} \cdots p_m^{\alpha_m}$ with one SOS multiplier σ_α per subset of the constraints. Putinar's form is the restriction of this sum to $|\alpha| \leq 1$, single constraints only, no products, so the quadratic module it generates sits inside Schmüdgen's preordering. The preordering assumes compactness, and at a fixed multiplier degree it can represent polynomials that the Putinar form cannot. The cost is the size. Schmüdgen's form has 2^m multipliers instead of Putinar's $m+1$, which is why SOS tooling, including the package we use, encodes the Putinar form. Putinar's theorem in turn requires the quadratic module to be archimedean, which we ensure in practice by including a ball constraint $R - \|\mathbf{x}\|_2^2 \geq 0$ among the p_i. The products $p_1^{\alpha_1} \cdots p_m^{\alpha_m}$ are fixed rational polynomials, so a Schmüdgen-form identity rationalizes and repairs exactly as the Putinar form does, only with more terms.

We verified the results of the SOS programming package used (`SumOfSquares.jl`) by outputting the polynomials $\sigma_1, \ldots, \sigma_m$ and computing the "residue" $p - \sigma_1 p_1 - \cdots - \sigma_m p_m$. In each case, we obtain a representation $\sigma_i = m(\mathbf{x})^\top Q_i m(\mathbf{x})$, wherein $m(\mathbf{x})$ is a monomial basis whose entries consist of monomials of degree $\leq \frac{d}{2}$ and Q_i is a $|m(\mathbf{x})| \times |m(\mathbf{x})|$ matrix. We verify that Q_i is positive semi-definite by computing its Cholesky decomposition. The C++ library `Eigen` was used for this purpose: we used 512 bit floating point representation to carry out the Cholesky decomposition.

Using this process, we verified that all the SOS programming results obtained in the process of generating the successive barrier functions were verified using high precision floating point arithmetic. The procedure has a known weakness. It is exact only in the limit of infinite precision. At any finite precision the residue is a small nonzero polynomial and the Cholesky test reports positive semi-definiteness only up to a numerical threshold. The fix is to move to rational arithmetic, where both the residue and the positive semi-definiteness test become exact. The obstacle is that the usual square-root Cholesky factorization does not directly produce a rational polynomial certificate. Even

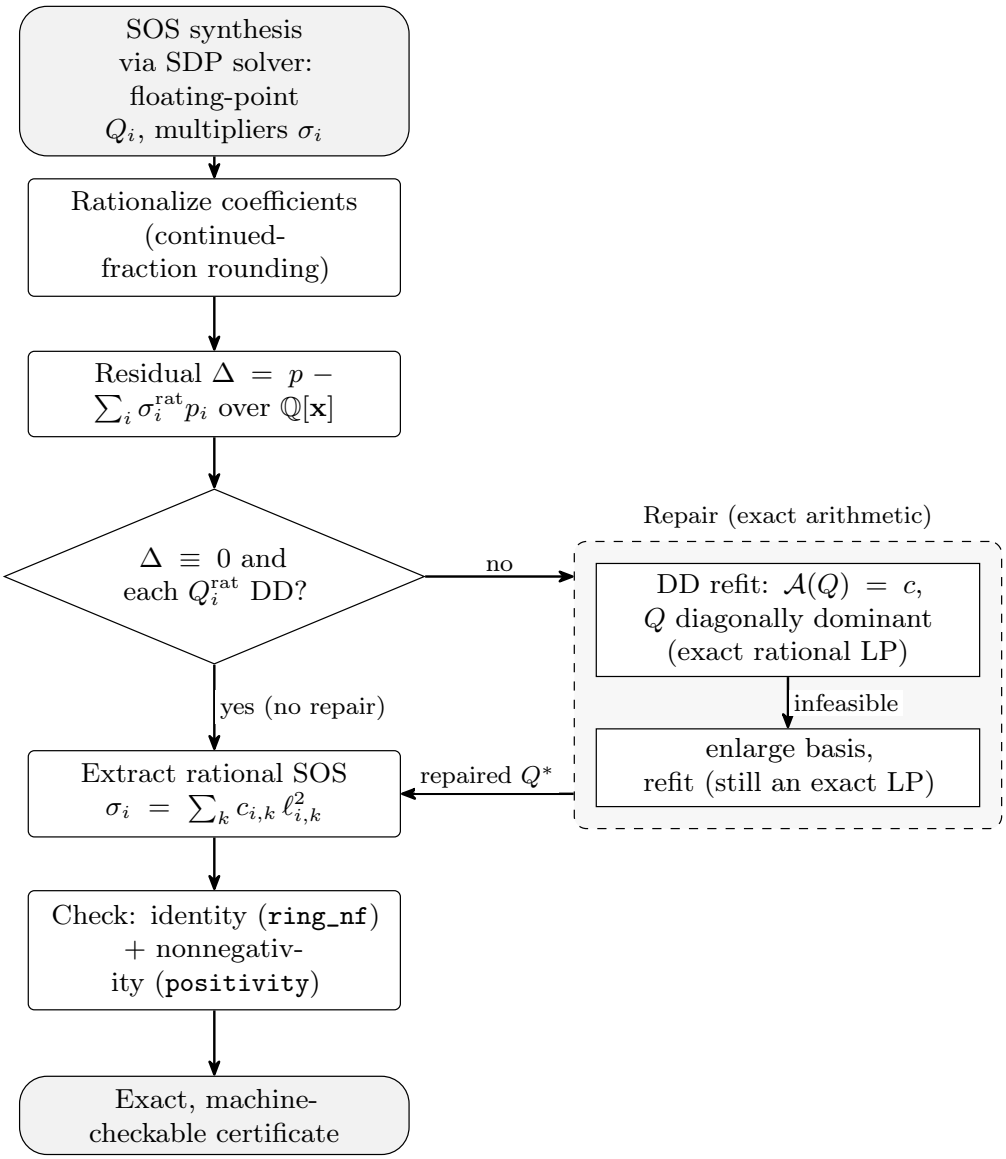


Figure 4.1: The robustification pipeline.

when Q has rational entries, the square roots in the factorization need not be rational. Roux et al. [75] take a different route to exactness. They run it in floating point with directed rounding and bound the rounding error, which certifies positive semi-definiteness rigorously without rational square roots. We explored that direction as well. The remainder of this chapter develops a different and complementary route. Rather than relying on the returned Q_i, we try to re-fit each SOS term inside a cone whose membership is described by rational linear constraints. When this refit is feasible, it yields a rational sum-of-squares decomposition and avoids square roots.

4.2 From High-Precision Floating Point to an Exact Certificate

4.2.1 Exact certificates

The verification in Section 4.1 is only a numerical one, checking whether the residue is small, and is each Q_i definite to a threshold. An exact certificate has three additional properties for the terms, $\sigma_0, \ldots, \sigma_m$. First, all coefficients and all entries of every Q_i are rational. Second, the identity

$$p(\mathbf{x}) \;=\; \sigma_0(\mathbf{x}) + \sigma_1(\mathbf{x})\, p_1(\mathbf{x}) + \cdots + \sigma_m(\mathbf{x})\, p_m(\mathbf{x}) \tag{4.1}$$

holds exactly in $\mathbb{Q}[\mathbf{x}]$, that is, after collecting terms every coefficient on the two sides agrees. Third, each σ_i is presented as an explicit *rational-weighted* sum of squares,

$$\sigma_i(\mathbf{x}) \;=\; \sum_k c_{i,k}\, \ell_{i,k}(\mathbf{x})^2, \qquad c_{i,k} \in \mathbb{Q},\; c_{i,k} \geq 0,\; \ell_{i,k} \in \mathbb{Q}[\mathbf{x}]. \tag{4.2}$$

Two facts make such a certificate checkable by a short trusted program, or by a theorem prover. The identity (4.1) is a polynomial equality over $\mathbb{Q}[\mathbf{x}]$, which is decided by normalizing both sides. The nonnegativity of each σ_i is evident from eq. (4.2): a nonnegative rational times a square is nonnegative, and a sum of nonnegative terms is nonnegative. Keeping the positive weights $c_{i,k}$ outside the square, rather than absorbing them as $(\sqrt{c_{i,k}}\,\ell_{i,k})^2$, is what keeps the whole object inside $\mathbb{Q}[\mathbf{x}]$. This is how we avoid the square-root obstacle noted in Section 4.1.

4.2.2 Rationalization and the residual polynomial

The solver returns each Q_i and each coefficient of σ_i in floating point. The first step is to replace these numbers by nearby rationals. We do this with continued-fraction (Stern-Brocot) rounding under a tolerance τ and a denominator bound $q_{\max}$. For a float c we take the convergent a/b of smallest denominator with $b \le q_{\max}$ and $|c - a/b| \le \tau$. Scaling each identity so that its coefficients have comparable magnitude before rounding keeps denominators small, and clean values such as $\frac{1}{2}$ and $\frac{1}{3}$ are recovered exactly in the easy cases [67].

The choice of continued-fraction rounding, in preference to rounding to a fixed number of decimal places, is deliberate, because it controls the size of the denominators that enter the certificate. Every real number c has a continued-fraction expansion $c = a_0 + 1/(a_1 + 1/(a_2 + \cdots))$ with $a_0 \in \mathbb{Z}$ and $a_i \in \mathbb{Z}_{>0}$ for $i \ge 1$, and truncating it after finitely many steps gives a rational a/b called a *convergent*. Convergents are best approximations in a strong sense. No rational with denominator at most b lies closer to c than a/b does. Scanning the convergents, together with the intermediate semiconvergents, for the first one within the tolerance τ therefore returns a rational whose denominator is as small as possible for that accuracy. This can be computed using Julia's `rationalize(c; tol)` method. Decimal rounding offers no such guarantee: forcing c to k digits fixes the denominator at 10^k whether or not a far simpler fraction would do, so a clean value such as $\frac{1}{3}$ is replaced by a fraction with a k-digit denominator. Small denominators matter downstream. They keep the exact LP of Section 4.3.5 well conditioned, and they keep the `Lean` ring-normalization check of Section 4.5 fast, since both manipulate the rational coefficients directly and their cost grows with the bit-size of those coefficients. However, the guarantee is only as good as the input. When the solver returns a genuinely irrational or badly conditioned value, no tolerance recovers a small denominator, and the bit-size can still grow. We mention this as a limitation in Section 4.8.

Remark 3 (Rationalizing the matrix versus its factorization) An alternative to rounding the entries of Q_i is to round its numerical Cholesky factor. If $Q_i \approx L_i L_i^\top$, rationalize the entries of L_i to $\tilde{L}_i$ and set $\tilde{Q}_i = \tilde{L}_i \tilde{L}_i^\top$. This has an advantage. $\tilde{Q}_i$ is positive semi-definite *by construction*,

whatever the rounding does, and the rows of $\tilde{L}_i$ are already the explicit rational squares, so nothing further needs to be checked or extracted on the nonnegativity side. What it gives up is the polynomial. The map $L \mapsto LL^\top$ is quadratic, and rounding L perturbs $\tilde{\sigma}_i = m(\mathbf{x})^\top \tilde{Q}_i\, m(\mathbf{x})$ away from σ_i, so the identity (4.1) gets a nonzero residual. Indeed, even when Q_i itself has exactly rational entries, its Cholesky factor involves square roots and is typically irrational, so rounding the factor perturbs a solution that entry-rounding would have recovered exactly. The two strategies therefore fail in complementary ways. Entry-rounding can preserve the identity but may lose positive semi-definiteness, while factor-rounding guarantees positive semi-definiteness and the decomposition but perturbs the identity. Rounding a decomposition and then adjusting to restore the identity is the strategy of Harrison [32] and of Peyrl and Parrilo [67], who project the rounded matrix back onto the coefficient subspace and rely on a definiteness margin to stay inside the PSD cone. The repair of Section 4.3 can be read as the same move with the margin assumption replaced by the diagonal-dominance constraint, which makes the projection an exactly solvable rational LP. Either way, the missing half is always the identity, and Section 4.3 restores it by construction through the constraint $\mathcal{A}(Q) = c$.

Let the rationalized data give σ_i^{rat} and matrices Q_i^{rat}. Define the *residual polynomial*

$$\Delta(\mathbf{x}) \;:=\; p(\mathbf{x}) - \sum_{i=0}^{m} \sigma_i^{\mathrm{rat}}(\mathbf{x})\, p_i(\mathbf{x}) \;\in\; \mathbb{Q}[\mathbf{x}], \qquad p_0 := 1, \tag{4.3}$$

which is the exact, rational version of the "residue" we already compute in Section 4.1. The residual is computable by polynomial subtraction over $\mathbb{Q}[\mathbf{x}]$, with no rounding.

Lemma 5 $\Delta \equiv 0$ in $\mathbb{Q}[\mathbf{x}]$ if and only if the identity (4.1) holds exactly for the rationalized data. If, in addition, every Q_i^{rat} is positive semi-definite, then $\sigma_0^{\mathrm{rat}}, \ldots, \sigma_m^{\mathrm{rat}}$ already form an exact certificate.

Proof. If $\Delta \equiv 0$ then (4.1) holds coefficientwise over $\mathbb{Q}$ by definition of Δ, and conversely. Given the identity and positive semi-definiteness of each Q_i^{rat}, each $\sigma_i^{\mathrm{rat}} = m(\mathbf{x})^\top Q_i^{\mathrm{rat}} m(\mathbf{x})$ is a sum of squares, so the right-hand side of (4.1) is nonnegative whenever the $p_i \geq 0$, which is the required entailment.

□

A rational positive semi-definite Gram matrix may already be enough for the proof object. A square-root-free $LDL^\top$ factorization, computed in exact rational arithmetic, writes $Q^{\text{rat}} = LDL^\top$ with L unit lower triangular and D diagonal, both rational. Then

$$\sigma \;=\; m(\mathbf{x})^\top Q^{\text{rat}}\, m(\mathbf{x}) \;=\; \sum_j D_{jj} \left(L^\top m(\mathbf{x})\right)_j^2$$

is the rational-weighted form (4.2), and the check $D_{jj} \geq 0$ certifies positive semi-definiteness. This is also why rationalizing a factorization (Remark 3) is sufficient on the nonnegativity side. The decomposition *is* the certificate. What neither observation supplies is the identity. When rounding leaves $\Delta \not\equiv 0$, or leaves Q^{rat} indefinite so that no factorization with $D \geq 0$ exists, a repair is needed, and that is the case the rest of the chapter addresses. The diagonally dominant cone enters not for the decomposition, which $LDL^\top$ handles for any rational PSD matrix, but for the *repair*. Diagonal dominance is a set of linear conditions, so restoring the identity and membership simultaneously is a rational LP (Section 4.3), and Lemma 7 then reads off the squares directly.

When $\Delta \equiv 0$ and each Q_i^{rat} is definite, we are done and no repair is needed. The interesting case is when rounding leaves $\Delta \not\equiv 0$, or leaves $\Delta \equiv 0$ but pushes some Q_i^{rat} towards indefiniteness. We discuss both before fixing them.

4.2.3 How rationalization fails

Roux et al. [75] identify four sources of inaccuracy in the SDP solution, and each has a counterpart when we try to rationalize.

The first is inexact termination: the solver stops at an approximate optimum rather than the exact one, so the equality constraints that encode the identity (4.1) hold only to its tolerance. The multipliers it returns therefore do not reproduce p exactly, and once we round them to rationals the residual Δ is a small nonzero polynomial. We call this identity drift.

The second, which they stress as structural rather than numerical, is the failure of strict

feasibility. An SDP is strictly feasible when it has a solution that is positive definite; when strict feasibility fails, every feasible Gram matrix is only positive semi-definite, hence singular, with a zero eigenvalue. The feasible region lies on the boundary of the PSD cone, which is the worst place for rounding. It can easily push $\lambda_{\min}(Q_i) \approx 0$ into $\lambda_{\min}(Q_i^{\text{rat}}) < 0$ leaving the rounded matrix no longer a valid witness. This degeneracy is common in entailment and invariant problems, where the consecution condition is often tight. It holds with equality at an equilibrium or wherever the certificate touches zero, which forces a multiplier to vanish there and its Gram matrix to lose rank.

The third is ill-conditioning, where near-active constraints make Q_i nearly singular and amplify the first two. The fourth is plain floating-point error in the solver.

All four are addressed by re-fitting Q_i in exact arithmetic, inside a cone whose membership we can verify exactly, which is the subject of Section 4.3.

4.3 Repair by Diagonally Dominant Refitting

In this chapter we focus on a diagonally-dominant repair of numerical SOS certificates. Given a polynomial term represented numerically as $m(x)^\top Q m(x)$, we search for a rational Gram matrix Q^* that represents the same polynomial and is diagonally dominant. The coefficient-matching constraints are linear in the entries of Q^*, and diagonal dominance can be expressed by linear inequalities after introducing absolute value slack variables. Hence, the repair problem is an exact rational linear program.

This restriction is conservative. Not every SOS polynomial admits a diagonally-dominant Gram representation in a fixed basis. Nevertheless, when the repair succeeds, the resulting certificate is especially easy to verify. Diagonal dominance implies positive semidefiniteness, and the corresponding polynomial admits an explicit rational weighted sum-of-squares decomposition. Thus, the certificate can be checked using exact rational arithmetic rather than relying on floating-point semidefinite solver output.

Scaled diagonally-dominant matrices and SDSOS polynomials provide a less conservative alternative, but they lead to second-order cone constraints rather than linear constraints. Since the

implementation and experiments in this chapter use only linear programming over the diagonally-dominant cone, SDSOS is not part of the main pipeline and is mentioned only as a possible extension.

The verification of Section 4.1 asks whether $Q_i \succeq 0$, and tests it with Cholesky. Positive semi-definiteness is the required condition, but it is described by an infinite family of linear inequalities (one per direction), which is why the test uses factorization rather than a finite check. Ahmadi and Majumdar [1] observed that two subsets of the PSD cone have finite, cheap descriptions and are still rich enough for many problems.

Definition 12 (DD and SDD matrices) A symmetric matrix $Q \in \mathbb{R}^{N\times N}$ is *diagonally dominant* (DD) if

$$Q_{jj} \;\geq\; \sum_{k\neq j} |Q_{jk}| \qquad \text{for every } j.$$

It is *scaled diagonally dominant* (SDD) if DQD is DD for some diagonal $D \succ 0$. We call $\sigma = m(\mathbf{x})^\top Q\, m(\mathbf{x})$ a DSOS (resp. SDSOS) polynomial when Q may be taken DD (resp. SDD).

The DD conditions are linear in the entries of Q, so optimizing over DD matrices is a linear program (LP). The SDD conditions are a family of 2×2 PSD blocks, which are second-order cone constraints, so optimizing over SDD matrices is a second-order cone program. Both are much cheaper than an SDP:

$$\text{DD} \;\subseteq\; \text{SDD} \;\subseteq\; \{Q : Q \succeq 0\}. \tag{4.4}$$

The inclusions are generally strict once there is more than one variable and the degree exceeds two [1]; this is the source of the conservatism we discuss in Section 4.8. We work in the DD cone throughout, because its membership is linear. The refit is then an LP, which we can solve exactly over $\mathbb{Q}$ (Section 4.3.5). `SumOfSquares.jl` provides the DSOS cone directly [1, 40], so the repair fits the tooling we use for synthesis.

Roux et al. [75] suggested investigating DSOS relaxations as future work; we use the DD

cone not for synthesis but for the repair step. The reason DD is the right cone for repair is that membership both implies positive semi-definiteness and yields a decomposition.

Lemma 6 (DD implies PSD) If Q is symmetric and diagonally dominant with $Q_{jj} \geq 0$ for all j, then $Q \succeq 0$.

Proof. Q is symmetric, so its eigenvalues are real. By Gershgorin's theorem [34] every eigenvalue λ lies within $\sum_{k \neq j} |Q_{jk}|$ of some diagonal entry Q_{jj}, hence $\lambda \geq Q_{jj} - \sum_{k \neq j} |Q_{jk}| \geq 0$ by diagonal dominance. All eigenvalues are nonnegative, so $Q \succeq 0$. □

Unlike the Cholesky test, Lemma 6 needs no square roots. It is a finite set of comparisons among rationals. This is the first half of the answer to the obstacle stated in Section 4.1.

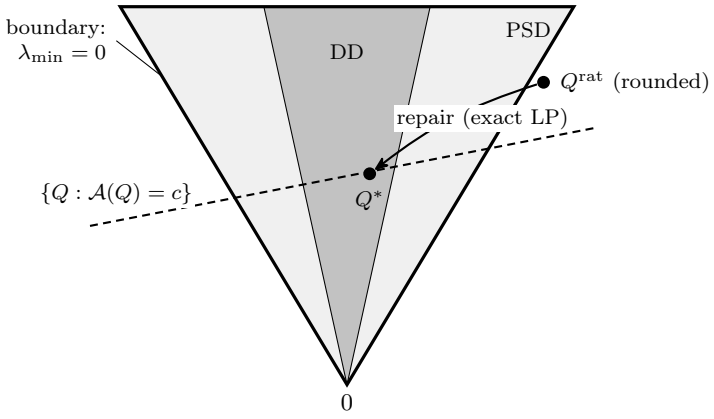


Figure 4.2: Geometry of the repair, in the space of symmetric Gram matrices.

4.3.1 The repair as an exact feasibility problem

Fix one SOS term σ_i, its monomial basis $m(\mathbf{x})$, and the exact rational polynomial σ_i^* it should equal. For a multiplier, this is the rounded multiplier polynomial after any accepted coefficient correction. For the residual term σ_0, the target must be chosen so that the full identity (4.1) holds exactly. This is the main algebraic constraint: the repair cannot arbitrarily absorb a sign-indefinite residual into an SOS term. Matching coefficients in $m(\mathbf{x})^\top Q\, m(\mathbf{x}) = \sigma_i^*$ is a system of *linear* equalities in the entries of Q; write it as $\mathcal{A}(Q) = c$, where $\mathcal{A}$ sends a symmetric matrix to the

coefficient vector of $m(\mathbf{x})^\top Q\, m(\mathbf{x})$ and c is the coefficient vector of σ_i^*. The repair searches for a Gram matrix in the DD cone meeting these equalities:

$$\text{find } Q = Q^\top \quad \text{such that} \quad \mathcal{A}(Q) = c, \qquad Q_{jj} \geq \sum_{k \neq j} |Q_{jk}| \ \text{ for all } j. \tag{4.5}$$

The absolute values are linearized with variables $t_{jk} \geq \pm Q_{jk}$ for $j \neq k$ and the constraints $Q_{jj} \geq \sum_{k \neq j} t_{jk}$. All data are rational, so (4.5) is a rational LP. To keep the repaired matrix close to the solver's output we minimize the ℓ_1 distance $\sum_{jk} |Q_{jk} - Q_i^{\text{rat}}|$, which is also linear. Any feasible Q^* is positive semi-definite by Lemma 6 and reproduces σ_i^* exactly, so $\sigma_i^* = m(\mathbf{x})^\top Q^* m(\mathbf{x})$ is an exact DSOS representation.

Figure 4.2 shows the geometric picture. The diagonally dominant (DD) cone sits inside the scaled cone (SDD), which sits inside the PSD cone; the outer boundary is where a matrix becomes singular ($\lambda_{\min} = 0$). The dashed line is the affine subspace $\{Q : \mathcal{A}(Q) = c\}$ of Gram matrices that reproduce the target polynomial, and its extent is the gauge freedom of Section 4.3.2. Rounding moves the solver's matrix to Q^{rat}, which can fall off the subspace and outside the PSD cone, most easily when the solution lay on the boundary (failure of strict feasibility). The repair re-fits along the subspace into the DD cone, returning an exact Q^* whose diagonal dominance is checkable without square roots.

This is a repair, not a re-synthesis. Once the target polynomial σ_i^* has been fixed, the basis $m(\mathbf{x})$ and the polynomial represented by the SOS term are unchanged. The LP changes only the Gram matrix, and only in directions that preserve that polynomial. If the rounded multipliers no longer satisfy the global identity, an additional coefficient-correction step is required before this Gram refit applies.

4.3.2 Why there is room to repair

Fixing σ_i^* does not fix Q outright, because the map $Q \mapsto m(\mathbf{x})^\top Q\, m(\mathbf{x})$ is not injective. Its kernel consists of symmetric matrices N with $m(\mathbf{x})^\top N\, m(\mathbf{x}) \equiv 0$, which exist whenever the

monomials in $m(\mathbf{x})$ satisfy multiplicative relations. With $m(\mathbf{x}) = (1, x, x^2)^\top$, for example, the monomial x^2 is produced both by the $(2,2)$ entry, from $x \cdot x$, and by the $(1,3)$ entry, from $1 \cdot x^2$; the matrix with $N_{22} = 1$, $N_{13} = N_{31} = -\frac{1}{2}$ and zeros elsewhere gives $m(\mathbf{x})^\top N\, m(\mathbf{x}) = x^2 - x^2 \equiv 0$. The repair (4.5) is free to move Q along this kernel while holding σ_i^* fixed, and it uses that freedom to enter the DD cone. In Figure 4.2 this freedom is the extent of the dashed subspace.

Remark 4 (Basis-dependence) The kernel above is nonempty only when $m(\mathbf{x})$ contains monomials with multiplicative relations, that is, monomials of degree ≥ 1 whose products coincide. For a pure quadratic form in a degree-one basis there is no such relation, the Gram matrix is determined uniquely, and the repair has no freedom at all. As an example, we take the polynomial $(1 + x + y)^2$. In the basis $(1, x, y)$ its Gram matrix is the all-ones matrix, which is neither diagonally dominant (it needs $1 \geq 1 + 1$) nor scaled diagonally dominant (the conditions $d_j \geq \sum_{k \neq j} d_k$ admit no positive solution), and the unique Gram leaves nothing to re-fit. A trivially square polynomial can therefore lie outside both repair cones in its natural basis; the only remedies are a larger basis or a full SOS solve. We return to this in Section 4.8.

Remark 5 (Adding the slack) One might hope to repair by adding a nonnegative slack Σ to the polynomial, writing $\sigma_i^* = m(\mathbf{x})^\top Q_i^{\text{rat}} m(\mathbf{x}) + \Sigma$. But with Q_i^{rat} held fixed, the identity forces Σ to equal the residual on that term exactly, and a residual is in general sign-indefinite, so a nonnegative Σ exists only in the special case that the residual is itself a DSOS polynomial. The freedom that makes repair work is not a sign-definite bump added to σ_i^*; it is the kernel freedom of the Gram parameterization described above. Phrased through that freedom, the slack is $m(\mathbf{x})^\top (Q^* - Q_i^{\text{rat}}) m(\mathbf{x})$, a matrix difference that need not be sign-definite even though the final Q^* lies in the DD cone.

4.3.3 When DD is too small

If (4.5) is infeasible, the DD cone cannot represent σ_i^* in the basis $m(\mathbf{x})$. The workaround of adding monomials of one higher degree gives the coefficient map more kernel directions (Section 4.3.2) and often brings a stubborn term inside the DD cone, while the refit stays a rational LP. Two larger

cones exist in principle, the scaled cone SDD, an SOCP, and the full SOS cone, an SDP. By (4.4) both repair strictly more polynomials. We do not use them as neither is solvable exactly over $\mathbb{Q}$. A second-order cone or semidefinite feasibility problem with rational data need not have a rational solution, so Proposition 1 has no analogue, and one would be back to rationalizing a numerical solution. Extending the repair to these cones, by solving them numerically and re-verifying the rationalized result, is left to future work.

4.3.4 Explicit sum of squares decomposition

The repaired matrix Q^* is diagonally dominant, and we now turn it into the explicit decomposition (4.2). The following identity does this directly, with rational coefficients.

Lemma 7 (A DD matrix gives a rational-weighted SOS) Let $Q \in \mathbb{Q}^{N\times N}$ be symmetric and diagonally dominant with $Q_{jj} \geq 0$. Put $w_j := Q_{jj} - \sum_{k\neq j} |Q_{jk}| \geq 0$ and, for $j < k$, $s_{jk} := \operatorname{sign}(Q_{jk}) \in \{-1, 0, 1\}$. Then, writing m_j for the jth entry of $m(\mathbf{x})$,

$$m(\mathbf{x})^\top Q\, m(\mathbf{x}) \;=\; \sum_{j=1}^{N} w_j\, m_j^2 \;+\; \sum_{j<k} |Q_{jk}|\,\big(m_j + s_{jk}\, m_k\big)^2. \tag{4.6}$$

Every weight w_j and $|Q_{jk}|$ is a nonnegative rational and every term inside a square is a binomial in $\mathbb{Q}[\mathbf{x}]$. Hence the right-hand side is a rational-weighted sum of squares, and $m(\mathbf{x})^\top Q\, m(\mathbf{x}) \geq 0$.

Proof. Expand the right-hand side. The coefficient of m_j^2 is $w_j + \sum_{k\neq j} |Q_{jk}| = Q_{jj}$, since each binomial $(m_j + s_{jk}m_k)^2$ contributes $|Q_{jk}|$ to the m_j^2 term. For $j < k$ the coefficient of $m_j m_k$ is $2|Q_{jk}|\, s_{jk} = 2Q_{jk}$, which matches the symmetric form $m(\mathbf{x})^\top Q\, m(\mathbf{x}) = \sum_j Q_{jj} m_j^2 + \sum_{j<k} 2Q_{jk} m_j m_k$. No other cross terms arise because each binomial involves a single pair (j, k). The two sides therefore agree coefficientwise. The weights are nonnegative by diagonal dominance and each squared binomial is nonnegative. □

Lemma 7 replaces the Cholesky decomposition used in Section 4.1. Where Cholesky produces $Q = LL^\top$ with irrational entries in L, and where an $LDL^\top$ form would leave the irrational factors

$\sqrt{D_{jj}}$ inside the squares, the decomposition (4.6) keeps the rational factor as a *weight* on a square that is itself rational. This is the second half of the answer to the square-root obstacle: a DD certificate is not only checkable over $\mathbb{Q}$, it is *constructed* over $\mathbb{Q}$.

4.3.5 Solving the LP exactly

We solve (4.5) first in floating point for speed and re-rationalize, and then in exact rational arithmetic. The exactness is guaranteed by a standard fact about linear programming over the rationals.

Proposition 1 If (4.5) is feasible, then it has a feasible solution with rational entries. An exact rational LP solver can therefore return a matrix Q^* satisfying $\mathcal{A}(Q^*) = c$ exactly and the diagonal-dominance inequalities exactly.

Proof. The equalities and inequalities in (4.5) have rational coefficients. A nonempty rational polyhedron contains a rational feasible point. For example, after introducing slack variables, any basic feasible solution of a nonempty feasible subsystem is obtained by solving a linear system with rational coefficients. Exact rational simplex or an equivalent exact LP feasibility method therefore returns rational entries whenever the feasibility problem succeeds. □

In our prototype the exact solve uses the rational simplex in `CDDLib` [27] through Julia's exact $\mathbb{Q}$ arithmetic; an alternative is `QSopt_ex` [6]. Because the residual is small and the DD constraints are simple, the exact solve is well conditioned and fast at the sizes that arise in barrier verification.

4.4 Soundness

We state what a checked certificate guarantees. The statement specializes the soundness of Putinar certificates to the repaired, rationally-weighted form, and it is the formal version of the entailment in Section 4.1.

Theorem 6 (Soundness) Suppose the repair produces rational polynomials $\sigma_0, \ldots, \sigma_m$, each presented as a rational-weighted sum of squares (4.2), such that the identity

$$p(\mathbf{x}) = \sigma_0(\mathbf{x}) + \sigma_1(\mathbf{x})p_1(\mathbf{x}) + \cdots + \sigma_m(\mathbf{x})p_m(\mathbf{x})$$

holds exactly in $\mathbb{Q}[\mathbf{x}]$. Then for every $\mathbf{x}$ with $p_1(\mathbf{x}) \geq 0, \ldots, p_m(\mathbf{x}) \geq 0$ we have $p(\mathbf{x}) \geq 0$; that is, the entailment $p_1 \geq 0, \ldots, p_m \geq 0 \models p \geq 0$ holds.

Proof. Fix $\mathbf{x}$ with every $p_i(\mathbf{x}) \geq 0$. Each $\sigma_i(\mathbf{x}) = \sum_k c_{i,k}\ell_{i,k}(\mathbf{x})^2 \geq 0$ because $c_{i,k} \geq 0$. Hence each product $\sigma_i(\mathbf{x})p_i(\mathbf{x}) \geq 0$ for $i \geq 1$, and $\sigma_0(\mathbf{x}) \geq 0$. Evaluating the identity at $\mathbf{x}$, which is legitimate because it holds in $\mathbb{Q}[\mathbf{x}]$, writes $p(\mathbf{x})$ as a sum of nonnegative terms, so $p(\mathbf{x}) \geq 0$. □

Remark 6 The theorem is one-directional. It does not say that a true entailment always yields a repairable certificate. The SOS relaxation may be too conservative, the degree d may be too low, or the residual term may fail to be a sum of squares at all. Section 4.8 discusses these. The theorem says only that if the checker accepts the identities, the entailment is true, which is exactly what certification needs.

Corollary 1 If $\Delta \equiv 0$ and every Q_i^{rat} is diagonally dominant after the extraction of Lemma 7, then no repair is needed and the rationalized solution already satisfies Theorem 6 with $\sigma_i = \sigma_i^{\mathrm{rat}}$.

4.5 Machine-Checked Verification

The certificate is small, structured data, and the checks of Section 4.2.1 are simple enough to run in a short program. The two requirements are an exact polynomial identity and the nonnegativity of each weighted sum of squares. Both are decided by collecting rational coefficients and inspecting signs. For the highest level of assurance we also prove them in the `Lean 4` theorem prover [59] with the `Mathlib` library [86].

In `Lean` the identity $p - \sigma_0 - \sum_i \sigma_i p_i \equiv 0$ is closed by the ring-normalization tactic (`ring`/`ring_nf`), which decides equalities in a commutative ring by reducing both sides to a normal form; over $\mathbb{Q}$ this

is exact. The nonnegativity of each $\sigma_i = \sum_k c_{i,k}\ell_{i,k}^2$ follows by combining the library fact that a square is nonnegative with the nonnegativity of the rational weights, which the `positivity` tactic proves once the explicit squares from Lemma 7 are supplied. We export the certificate as a structured record holding the state variables, the basis $m(\mathbf{x})$ with rational coefficients, and the list of weighted squares for each σ_i, and a small generator turns this record into the `Lean` definitions and the two goals above. This is similar to the approach used by recent `Lean 4` sum-of-squares tooling [46, 47], in which an untrusted external solver outputs the certificate and `Lean` re-checks it. Our contribution is the repair step as the certificate carries a DSOS witness in place of the solver's Gram matrix, so it can be checked even when the raw solver output cannot.

This layer is optional. The standalone check suffices for the results in this thesis; `Lean` is reserved for the cases where neither the script nor its runtime should be trusted.

4.6 Case Study: Repairing a Successive Barrier Certificate

We implemented the pipeline in Julia, using `DynamicPolynomials` for polynomial algebra, `JuMP` for modeling, `GLPK` for the floating-point LP, and `CDDLib` with `Rational{BigInt}` arithmetic for the exact rational LP. The synthesis itself uses `SumOfSquares.jl`, as in the rest of the thesis.

4.6.1 A scalar example

Consider the polynomial $p(x) = x^4 + 2x^2 + 1 = (1 + x^2)^2 \geq 0$. With basis $m(x) = (1, x, x^2)^\top$, coefficient matching gives

$$Q_{11} = 1, \quad 2Q_{12} = 0, \quad Q_{22} + 2Q_{13} = 2, \quad 2Q_{23} = 0, \quad Q_{33} = 1.$$

One feasible Gram matrix is

$$Q_1 = \begin{pmatrix} 1 & 0 & \frac{1}{2} \\ 0 & 1 & 0 \\ \frac{1}{2} & 0 & 1 \end{pmatrix},$$

which is positive definite. The same polynomial also admits the diagonally dominant representative

$$Q^* = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 1 \end{pmatrix}, \qquad m(x)^\top Q^* m(x) = 1 + 2x^2 + x^4 = (1 + x^2)^2.$$

This example shows the kernel freedom used by the repair. The coefficient of x^2 is fixed only through $Q_{22} + 2Q_{13} = 2$, so the Gram representative can move from $(Q_{22}, Q_{13}) = (1, 1/2)$ to $(0, 1)$ without changing the polynomial. The repaired matrix is singular but diagonally dominant, and Lemma 7 gives the explicit rational square $(1 + x^2)^2$.

4.6.2 Repairing barrier Gram matrices

The Van der Pol successive barrier certificate from Chapter 3 produced 60 Gram matrices, of sizes 1×1, 3×3, and 6×6. All were positive semi-definite as returned, but most only marginally: their smallest eigenvalues ranged from about 10^{-16} to order one, clustered near 10^{-12}. In the language of Section 4.2.3 this is the signature of near-failure of strict feasibility: the solutions sit almost on the boundary of the PSD cone, which is both why the Cholesky test of Section 4.1 is sensitive to precision here and why rounding so easily makes these blocks not PSD. To stress the exact repair we perturbed each matrix shifting its spectrum so that $\lambda_{\min} = -10^{-6}$, and then ran the diagonally-dominant projection in exact rational arithmetic.

The exact repair returned an optimal diagonally dominant matrix for all 60 inputs. By

Gram block size	count	exact repair	repaired $\lambda_{\min}$
1×1	14	optimal	$= 0$ (exact)
3×3	30	optimal	≥ 0 (exact)
6×6	16	optimal	≥ 0 (exact)

Table 4.1: Exact diagonally-dominant repair of the 60 Van der Pol barrier Gram matrices after each was forced to $\lambda_{\min} = -10^{-6}$. Every repair succeeded and produced an exactly positive semi-definite matrix, where a floating-point repair leaves small negative eigenvalues in the ill-conditioned cases.

Lemma 6 every repaired matrix is positive semi-definite, and many sit on the boundary of the cone with $\lambda_{\min} = 0$ exactly, the expected outcome of a minimal repair. The value of working over $\mathbb{Q}$ is visible in the ill-conditioned cases. Take one of the 6×6 matrices as an example. The floating-point repair returned a matrix that had smallest eigenvalue around -5×10^{-8}, a double-precision artifact rather than a real failure to repair. The exact rational repair of the same matrix left a smallest eigenvalue of order 10^{-20}, which is zero to within the precision of the eigenvalue computation. This is the case the pipeline is built for. The exact solve removes the residual floating-point error that a numerical repair leaves behind, turning an "almost positive semi-definite" matrix into one that is provably positive semi-definite over $\mathbb{Q}$. Table 4.1 summarizes the results.

In the full pipeline the repair must be carried out inside the affine subspace $\mathcal{A}(Q) = c$ that fixes the polynomial coefficients. This distinction is important. Projecting a perturbed matrix into the DD cone is not enough by itself; the repaired matrix must still represent the intended polynomial exactly.

Remark 7 A caveat on scale. The blocks reported here are small; the largest is 6×6. For a barrier the demanding condition is consecution (the Lie-derivative inequality), whose Gram block grows with the product of the barrier degree and the vector-field degree, the same blocks that Roux et al. found too large for exact rational recovery [75]. Our refit is an exact rational *LP* rather than the exact rational *SDP* those recovery methods solve, which lowers the per-step cost, but the rational arithmetic still grows with block size, and we have not yet stress-tested the consecution blocks at

high degree. We regard scalability to those blocks as the main open question for the method.

4.7 Relation to the Padding Approach

Roux et al. [75] propose a different and cheaper route to a verified SDP, and it helps to contrast it with ours. Rather than recover an exact solution, they pad the problem. They add a small positive constant c to the target polynomial so that the padded problem becomes strictly feasible, and then prove that an exact solution exists near the numerical one, without computing it, using a verified floating-point Cholesky test. The padding constant c plays the same role as the synthesis margin ρ we recommend below.

The two methods answer different questions. Padding can certify soundness cheaply when its hypotheses and error bounds are satisfied. It proves the *existence* of an exact witness close to the numerical one but does not produce the explicit witness used here. Our repair is more expensive and more conservative, but it produces the witness: an explicit rational sum of squares that a separate checker, or Lean, can re-verify. When the goal is a transferable, machine-checkable proof object, construction is necessary and existence does not suffice; when the goal is only to decide soundness, padding is the better tool. Also padding, equivalently a margin ρ, moves the solution into the interior of the cone, which is where the repair of Section 4.3 is most likely to find a diagonally dominant representative.

4.8 Limitations

The method is conservative, and its limitations are discussed below.

The first limitation is cone conservatism. The repair certifies a term only when it lies in the DD cone for the chosen basis. By (4.4) these are strict subsets of the SOS cone, so there are sum-of-squares terms the repair cannot represent without raising the degree of $m(\mathbf{x})$ or returning to a full SDP. The gap can appear even at degree two: as Remark 4 shows, the square $(1 + x + y)^2$ is neither DSOS nor SDSOS in its basis. In our Van der Pol blocks the DD cone sufficed, but that is an empirical observation, not a guarantee; the fallback when DD repair fails is a higher-degree basis,

facial reduction [75], or a full SDP.

The second limitation is more fundamental, since it bounds what any SOS-based method can do. Not every nonnegative polynomial is a sum of squares. Hilbert showed that nonnegativity and SOS coincide only for univariate polynomials, quadratics, and bivariate quartics; outside these cases there are nonnegative polynomials that are not SOS, one being Motzkin's polynomial [58, 74]

$$M(x,y) = x^4y^2 + x^2y^4 - 3x^2y^2 + 1 \ \geq 0, \qquad M \notin \mathsf{SOS}[x,y].$$

No repair that adds a sum-of-squares slack can certify such a polynomial directly. The SOS cone is closed and M lies at a positive distance from it, so a small SOS term cannot move a fixed non-SOS polynomial into a cone it is outside of. The classical remedies remain available. Use denominator or multiplier constructions as in Hilbert's seventeenth problem and the Artin-Reznick theory [13, 73]; restrict to a compact semialgebraic domain where Positivstellensatz representations such as Putinar's apply [70]; or prove the inequality with a strict margin where one exists. The repair targets the common case in which the synthesized term is SOS, or becomes SOS after a tiny margin.

The third limitation is growth in the bit-size of the rationals. The continued-fraction rounding of Section 4.2.2 returns the smallest-denominator rational within tolerance, but the smallest available denominator can still be large when the solver value is badly conditioned or effectively irrational; loosening the tolerance trades accuracy for size and may reintroduce a nonzero residual. A large residual can in turn force a higher-degree slack. Both effects enlarge the coefficients, which slows the exact LP and the `Lean` ring-normalization check, since their cost grows with that bit-size. The mild remedies are the ones already noted: scale each identity so its coefficients are comparable before rounding, synthesize with a small margin $\rho > 0$ so the solution sits in the interior of the cone rather than on its boundary where conditioning is worst, and split a large proof into one lemma per identity so no single ring normalization is overwhelmed.

Finally, the certificate proves the polynomial entailments exactly, and nothing more. It does not establish that the polynomial model is faithful to the physical system, that the domain description

is correct, or that the chosen template was appropriate. Those remain modeling assumptions, as in any SOS-based verification.

4.9 Summary

We began by recalling how the barrier results in this thesis are certified: we output the multipliers σ_i, compute the residue, and test each Gram matrix Q_i for positive semi-definiteness with a 512-bit Cholesky decomposition. That procedure is exact only in the limit of infinite precision, and the obvious upgrade to rational arithmetic is blocked by the square roots in Cholesky. This chapter removed that block. We rationalize the solver output, measure the failure of the exact identity with a residual polynomial Δ, and, when rounding breaks positive semi-definiteness or the identity, re-fit the offending Gram matrices inside the diagonally dominant cone. Diagonal dominance both guarantees positive semi-definiteness, without square roots (Lemma 6), and yields an explicit rational-weighted sum of squares, again without square roots (Lemma 7). The re-fit is a rational linear program, solvable exactly (Proposition 1), and the resulting certificate is sound (Theorem 6) and checkable by a short program or by `Lean 4`. The Van der Pol case study reports a prototype repair on the barrier Gram matrices and identifies the exact certificate check that must accompany any such numerical result. The method does not overcome the gap between nonnegativity and sums of squares. But for the common case where synthesis succeeds and only the numerics fail, it turns a high-precision floating-point check into an exact, auditable proof.

Chapter 5

Successive Control Lyapunov Functions

"A journey of a thousand miles begins with a single step."
— Lao Tzu

This chapter develops the reachability counterpart of Chapter 3: given a target set, find a large region from which every trajectory reaches the target in finite time while remaining in the region. A single control Lyapunov function yields a conservative region of attraction, so the chapter fixes the input to a finite set, justified by the finite-control-set theorem, and synthesizes a hierarchy of Lyapunov-like functions in which each level treats the union of all previously certified regions as its new target. The cumulative region is substantially larger than any single CLF's, and a state-dependent switching controller with finitely many switches drives the system level by level into the target. The method is evaluated on nonlinear benchmarks, where it compares favorably with neural CLF baselines.

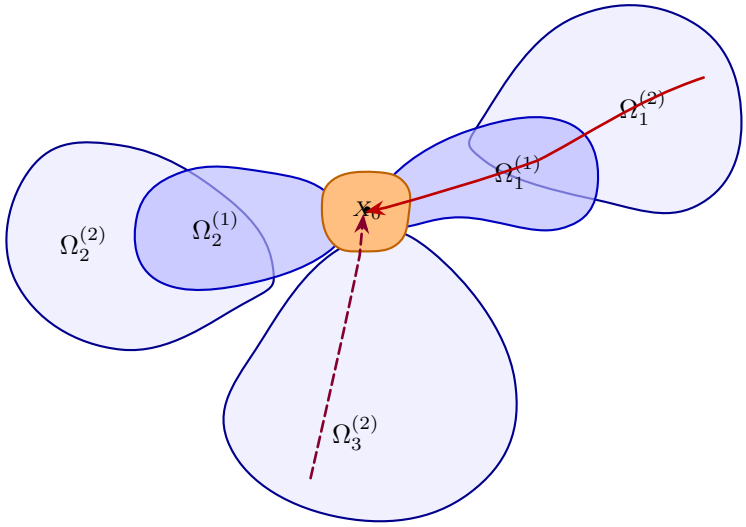


Figure 5.1: Illustration of successive control Lyapunov-Like functions and the set of nested regions associated with each. The controller for reaching target X_0 is synthesized by switching between multiple controllers, each with a region of attraction and corresponding Lyapunov-like function.

5.1 Introduction

Chapter 3 applied the successive paradigm to *safety*: a hierarchy of barrier functions enlarging a controlled-invariant set. This chapter develops its stability counterpart, the *reachability* specification of Section 2.2 in which a hierarchy of control Lyapunov-like functions progressively enlarge the region of attraction of a target set. It is the "3 $\rightarrow$ 5" edge of the dissertation's dependency structure.

Formally, given $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ and a target X_0, we seek a region $\Omega^* \supseteq X_0$ and a feedback such that every trajectory initialized in Ω^* reaches X_0 in finite time while remaining in Ω^*. It is the signal-temporal-logic property "$\Omega^* \mathsf{until} X_0$" of Section 2.2, with Ω^* as large as possible. A single control Lyapunov function (Definition 7) yields a conservative region of attraction, and the joint search for the certificate and the feedback is bilinear (Section 1.3). We resolve both issues exactly as in Chapter 3: we *fix* the control input to a finite set, so each level is a convex SOS feasibility problem, justified by the finite-control-set theorem (Theorem 2). We *combine* the resulting fixed-input regions, treating the union of all previously certified regions as the new target at each level. The cumulative region $\Omega^* = \bigcup_i \Omega_i$ is substantially larger than any single CLF's region of attraction.

We address the difficulties of CLF synthesis through multiple and successive control Lyapunov-Like functions,[1] extending our previous work on successive barrier functions [92]. The key idea is to synthesize a hierarchy of Lyapunov-like functions $V_1, \ldots, V_k$ associated with corresponding domains $\Omega_1, \ldots, \Omega_k$, such that each function V_i can be used to derive a control strategy that guarantees that a system initialized in the region Ω_i will eventually reach Ω_{i-1} for $i \geq 1$ with $\Omega_0 = X_0$, the target region. In other words, we decompose the over-arching temporal logic property "Ω^* until X_0 into smaller "steps" Ω_{i+1} until Ω_i, and in this process achieve a cumulative region of attraction $\Omega^* = \bigcup_{i=1}^{k} \Omega_i$, as illustrated in Figure 5.1. However, the key tradeoff is that each function V_i is synthesized by fixing the control input $\mathbf{u} = \mathbf{u}_i$ for a fixed control input sampled from the space of control inputs. As a result, our approach ensures that each region and Lyapunov function can be synthesized using sum-of-squares (SOS) programming without resorting to bilinear problem formulations.

Contributions of this chapter.

- The successive-CLF framework and the conditions under which a hierarchy of Lyapunov-like functions combine to guarantee reachability of X_0;
- An SOS procedure that synthesizes each level for a fixed control input and iteratively extends the region;
- A *control-quantization* analysis establishing that fixing finitely many inputs and switching among them *finitely* often loses no generality; and
- An empirical study on nonlinear benchmarks, including polynomial-approximated trigonometric dynamics, a wheeled path-follower, and the Caltech ducted fan, with neural-certificate comparisons.

[1] Throughout this chapter, we will study Lyapunov-like function that satisfy many of the same properties of Lyapunov functions, but with some key differences in order to establish reaching a target set rather than asymptotic stability

5.1.1 Related Work

Control Lyapunov Functions (CLFs): The concept of control Lyapunov functions for stabilizability was formalized in the 1980s by Artstein [7] following ideas originally proposed by Sontag. A CLF not only certifies the stability of a closed-loop system but also defines a feedback control law to stabilize the system [80]. Early results showed that the existence of a smooth CLF on a given domain leads to the construction of a stabilizing feedback controller. However, searching for a valid CLF for a nonlinear system is nontrivial. Analytical methods for constructing CLFs only exist for special classes of systems, such as linear systems via LQR design, or special classes of nonlinear systems through backstepping. In the general case, one normally resorts to numerical or computational approaches to find valid CLFs. Our work continues this tradition by employing sum-of-squares programming to search for polynomial Lyapunov functions. Unlike classical approaches, we do not stop at a single Lyapunov function and propose sequentially synthesizing them to enlarge the region of attraction.

Sum-of-Squares (SOS) methods for CLF synthesis: The past two decades have seen the emergence of sum-of-squares optimization as a powerful technique for verifying the stability and safety of polynomial dynamical systems. By formulating the Lyapunov inequalities as SOS constraints, one can use semi-definite programming to search for polynomial Lyapunov functions [64]. Several attempts have been made to estimate regions of attraction (ROA) for polynomial dynamical systems using this approach [36, 69]. These techniques have also been applied to high-dimensional robotics problems by decomposition or restricting to lower-degree polynomials [50].

Constructing multiple CLFs: A prominent approach in the robotics context is the LQR-trees framework [85], where a library of local LQR-stabilized trajectories is constructed, and SOS verification is used to compute each trajectory's ROA by combining many local "funnels". This results in covering a large portion of the state space with certified regions. Our approach is conceptually similar; however, we don't rely on system linearization and do not explicitly compute trajectories that need stabilization. We directly enlarge the invariant set by computing successive CLFs in the

regions not covered by previous ones.

Multiple Lyapunov functions and switching: Employing multiple Lyapunov functions to stabilize hybrid/switched systems under switching has been studied well [15, 41]. The main idea is to have separate CLFs for each mode and ensure the decrease condition for each of them via dwell-time, hysteresis, or state-dependent conditions. Another notion is that of *patchy* CLFs, where an ordered family of local CLFs is defined over overlapping domains with compatibility conditions to enable a hybrid stabilizing feedback law [30]. This approach is somewhat similar to ours in spirit, but ours differs in the sense that it is guaranteed to make progress on the region of attraction due to the successive hierarchy. Another orthogonal paradigm of studying asymptotic stability is that of using Matrosov-type auxiliary functions. If the Lyapunov decrease condition is only negative semidefinite, the primary Lyapunov function is augmented with an ordered set of secondary functions whose derivatives satisfy a nested implication structure [45]. The concept has also been extended to hybrid systems where regularity assumptions counter any Zeno behavior [76]. In comparison, our successive CLFs enforce a strict decrease on each Lyapunov function, and they are neither required to be explicitly nested, nor are the lower-level functions required to fully cover the target set.

Hamilton-Jacobi (HJ) Reachability: An alternate formal method to compute reachable sets in nonlinear systems is via HJ reachability analysis. This approach formulates the reachability problem as a Hamilton-Jacobi-Isaacs partial differential equation, computing a value function whose sublevel sets correspond to reachable sets [56]. Tools based on HJ reachability can compute backward reachable sets, which are regions of attraction under optimal control. However, these approaches suffer from the curse of dimensionality and become intractable for higher-dimensional nonlinear dynamical systems. Our successive CLF approach, though not optimal, tries to alleviate the high computational cost by relying on iterative computation.

Whereas CLFs are used to guarantee convergence, Control Barrier Functions (CBFs) are employed to certify invariance in a safety context [4, 68]. While CBFs also face synthesis challenges, there has been progress on using data-driven or SOS-based methods for their synthesis. There already exists a concept of *successive control barrier functions* [92]. Our successive CLF approach is

parallel to that, except that we target reachability instead of safety.

5.2 Lyapunov-Like Functions and Reachability

We recall from Section 2.3.1 the Lyapunov-like function (Definition 3) and the control Lyapunov function (Definition 7), together with the domain $D = B_R(\mathbf{x}^*)$, the boundary level $d_V = \inf_{\partial D} V$, and the sublevel set $\Omega_V = \{\mathbf{x} \in D : V \leq \gamma_V\}$ for $0 < \gamma_V < d_V$.

If V is a valid Lyapunov-like function, then Ω_V is an invariant region of attraction for the closed loop system $\dot{\mathbf{x}} = f(\mathbf{x}, \kappa(\mathbf{x}))$, as shown by the following theorem.

Theorem 7 If V is a polynomial Lyapunov-like function (Definition 3) for a fixed control feedback law κ, then the set Ω_V has the following properties:

(1) Ω_V is a non-empty and compact set and $\mathbf{x}^* \in \Omega_V$;

(2) $\Omega_V \cap \partial D = \emptyset$ (thus $\Omega_V \subsetneq D$);

(3) For any trajectory $\mathbf{x}(t)$ of the closed loop system such that $\mathbf{x}(0) \in \Omega_V$, there exists a time $t^* < \infty$ such that $\mathbf{x}(t^*) \in X_0$ and for all $s \in [0, t^*]$, $\mathbf{x}(s) \in \Omega_V$ (finite time reachability).

Proof. (1) Nonemptiness follows because $V(\mathbf{x}^*) = 0$ and therefore, $\mathbf{x}^* \in \Omega_V$. Compactness follows because Ω_V is a closed set, D is compact, and $\Omega_V \subseteq D$.
(2) $\forall \mathbf{x} \in \partial D$, we have $V(\mathbf{x}) \geq d_V > \gamma_V$. On the other hand, for all $\mathbf{x} \in \Omega_V$, $V(\mathbf{x}) \leq \gamma_V$. Therefore, $\Omega_V \cap \partial D = \emptyset$. This also establishes that $\Omega_V \subsetneq D$, since $\Omega_V \subseteq D$ by definition.
(3) Let $\mathbf{x}(0) \in \Omega_V$ and let $\hat{X}_0$ be the topological interior of X_0. If $\mathbf{x}(0) \in X_0$, then the statement is trivially true for $t^* = 0$. Consider the case wherein $\mathbf{x}(0) \in \Omega_V \setminus X_0$. Let's define $T = \{t \mid t > 0 \wedge \forall s \in [0, t),\ \mathbf{x}(s) \in \Omega_V \setminus X_0\}$.

- Clearly $0 \in T$ since $\mathbf{x}(0) \in \Omega_V \setminus X_0$. We now show that T contains an interval $[0, t_j)$ for some $t_j > 0$. First, $\mathbf{x}(t)$ is a twice differentiable function and so is V and therefore, $V(\mathbf{x}(t)) = V(\mathbf{x}(0)) + t \times dV/dt + t^2/2 \times (d^2V)/(dt^2)|_{\mathbf{x}=\mathbf{x}(s), s\in[0,t)}$ by Taylor series expansion.

Since $\dot{V}(\mathbf{x}(0)) \leq -\xi$, we conclude that there exists $t = t_0 > 0$, $\forall s \in [0, t_0], V(\mathbf{x}(s)) \leq \gamma_V$, and thus $\mathbf{x}(s) \in \Omega_V$, for all $s \in [0, t_0]$. Also, $\mathbf{x}(0) \notin X_0$, and since X_0 is closed, there is an open neighborhood U containing $\mathbf{x}(0)$ such that $U \cap X_0 = \emptyset$. Combining the two observations, we conclude that at some time $t_j = t_0/2^j$, we will have $\mathbf{x}(s) \in \Omega_V \setminus X_0$ for all $s \in [0, t_j)$.

- Next, we observe that $t^* = \sup T < \infty$. For each $t \in T$,

$$V(\mathbf{x}(t)) \leq V(\mathbf{x}(0)) \ + \int_0^t \dot{V}(\mathbf{x}(s), \kappa(\mathbf{x}(s)))ds \leq V(\mathbf{x}(0)) - \xi t.$$

  Since V is positive semi-definite, it follows that t^* must be finite and furthermore by continuity of V, we obtain $V(\mathbf{x}(t^*)) \leq \gamma_V$ and therefore $\mathbf{x}(t^*) \in \Omega_V$.

- We conclude that $\mathbf{x}(t^*) \in X_0$. Thus, we obtain a time $t^* < \infty$ such that for all $s \in [0, t^*)$, $\mathbf{x}(s) \in \Omega_V \setminus X_0$ and $\mathbf{x}(t^*) \in X_0 \cap \Omega_V$.

□

As a result, a Lyapunov-like function V over a domain $D \supseteq X_0$ under a feedback law $u = \kappa(\mathbf{x})$ defines a sub-level set $\Omega_V = \{\mathbf{x} \mid V(\mathbf{x}) \leq \gamma_V\}$ such that $\Omega_V \subset D$, $\Omega_V \cap X_0 \neq \emptyset$ and every trajectory initialized in Ω_V under the feedback law κ satisfies the temporal logic property "Ω_V until X_0".

5.3 Successive Lyapunov-Like Functions

In this section, we describe the overall approach of successive control Lyapunov-like functions, which is the main contribution of this paper. We fix a differential equation model $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$, a target set X_0 with $\mathbf{x}^* \in X_0$, and a finite set of feedback laws $U_f = \{\kappa_1, \ldots, \kappa_j\}$ with each $\kappa_i : X \to U$ as a smooth function. The choice of the feedback functions will be discussed in Section 5.4.

5.3.1 Successive Lyapunov-Like Functions Strategy

The strategy involving successive functions starts with the given target region X_0 and a domain D to compute a region $\Omega_1 \subseteq D$ from which all trajectories can be controlled using a feedback law

from U_f to reach X_0, while remaining in Ω_1. We then treat Ω_1 as our new target in order to compute Ω_2 which guarantees that trajectories initialized inside it will eventually reach Ω_1. Iterating this process, we can treat $\bigcup_{j\leq i}\Omega_j$ as the target set to obtain a new region Ω_{i+1}, such that starting from any state in Ω_{i+1}, we have a control strategy such that the trajectory remains inside Ω_{i+1} until we reach Ω_j for $j \leq i$. Thus, $\bigcup_j \Omega_j$ denotes a region wherein we have a control strategy that switches between feedback functions in the set U_f to reach the target region X_0.

Initial Expansion: Starting with the target set X_0 and a domain $D = \{\mathbf{x} \mid \|\mathbf{x} - \mathbf{x}^*\| \leq R\} \supset X_0$ for a fixed radius R, we synthesize multiple Lyapunov-like functions, each with an associated control in the finite set U_f.

(1) Lyapunov-like functions $V_1^{(1)}, \ldots, V_{n_1}^{(1)}$ satisfying the conditions of Definition 3,

(2) These functions are associated with control policies $\kappa_1^{(1)}, \ldots, \kappa_{n_1}^{(1)} \in U_f$, wherein $\kappa_i^{(1)}$ corresponds to the function $V_i^{(1)}$, and

(3) Sub-level sets $\Omega_1^{(1)}, \ldots, \Omega_{n_1}^{(1)} \subseteq D$, wherein each $\Omega_i^{(1)} = \{\mathbf{x} \mid V_i^{(1)}(\mathbf{x}) \leq \gamma_i^{(1)}\}$.

Assuming polynomial dynamics and feedback laws, we can automate the synthesis through SOS programming techniques, as described in Section 2.5. Let $\Omega_1 = \Omega_1^{(1)} \cup \cdots \Omega_{n_1}^{(1)}$ be the union of the regions for each Lyapunov-like function.

Lemma 8 For any state $\mathbf{x}_0 \in \Omega_1$, there exists a control strategy $\kappa \in U_f$ such that the closed loop system $\dot{\mathbf{x}} = f(\mathbf{x}, \kappa(\mathbf{x}))$ initialized at $\mathbf{x}(0) = \mathbf{x}_0$ will (a) reach the target region X_0 at some finite time $T \geq 0$ and (b) remain inside the set Ω_1 for all times $t \in [0, T)$.

Proof. Since $\mathbf{x}_0 \in \Omega_1$, we know that $\mathbf{x}_0 \in \Omega_1^{(i)}$ for some i. The corresponding Lyapunov-like function $V_i^{(1)}$ and feedback $\kappa = \kappa_i^{(1)}$ certify that (a) and (b) will hold as a direct consequence of Theorem 7. □

Extending Ω_1: For the next successive function, the new target region becomes $X_1 = X_0 \cup \Omega_1$. We repeat the process of synthesizing Lyapunov-like functions $V_1^{(2)}, \ldots, V_{n_2}^{(2)}$ satisfying Def. 3, associated control policies $\kappa_1^{(2)}, \ldots, \kappa_{n_2}^{(2)}$ and regions of attractions $\Omega_1^{(2)}, \ldots, \Omega_{n_2}^{(2)}$.

Repeating this process, for the $(i+1)^{th}$ step, we set the target region $X_i = \bigcup_{j=1}^{i} \Omega_j \cup X_0$ and synthesize Lyapunov-like functions $V_l^{(i+1)}$ with corresponding controls $\kappa_l^{(i+1)} \in U_f$ and region of attraction $\Omega_l^{(i+1)}$ for $l = 1, \ldots, n_{i+1}$.

Lemma 9 For each $i \geq 1$ and $l \in [1, \cdots, n_i]$, suppose we choose an initial state $\mathbf{x}(0) \in \Omega_l^{(i)}$ and suppose we apply the control feedback $\kappa_l^{(i)}$, there exists a time $t \geq 0$, such that $\mathbf{x}(t) \in X_{i-1}$ and $\mathbf{x}(s) \in \Omega_l^{(i)}$ for all $s \in [0, t)$.

Proof. This is a direct corollary of Theorem 7, applied to the Lyapunov-like function $V_l^{(i)}$. □

5.3.2 Synthesizing Successive Lyapunov-like Functions

The synthesis of successive Lyapunov-like (SLL) functions proceeds by using the SOS programming approach described in Section 2.5, assuming polynomial dynamics, polynomial feedback laws and X_0 described by a basic semi-algebraic set. Our iterative approach ensures that each X_i is represented as the union of basic semi-algebraic sets:

$$X_i = X_0 \cup \bigcup_{j=1}^{N_i} \{\mathbf{x} \in D \mid V_j(\mathbf{x}) \leq \gamma_j\} \ .$$

Let $X_0 = \{\mathbf{x} \in D \mid g_0(\mathbf{x}) \geq 0\}$. Here V_j represents one of the SLL functions $V_i^{(l)}$ with an appropriate γ_j chosen so that Theorem 7 holds (Section 5.5 discusses the choice of γ_j). Since the set $D = \{\mathbf{x} \mid \|\mathbf{x} - \mathbf{x}^*\|^2 \leq R^2\}$ is a basic semi-algebraic set, we can write the closure of the set $D \setminus X_i$ as a basic semi-algebraic set:

$$\mathsf{cl}(D \setminus X_i) = \left\{ \mathbf{x} \;\middle|\; \begin{array}{c} \|\mathbf{x} - \mathbf{x}^*\|^2 \leq R^2 \ \wedge \\ \bigwedge_{j=1}^{N_i} \ V_j(\mathbf{x}) \geq \gamma_j \ \wedge \\ g_0(\mathbf{x}) \leq 0 \end{array} \right\} . \tag{5.1}$$

Algorithm 3: Algorithm for successive Lyapunov-like function computation.

Input: System Σ, domain D, finite control set U_f, initial target region X_0, degree bound d, max iterations M

Output: $S = \{(V_j^{(i+1)}, \kappa_j^{(i+1)}, \gamma_j^{(i+1)})\}_{j=1}^{N_i}$

1 $S \leftarrow \emptyset$
2 $X_0 \leftarrow \{\|\mathbf{x} - \mathbf{x}^*\|^2 - R^2 \leq 0,\ g_0 \geq 0\}$
3 **for** $i = 0$ **to** M **do**
4 $\quad X_i \leftarrow X_0 \cup \{V \leq \gamma_V \mid (V, \kappa, \gamma_V) \in S\}$
5 $\quad$ **foreach** $\kappa \in U_f$ **do**
6 $\quad\quad$ Find polynomial V of degree d such that Eq. (5.2) holds for $\mathbf{u} = \kappa(\mathbf{x})$ and target region X_i
7 $\quad\quad$ **if** *a polynomial* V *is found* **then**
8 $\quad\quad\quad$ Find $d_V \leq \inf_{\mathbf{x} \in \partial D} V(\mathbf{x})$
9 $\quad\quad\quad$ Choose $\gamma_V \in (0, d_V)$
10 $\quad\quad\quad$ $S \leftarrow S \cup \{(V, \kappa, \gamma_V)\}$
11 **return** S

In order to find a new SLL function, we reformulate the SOS feasibility problem as follows:

$$\left.\begin{array}{l} V - \epsilon\|\mathbf{x} - \mathbf{x}^*\|_2^2 \ \mathsf{SOS} \\ -\xi - \dot{V} + \sigma_0 g_0 + \pi(\|\mathbf{x} - \mathbf{x}^*\|^2 - R^2) + \sum_{j=1}^{N_i} \sigma_j(\gamma_j - V_j) \ \mathsf{SOS} \\ \sigma_0, \sigma_1, \ldots, \sigma_{N_i}, \pi \ \mathsf{SOS}\,. \end{array}\right\} \tag{5.2}$$

Notice that the second SOS constraint represents the "positivstellensatz encoding" for the condition $\mathbf{x} \in \mathsf{cl}(D \setminus X_i) \ \Rightarrow\ \dot{V} \leq -\xi$. Here we write $V = \sum_{k=1}^{M} c_k \phi_k$ for a basis function ϕ_k.

Algorithm 4 presents the basic scheme for computing successive Lyapunov functions. Each iteration starts with a target region X_i as defined above and running through each control feedback law in our set while solving the SOS problem in Eq. (5.2). If a feasible solution is found, we compute the largest sub-level set that lies within D by solving the optimization problem $\inf_{\mathbf{x} \in \partial D} V(\mathbf{x})$. A lower bound to this optimum d_V can be found using SOS programming (line 8), as well. We choose $\gamma_V < d_V$ as the appropriate sub-level set.

The concept of these successive functions can be iterated. Let $\mathcal{V}^{(0)}$ be the initial set of control Lyapunov functions. We can compute a successive CLF $\mathcal{V}^{(1)}$ which satisfies the derivative condition

only when $\mathbf{x} \notin \Omega^{(0)}$. Next, we can use this to derive another function $\mathcal{V}^{(2)}$ having multiple polynomial Lyapunov functions, where again for each of them the derivative condition holds only if both $\mathbf{x} \notin \Omega^{(0)}$ and $\mathbf{x} \notin \Omega^{(1)}$. This leads to a sequence of successive Lyapunov functions.

Algorithm 4: Algorithm for successive Lyapunov function computation

Data: System Σ, domain D, finite control set U_{fin}, previous successive CLF set $\mathcal{V} = \langle \mathcal{V}^{(0)}, \mathcal{V}^{(1)}, \ldots, \mathcal{V}^{(k)} \rangle$, constant $\epsilon > 0$, $\delta > 0$, degree bound d.
Result: $k+1$ level successive CLF $\mathcal{V}^{(k+1)}$.

1 $\mathcal{V}^{(k+1)} \leftarrow \infty$ /* Initialize to trivial function */
2 **for** $\hat{\mathbf{u}} \in U_{fin}$ **do**
3 | find polynomial V of degree d
4 | s.t. $(\forall \mathbf{x} \in D)\ V(\mathbf{x}) \geq \epsilon \|\mathbf{x} - \mathbf{x}^*\|_2^2$
5 | $(\forall \mathbf{x} \in D \setminus \Omega^{(k)}) \bigwedge_{l=1}^{k} V^{(l)} \geq \gamma^{(l)} \ \Rightarrow\ \nabla V \cdot (f(\mathbf{x}, \hat{\mathbf{u}})) \leq -\xi$
6 | $(\forall \mathbf{x} \in \partial D)\ V(\mathbf{x}) \geq \gamma + \delta$
7 | **if** *a polynomial V found* **then**
8 | | $\mathcal{V}^{(k+1)} \leftarrow \mathcal{V}^{(k+1)} \cup \{(V, \gamma, \hat{\mathbf{u}})\}$
9 **return** $\mathcal{V}^{(k+1)}$

Next, we outline an iterative scheme for successive Lyapunov function synthesis as depicted in Algorithm 4. This approach maintains a family of sets of successive Lyapunov functions, $\mathcal{V} = \langle \mathcal{V}^{(0)}, \mathcal{V}^{(1)}, \ldots, \mathcal{V}^{(k)} \rangle$. In the algorithm, $\mathcal{V}$ is initialized to be the *ancestors* (all the previous successive Lyapunov functions).

The iteration is performed on the control inputs $\mathbf{u}_i \in U_{fin}$. The algorithm attempts to synthesize a polynomial Lyapunov function V with maximum degree d such that there exists a constant $\epsilon > 0$ satisfying (a) $V(\mathbf{x}) \geq \epsilon \|\mathbf{x} - \mathbf{x}^*\|_2^2$ for all $\mathbf{x} \in D$, (b) for all $\mathbf{x} \in D \setminus \mathbf{x}^*$ there exists a control input $\mathbf{u} \in U$ such that $\nabla V \cdot (f(\mathbf{x}, \mathbf{u})) \leq -\xi$, and (c) we force the constraint $(\forall \mathbf{x} \in \partial D)\ V(\mathbf{x}) \geq \gamma$ which makes sure the corresponding region of attraction is inside the domain D. We use sum of squares (SOS) programming to find such a Lyapunov function (Lines 3-8). In case of a successful Lyapunov candidate, it is added to the list of successive Lyapunov functions (Line 8).

5.4 Control Quantization

In this section, we discuss a strategy for selecting a finite set of control laws, assuming that the control inputs $\mathbf{u} \in U$, wherein U is a compact set. We establish the following key result: let

us assume that there is a smooth control strategy $\mathbf{u} = \kappa(\mathbf{x})$ that guarantees that the target set X_0 is eventually reached while remaining inside a set Ω_V, as witnessed by a smooth Lyapunov-like function V satisfying the conditions of Def. 3. We demonstrate that there exists a finite subset $U_f \subseteq U$ and a switching strategy that switches between control inputs $\mathbf{u}_i \in U_f$ held constant for some minimum dwell time Δ_i, such that for any state $\mathbf{x}_0 \in \Omega_V$, the resulting closed loop trajectory reaches X_0 within *finitely many* control input switches. As a result, we view the selection of control laws as a process of *quantizing* the space of control inputs U.

In general, one may implement numerous schemes to select feedback laws. For instance, one may linearize the nonlinear dynamics around the state $\mathbf{x}^*$ and assuming that the linearized dynamics are controllable, we may use LQR with various costs to synthesize a set of control feedback functions κ_j. However, these functions are not necessarily guaranteed to respect the control input limits represented by the set U. Therefore, a (finite) quantization of the set U by suitably chosen set of samples $U_f \subseteq U$ yields feedback functions of the form $\kappa_i(\mathbf{x}) = \mathbf{u}_i$ for $\mathbf{u}_i \in U_f$ that simply holds the input constant.

Let us assume a smooth function $V(\mathbf{x})$ along with a differentiable control feedback $\mathbf{u} = \kappa(\mathbf{x})$ that satisfies the conditions in Def. 3 for sets X_0, D and constants $\epsilon > 0$ and $\xi > 0$.

Theorem 8 Suppose V is a smooth Lyapunov-like function satisfying the conditions of Def. 3, there exists a finite set $U_f \subseteq U$ such that for every $\mathbf{x} \in D \setminus X_0$, there exists $\underline{\mathbf{u} \in U_f}$ such that $\nabla V \cdot f(\mathbf{x}, \mathbf{u}) \leq -\xi/2$.

Proof. Since V is a smooth function, the function $\nabla V \cdot f(\mathbf{x}, \kappa(\mathbf{x}))$ is continuous and differentiable for all $\mathbf{x}$. Therefore, for any $\mathbf{x} \in D$ and $\mathbf{u} \in U$, there exists a Lipschitz constant $L > 0$ such that

$$\nabla V \cdot f(\mathbf{x}, \mathbf{u}) - \nabla V \cdot f(\mathbf{x}, \kappa(\mathbf{x})) \leq L\|\mathbf{u} - \kappa(\mathbf{x})\|.$$

Suppose we choose U_f to be a $\xi/(2L)$-packing of the set U so that for all $\mathbf{u} \in U$, there exists $\mathbf{u}_f \in U_f$ such that $\|\mathbf{u} - \mathbf{u}_f\| \leq \xi/2L$. For such a U_f, we can choose $\hat{\mathbf{u}} \in U_f$ such that $\|\hat{\mathbf{u}} - \kappa(\mathbf{x})\| \leq \xi/(2L)$.

Therefore,

$$\begin{aligned}\nabla V \cdot f(\mathbf{x}, \hat{\mathbf{u}}) &\leq \nabla V \cdot f(\mathbf{x}, \kappa(\mathbf{x})) + L\|\mathbf{u} - \kappa(\mathbf{x})\| \\ &\leq -\xi + L\xi/(2L) \leq -\xi/2\,.\end{aligned}$$

□

Remark 8 Note that if U is a polyhedral set and f is a control affine function, the set U_f can be chosen to be the set of vertices of the set U, providing a simpler alternative to the Lipschitz argument used in Theorem 8.

Theorem 8 provides us a set of inputs of the form $\kappa(\mathbf{x}) = \hat{\mathbf{u}} \in U_f$ that simply "quantize" the control input. However, it is still possible that we may need to switch infinitely often from this set to reach the set X_0 starting from some $\mathbf{x}_0 \in \Omega_V$. We will now rule out this possibility. For convenience, assume that the original CLF V satisfies Def. 3 for decrease constant 2ξ and the set U_f is chosen to yield a decrease with constant ξ following Theorem 8. We will establish a finite dwell time requirement and a "strict decrease" guarantee by exploiting the compactness of the domain D.

Let U_f be the finite set of quantized controls. Let $\mathbf{x}_0 \in \Omega_V \setminus X_0$ and $\mathbf{u}_i \in U_f$ be the input that satisfies the decrease condition: $\nabla V \cdot f(\mathbf{x}, \mathbf{u}_i) \leq -\xi$. For convenience, we write $V_{i,1} = \nabla V \cdot f(\mathbf{x}, \mathbf{u}_i)$ and $V_{i,2} = \nabla V_{i,1} \cdot f(\mathbf{x}, \mathbf{u}_i)$ be the second derivative. Since we have assumed the differentiability of V, f, we conclude that $V_{i,2}$ is continuous. Since D is compact, let $V_{i,2}(\mathbf{x}) \leq a_i$ for all $\mathbf{x} \in D$.

Lemma 10 For any $\mathbf{u}_i \in U_f$, there exists a dwell time $\tau_i > 0$ and a decrease constant $\Delta_i > 0$ such that for any $\mathbf{x}_0 \in \Omega_V \setminus X_0$ with $\mathbf{u}_i \in U_f$ satisfying the decrease condition $V_{i,1}(\mathbf{x}_0) = \dot{V}(\mathbf{x}_0, \mathbf{u}_i) \leq -\xi$, the time trajectory $\mathbf{x}(s)$ for the dynamics $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}_i)$ initialized at $\mathbf{x}(0) = \mathbf{x}_0$ satisfies $V(\mathbf{x}(\tau_i)) \leq V(\mathbf{x}_0) - \Delta_i$ and $V(\mathbf{x}(s)) \leq V(\mathbf{x}_0)$ for all $s \in [0, \tau_i)$.

Proof. Let $V_i(t) = V(\mathbf{x}(t))$, $V_{i,1}(t) = \nabla V \cdot f(\mathbf{x}(t), \mathbf{u}_i)$ and $V_{i,2}(t) = \nabla V_{i,1} \cdot f(\mathbf{x}(t), \mathbf{u}_i)$. Let $a_i = \max_{\mathbf{x} \in D} V_{i,2}(\mathbf{x})$.

Case-1: Assume $a_i > 0$. Let $\hat{\tau}_i = \frac{2\xi}{a_i}$. We will first establish that for all $s \in [0, \hat{\tau}_i]$, $V_i(s) \leq V_i(0) = V(\mathbf{x}(0))$. As a result, $\mathbf{x}(s) \in \Omega_V$ for $s \in [0, \hat{\tau}_i]$.

Define $T = \{t > 0 \mid \forall s \in [0, t),\ V_i(s) \leq V_i(0)\}$. T is non-empty, since V_i is differentiable, and $\dot{V}_i(0) \leq -\xi < 0$. Suppose $\sup T = \infty$, then $V_i(s) \leq V_i(0)$ for all $s > 0$. Otherwise, let $\tau = \sup T < \infty$. First, $V_i(\tau) = V_i(0)$, or else, we have some time $\tau' \in T$ such that $\tau' > \tau$ by the continuity of V_i. Using Taylor series expansion, we have

$$V_i(\tau) = V_i(0) + \tau \dot{V}_i(0) + \frac{\tau^2}{2} \ddot{V}_i(s)\,, \text{ for some } s \in [0, \tau)\,.$$

Therefore, $\tau \dot{V}_i(0) + \frac{\tau^2}{2} \ddot{V}_i(s) = 0$, and thus, $\tau = \frac{-2\dot{V}_i(0)}{\ddot{V}_i(s)} \geq \frac{2\xi}{a_i}$ since $\dot{V}_i(0) \leq -\xi$ and $0 < \ddot{V}_i(s) \leq a_i$ (this is because $s \leq \tau$, therefore $V_i(s) \leq V_i(0)$, and hence, $\mathbf{x}(s) \in \Omega_V$). Let $\hat{\tau}_i = \frac{2\xi}{a_i}$. We note that $\sup T \geq \hat{\tau}_i$. Let $0 \leq \tau \leq \hat{\tau}_i$. We have

$$V_i(\tau) = V_i(0) + \dot{V}_i(0)\tau + \frac{\tau^2}{2} \ddot{V}_i(s) \leq V_i(0) - \xi\tau + \frac{\tau^2}{2} a_i\,.$$

Choosing $\tau = \hat{\tau}_i/2 = \frac{\xi}{a_i}$, we obtain

$$V_i(\tau) \leq V_i(0) - \frac{\xi^2}{2a_i}\,.$$

Choosing $\tau_i = \frac{\xi}{a_i}$ and $\Delta_i = \frac{\xi^2}{2a_i}$, we conclude that $V_i(\tau_i) \leq V_i(0) - \Delta_i$, and furthermore, since $\tau_i \in T$, $V_i(s) \leq V_i(0)$ for all $s \in [0, \tau)$.

Case-2: Assume $a_i \leq 0$. Once again, we will define the set T as before and note that it is non-empty. Next, note that $\sup T = \infty$. Assuming otherwise, let $\tau = \sup T < \infty$. We note that $V_i(\tau) \leq V_i(0) - \xi\tau + \frac{\tau^2}{2} a_i < V_i(0)$. Therefore, there must be some time $\tau' > \tau$ such that $\tau' \in T$. Therefore, the result holds for any dwell time $\tau_i > 0$ and $\Delta_i = \xi\tau_i$.

□

Consider the following switched control strategy for a state $\mathbf{x}_i \in \Omega_V$:

(1) Choose a control input $\mathbf{u}_i \in U_f$ that satisfies the decrease condition at $\mathbf{x}_i$.

(2) Apply the control input $\kappa_i(\mathbf{x}) = \mathbf{u}_i$ for maximum time τ_i obtained in Lemma 10.

(3) If X_0 is not reached, iterate the control strategy from state $\mathbf{x}_{i+1}$ reached at time τ_i (Lemma 10 guarantees that $\mathbf{x}_{i+1} \in \Omega_V$).

We conclude that for any state $\mathbf{x}_0 \in \Omega_V$ finitely many iterations of the control strategy suffice to reach X_0.

Theorem 9 For any $\mathbf{x}_0 \in \Omega_V$, there is a finite number N of iterations of the control strategy before the trajectory reaches X_0.

In fact as a direct consequence of Lemma 10, we note that the control strategy ensures that the trajectory of the switched system remains in the set Ω_V until the set X_0 is reached.

As a result of Theorems 8 and 9, we conclude that if there is a CLF and a continuous feedback controller, then (a) there is a choice of a finite set of control inputs U_f, and (b) a switching strategy between the controls in U_f that in finitely many steps ensures that X_0 is reached starting from any state in Ω_V. In other words, the central idea of fixing finitely many control inputs and switching between them finitely many times does not lose generality.

5.5 Implementation

We now present some details of the implementation of our approach: in particular, (a) the "gradual" increase of the domain D to facilitate the synthesis of successive CLFs; and (b) the use of *test points* inside the desired region of attraction D^* to ensure appropriate choices for the functions $V_i^{(j)}$ from the SOS feasibility problems in Eq. (5.2).

5.5.1 Successive Domain Expansion

Thus far, we have kept the domain D fixed throughout the process of synthesizing successive CLFs. In practice, this proves to be restrictive and yields infeasible instances for the SOS feasibility problem in Eq. (5.2). As an alternative, we start from an initial domain $D_0 \supset X_0$ described by a ball of radius R_0. The initial CLFs $V_i^{(0)}$ and corresponding regions $\Omega_i^{(0)}$ are synthesized using D_0 as the overall domain. In the next iteration, we expand the radius R_0 by a factor $\alpha > 1$ to obtain a

new domain D_1 described by a ball of radius $\alpha R_0 > R_0$. Thus, the CLFs $V_j^{(1)}$ and corresponding regions $\Omega_j^{(1)}$ are based on setting the domain to D_1. This process yields larger regions from which we can guarantee reachability of the target set in practice. The only drawback is computational: the description of each region $\Omega_j^{(k)}$ becomes $\Omega_j^{(k)} = \{\mathbf{x} \in D_k \mid V_j^{(k)}(\mathbf{x}) \leq \gamma_j^{(k)}\}$. In this case, we need to reformulate how the decrease condition (Eq. 5.2) is formulated. The formulation for a single domain D does not work in the case of successive domain expansion since the constraint $\mathbf{x} \in D_{k+1} \ \wedge \ V_j^{(k)}(\mathbf{x}) > \gamma_j^{(k)}$ no longer represents the set $\mathsf{cl}(D_{k+1} \setminus \Omega_j^{(k)})$. This is because there could be points in $D_{k+1} \setminus D_k$ that satisfy $V_j^{(k)} \leq \gamma_j^{(k)}$ and such points do not belong to $\Omega_j^{(k)}$. As a result, we encode the decrease condition in multiple stages as follows:

$$
\begin{aligned}
&\nabla V \cdot f(\mathbf{x}, \kappa_i(\mathbf{x})) \leq -\xi,\ \forall \mathbf{x} \in \mathsf{cl}(D_{k+1} \setminus D_k) \\
&\nabla V \cdot f(\mathbf{x}, \kappa_i(\mathbf{x})) \leq -\xi,\ \forall \mathbf{x} \in \mathsf{cl}(D_k \setminus \bigcup_{j=1}^{n_k} \Omega_j^{(k)}) \\
&\qquad\qquad \vdots \\
&\nabla V \cdot f(\mathbf{x}, \kappa_i(\mathbf{x})) \leq -\xi,\ \forall \mathbf{x} \in \mathsf{cl}(D_0 \setminus (\bigcup_{j=1}^{n_0} \Omega_j^{(0)} \cup X_0))
\end{aligned}
$$

For each stage, we use the SOS formulation given in Eq. (5.2) with the domain D_k and SLL functions synthesized at level k.

5.5.2 Coverage Using Test Points

Another implementation issue lies in ensuring that the feasibility problem in Eq. (5.2) yields a SLL function whose region Ω_V includes as large a set of states as possible. This can be ensured by adding an objective function to the problem so that we select regions Ω_V that are as large as possible. In practice, this is quite hard to ensure by means of a simple and convex objective function. We resort to an implementation trick using *test points* that was first described in our previous work on successive control barrier functions [92].

At each iteration k, we select a finite set of test states $T_k \subseteq D_k \setminus X_k$ that we wish to cover using the SLLs $V_j^{(k)}$ that we select. To this end, we add the constraint $V_j^{(k)}(\mathbf{x}_T) \leq (1-\lambda)\epsilon R_k^2$ for a selected test point $\mathbf{x}_T \in T_k$. Here $\epsilon > 0$ is the parameter that ensures the radial unboundedness of the function $V_j^{(k)}$ (see Eq. (5.2)) and R_k is the radius of the domain D_k. $\lambda \in (0,1)$ is a user-defined parameter. Since $\gamma_j^{(k)}$ is chosen to be close to ϵR_k^2, we attempt to ensure that $\mathbf{x}_T \in \Omega_j^{(k)}$. If the resultant feasibility problem from Eq. (5.2) with the added constraint at the test point $\mathbf{x}_T$ is feasible, we mark $\mathbf{x}_T$ as covered and remove it from the set T_k. Otherwise, we attempt to cover $\mathbf{x}_T$ in the future with a different choice of control input.

The strategy of using test points to try and increase the regions $\Omega_j^{(k)}$ that we obtain at each step is an important component in ensuring that our approach computes as large a region as possible from which we may guarantee a control strategy to reach X_0.

5.5.3 Controller Synthesis

The closed loop control strategy is quite simple. Each state $\mathbf{x}$ in the overall region of attraction belongs to some domain D_k and some region $\Omega_j^{(k)}$ with SLL function $V_j^{(k)}$ and corresponding control input $\mathbf{u}_j^{(k)}$. We will hold this control input constant until the state reaches the region X_{k-1}. Therefore, we assume a continuous time monitor that tracks which SLL and control input can satisfy the decrease condition for the current state $\mathbf{x}(t)$. This can be performed by tracking: (a) the current domain $D_k : \{\mathbf{x} \mid \|\mathbf{x} - \mathbf{x}^*\| \leq R_k\}$ and (b) the current SLL $V_j^{(k)}$ such that the current state satisfies $V_j^{(k)}(\mathbf{x}(t)) \leq \gamma_j^{(k)}$. When the system transitions from the domain D_k to D_{k-1}, we now monitor for each function $V_i^{(k-1)}$, whether the system has entered the sub-level set $V_i^{(k-1)}(\mathbf{x}) \leq \gamma_i^{(k-1)}$. When this transition happens, we switch the control inputs. Thus, the overall controller is event triggered and assumes that we can continuously track the values of functions $V_i^{(k)}$ over the trajectory. Events are triggered whenever the trajectory moves from a domain D_k to a smaller domain D_{k-1} (the reverse is also possible since the domains themselves are not guaranteed to be positive invariants).

5.6 Empirical Evaluation

In this section, we will illustrate our approach with a few numerical examples. We demonstrate that the approach of computing successive control Lyapunov functions yields larger regions of attraction.

All experiments were run on macOS, Apple M1 pro, 16 GB RAM using the `SumOfSquares.jl` package with `Mosek` as the SDP solver. Table 5.1 summarizes the benchmarks configuration and synthesis times. The nonlinear benchmarks are described below. Our implementation and all benchmark configurations are available at https://github.com/rameezw/Successive_CLFs.

5.6.1 Zubov-Davidson Model

Consider a nonlinear system with two state variables (x_1, x_2) and a control input $u \in [-0.1, 0.1]$.

$$\dot{x}_1 = -x_1 + 2(x_1^2 x_2),$$

$$\dot{x}_2 = -x_2 + u,$$

We take $U_{\text{fin}} = \{u \mid u = \pm 0.1\}$. The initial region of attraction is taken to be $X_{eq} = \{(x_1, x_2) | x_1^2 + x_2^2 \leq 0.2\}$.

5.6.2 Lotka-Volterra System

Consider a nonlinear system with two state variables (x_1, x_2) and a control input $u \in [-0.1, 0.1]$.

$$\dot{x}_1 = -0.42x_1 - 1.05x_2 - 2.3x_1^2 - 0.5x_1x_2 - x_1^3,$$

$$\dot{x}_2 = 1.98x_1 + x_1x_2 + u,$$

We take $U_{\text{fin}} = \{u \mid u = \pm 0.1\}$. The initial region of attraction is taken to be $X_{eq} = \{(x_1, x_2) | x_1^2 + x_2^2 \leq 0.1\}$.

5.6.3 Inverted Pendulum

Consider a nonlinear system with two state variables (x_1, x_2) and a control input $u \in [-3, 3]$.

$$\dot{x}_1 = x_2,$$
$$\dot{x}_2 = \frac{g}{l}(x_1 - \frac{x_1^3}{6}) + \frac{1}{ml^2}u,$$

where, $g = 9.81, m = 0.15$ and $l = 0.5$. We take $U_{\text{fin}} = \{u \mid u = \pm 3\}$. The initial region of attraction is taken to be $X_{eq} = \{(x_1, x_2) | x_1^2 + x_2^2 \leq 0.7\}$.

5.6.4 van der Pol oscillator

We consider the reversed van der Pol system with 2 state variables and control input $u \in [-0.5, 0.5]$.

$$\dot{x}_1 = -x_2,$$
$$\dot{x}_2 = x_1 - \mu(1 - x_1^2)x_2 + u$$

where, the stiffness parameter $\mu = 1.0$. We take $U_{\text{fin}} = \{u \mid u = \pm 0.5\}$. The initial region of attraction is taken to be $X_{eq} = \{(x_1, x_2) | x_1^2 + x_2^2 \leq 0.2\}$.

5.6.5 Wheeled Path Follower

Consider a nonlinear system with three state variables (x_1, x_2, x_3) and a control input $u \in [-1, 1]$.

$$\dot{x}_1 = v\frac{1 - x_3^2/2}{1 - x_2},$$
$$\dot{x}_2 = v(x_3 - x_3^3/6),$$
$$\dot{x}_3 = v\frac{u + u^3/3}{L} - v\frac{1 - x_3^2/2}{1 - x_2}$$

We take $U_{\text{fin}} = \{u \mid u = \pm 0.1\}$. The initial region of attraction is taken to be $X_{eq} = \{(x_1, x_2) | x_1^2 + x_2^2 \leq 0.1\}$.

Table 5.1: Benchmark configuration and synthesis times. n/m: state and input dimensions; $\deg V$: degree of the SLL functions; levels: number of successive levels synthesized; #CLFs: total SLL functions across levels; t_ℓ: CPU time to synthesize level ℓ.

system	n	m	$\deg V$	levels	#CLFs	t_1 (s)	t_2 (s)	t_3 (s)
Zubov-Davidson (5.6.1)	2	1	6	3	15	29	56	125
Lotka-Volterra (5.6.2)	2	1	6	3	5	15	21	39
inv. pendulum (5.6.3)	2	1	6	2	2	267	11	–
van der Pol (5.6.4)	2	1	6	3	3	32	24	47
wheeled path follower (5.6.5)	3	1	6	3	3	71	54	79
ducted fan (5.6.6)	6	2	2	2	44	1246	3	–

5.6.6 Caltech Ducted Fan

Consider the planar ducted fan model (in hover mode) [35], with six state variables $(x_1, x_2, x_3, x_4, x_5, x_6)$ and two control inputs $u_1, u_2 \in [-3.2, 3.2]^2$.

$$\begin{aligned}
\dot{x}_1 &= x_4, \\
\dot{x}_2 &= x_5, \\
\dot{x}_3 &= x_6, \\
\dot{x}_4 &= (-dx_1 + u_1 \cos x_3 - (u_2 + mg) \sin x_3)/m, \\
\dot{x}_5 &= (-dx_2 + u_1 \sin x_3 + (u_2 + mg) \cos x_3)/m, \\
\dot{x}_6 &= ru_1/J
\end{aligned}$$

The values used for the experiments were; $m = 11.2kg$, $g = 0.28m/s^2$, $J = 0.0462kg/m^2$, $r = 0.156m$, $d = 0.1Ns/m$. We take U_{fin} as $[-3.2, 0.0, 3.2]^2$. The initial region of attraction is taken to be $X_{eq} = \{(x_1, x_2, x_3, x_4, x_5, x_6) \mid x_1^2 + x_2^2 + 0.25x_3^2 + 0.1x_4^2 + 0.1x_5^2 + 0.05x_6^2 \leq 0.7\}$.

5.6.7 Comparison with FOSSIL2.0

FOSSIL[23] is a tool to compute verified neural certificates. A comparison showing the effectiveness of our approach for CLF synthesis was conducted for multiple dynamical systems. The neural certificate was trained on a 2-layer network having 10 neurons each, with sigmoid and square activations, respectively, and was verified using dReal. The comparison was the size of the region of attraction computed using the two approaches. The results are shown in Table 5.2 and include the system dynamics for which FOSSIL could successfully output a certificate. It turns out that the successive CLFs approach improves the size of the regions of attraction defined by the neural CLFs computed using FOSSIL. Hence, using successive Lyapunov functions results in less conservative approximations of the region of attraction.

Table 5.2: Regions of attraction comparison for 1000 test points against FOSSIL2.0 [23]. I denotes points inside the region of attraction and O denotes points outside it

system	dimensions	inputs	FOSSIL(I)	Ours(I)	FOSSIL(O) & Ours(I)	Ours(O) & FOSSIL(I)
Zubov-Davidson	2	1	73	**403**	**330**	0
Lotka-Volterra	2	1	421	**513**	**262**	170
wheeled path follower	3	1	10	**329**	**319**	0

5.6.8 Comparison with neural Lyapunov control

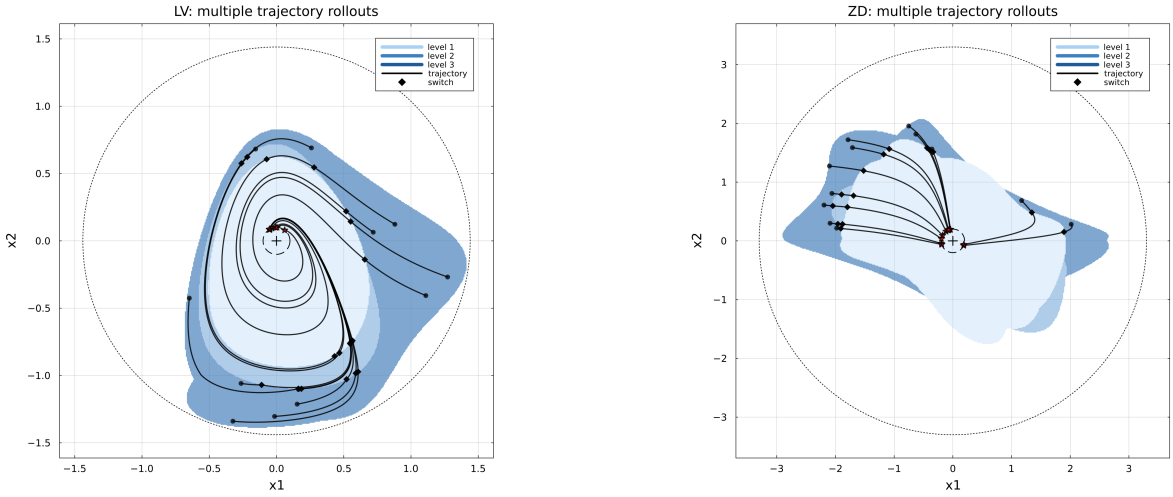
The neural Lyapunov control approach [19] was one of the earlier neural network based approaches to compute Lyapunov functions and use dReal to verify regions of attraction. We compare the size of the region of attraction computed using the two approaches. The results are shown in Table 5.3. Again, the successive CLF approach predictably has larger regions of attraction.

5.7 Summary

This construction is the reachability dual of the successive barrier functions of Chapter 3. The limitations shared across the technical chapters are collected in Chapter 7.

Table 5.3: Regions of attraction comparison for 1000 test points against neural Lyapunov [19].

system	dimensions	inputs	NN-CLF(I)	Ours(I)	NN-CLF(O) & Ours(I)	Ours(O) & NN-CLF(I)
inv pendulum	2	1	50	**414**	**373**	9
wheeled path follower	3	1	126	**329**	**238**	35
ducted fan	6	2	136	**149**	**140**	127



(a) **(b)**

Figure 5.2: Plots showing the RoAs corresponding to the synthesized SCLFs and multiple trajectory rollouts for **(a)** Lotka-Volterra and **(b)** Zubov-Davidson models.

Chapter 6

Repulsive Control Barrier Functions for Moving Obstacles

"When you're ready, you won't have to."
— Morpheus, *The Matrix* (on dodging bullets)

Chapter 3 certifies safety against a fixed unsafe set; this chapter treats an obstacle that moves, is measured only periodically, and may move adversarially between measurements. For dynamics with translational symmetry, recentering the workspace on the obstacle at each measurement turns the problem into a hybrid system with translational reset maps on a compact set of relative coordinates. A *repulsive* barrier function combines a continuous-time decrease condition with robustness to the worst-case recentering jump, yielding an infinite-horizon safety guarantee, and an event-triggered safety filter overrides the nominal controller only when the jump-robust margin is at risk.

6.1 Introduction

The barrier constructions of Chapter 3 certify safety against a *fixed* unsafe set. This chapter takes up the dynamic case: an obstacle that *moves*, whose future position is known only through periodic measurements and is otherwise nondeterministic between them. The difficulty is twofold. Each measurement updates our knowledge of the obstacle discontinuously, so the safety argument must contend not only with the continuous flow but with measurement-induced jumps. Also, the obstacle may move adversarially within its motion constraints, so the certificate must be robust to the worst admissible displacement.

Our approach exploits a structural property common to vehicular dynamics: *translational symmetry*. For systems whose dynamics do not depend on absolute workspace position, we may, at each measurement, *recenter* the workspace so that the obstacle sits at the origin, turning the moving-obstacle problem into a hybrid system (Section 2.2, [31]) with translational reset maps, on a single compact set of relative coordinates. We then synthesize a *repulsive* barrier function that satisfies a continuous-time decrease condition *and* is robust to the worst-case recentering jump, yielding an infinite-horizon safety guarantee. As in Chapter 3, a single fixed-input barrier is conservative, so we again combine multiple fixed-input barriers, synthesized by SOS programming under the finite-control-set theorem (Theorem 2), to enlarge the control-invariant region, and wrap them in an event-triggered safety filter of the kind introduced in Section 1.2.

The key idea in *repulsive barrier functions* is to combine the standard barrier decrease condition over the system trajectories and the robustness to the worst-case jump caused by obstacle recentering. This leads to a hybrid safety argument with an explicit safety margin. The safety controller must drive the barrier sufficiently far below the safety threshold so that the subsequent translational jump cannot bring the system back into the unsafe region. In contrast to the standard CBF conditions, the resulting certificate is tailored to the sampled measurement model and yields an infinite horizon safety guarantee despite nondeterministic obstacle motion between measurements.

Contributions of this chapter.

- An *obstacle-centered* formulation that exploits translational symmetry to recenter the dynamics at each measurement;
- *Repulsive* barrier functions that maintain invariance under the hybrid jumps induced by recentering, with an explicit safety margin and an infinite-horizon guarantee;
- An SOS synthesis procedure, and the use of multiple fixed-input barriers to expand the control-invariant set; and
- An event-triggered safety filter alternating passive monitoring with active takeover, evaluated on nonlinear vehicle models.

6.2 Related Work

Control Barrier Functions (CBFs) [4, 95] extend the idea of barrier certificates used in safety verification [68] to safety enforcement by imposing inequalities on the control input that render a safe set forward invariant. Quadratic programming based safety filters have been widely used in many robotics applications [5, 97]. We distinguish between two distinct classes of approaches.

Barrier certificate synthesis A substantial line of work treats the barrier function itself as the object to be synthesized. In this viewpoint, one fixes either the open-loop dynamics or a nominal feedback law and then searches for a certificate whose sign conditions separate safe and unsafe sets and the Lie-derivative inequalities certify forward invariance. Prajna et al. [68, 69] formulated barrier certificates for nonlinear and hybrid systems as convex conditions that avoid explicit reachable set computation. This perspective fits naturally with SOS optimization, yielding tractable sufficient conditions for safety verification[40, 62]. Related developments include exponential barrier certificates for hybrid verification and other SOS-based certificate constructions for nonlinear systems [37]. The advantage of this paradigm is that once a barrier certificate is synthesized, invariance is certified offline and does not rely on solving an online optimization problem during execution. The main limitation here is conservatism as the controlled invariant set can be substantially smaller than the

true safe set. However, recent approaches employing a hierarchy of these safety certificates have helped reduce this limitation [92].

Controller synthesis approach A complementary line of work simply assumes a function $h(\mathbf{x})$ (usually defined as some measure of distance from the unsafe set) and computes the control input online so that the barrier's value is non-increasing over time (in the convention used for this paper). These approaches use a QP to compute a control input at each point in time to attempt to enforce safety by ensuring that $h(\mathbf{x})$ decreases over time[4, 5, 17]. Subsequent extensions addressed robustness and higher relative-degree constraints [60, 98]. These approaches are attractive because they compute the controller online for a fixed barrier candidate, thereby exploiting available control authority more directly than a fixed feedback certification. However, the safety guarantees depend on the feasibility of the online optimization problem. In presence of input limits or incorrect formulation of the barrier function, the QP may become infeasible. This weakens the invariance guarantee unless additional backup or robustness mechanisms are introduced[5, 98].

Our work follows the barrier synthesis approach and is most closely related to efforts on safety in dynamic environments. But it differs from existing work in both modeling and certification strategy. Beyond barrier-based control, dynamic collision avoidance has long been studied through geometric methods such as velocity obstacles, reciprocal velocity obstacles, and ORCA, which provide efficient reactive collision avoidance in velocity space [24, 79, 88, 89]. These methods are fast and effective, but their guarantees are primarily geometric and kinematic as opposed to our work which is based on state-space invariance certificates.

FaSTrack similarly combines offline computation with online safety enforcement [20, 33]. However, their approach uses a controller that can track a nominal reference which is responsible for maintaining the safety property with sufficient margin so that tracking errors do not cause safety violations. Furthermore, the approach is not directly designed to treat dynamic obstacles.

Some recent work has also pushed barrier certificates and safety filters toward uncertainty-aware and dynamic environment settings. CVaR barrier functions and related distributionally robust invariants incorporate tail-risk notions into safety-critical control [2, 93]. More recently, collision-cone

CBFs [83] adapt relative-velocity geometry and dynamic parabolic CBF [63] approaches adapt the online CBF framework to moving obstacle scenarios. However, these methods remain largely within the online CBF-QP paradigm discussed earlier. Our contribution is complementary in the sense that instead of redesigning the online filter, we exploit translational symmetry to build the safety certificates.

Sampled-data and hybrid extensions of barrier certificates have addressed instances where the state is periodically evaluated, unknown disturbances and unknown dynamics are present [48, 61, 84]. Our contribution differs in two ways: (a) we assume moving obstacles which can move arbitrarily while satisfying some constraints on its motion rather than uncertainty in the vehicle dynamics; and (b) we also exploit symmetry properties of the dynamics to maintain an obstacle-centered approach that in turn allows us to reason about compact sets of states in practice.

Prior work has exploited symmetry to improve reachability analysis. Sibai et al. [78] use symmetry transformations to reuse reachable sets across modes and agents by applying the symmetry transformation to map trajectories and thus avoid the overhead of repeated reachability computation. A similar approach is carried out in previous work by Chou et al. [21]. Maidens and Arcak [49] reduce the dimensionality of backward reachability via invariant representations. In contrast, our work applies symmetry at the level of certificate synthesis, using translational invariance to construct control barrier functions robust to both continuous dynamics and symmetry-induced hybrid jumps, rather than accelerating reachability computations.

6.3 Problem Statement

Let $\mathbf{x} \in \mathbb{R}^n$ denote a vector of state-variables of a dynamical system that includes quantities such as position, angles, velocities and angular velocities. Let us assume that the state variables are partitioned as $\mathbf{x} : (\mathbf{y}, \mathbf{z})$ wherein $\mathbf{y} \in \mathbb{R}^m$ are workspace variables and $\mathbf{z} \in \mathbb{R}^{n-m}$ are the non-workspace state variables of the system. For instance, consider a vehicle whose states include (x, y, v_x, v_y), representing positions (x, y) and velocities (v_x, v_y). We designate $\mathbf{y} = (x, y)$ as the workspace variables.

The dynamics are given by $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ for a Lipschitz-continuous function f and control inputs $\mathbf{u} \in U \subseteq \mathbb{R}^k$.

Assumption 1 (Workspace Invariance) Let us assume that there exists a function g such that $f(\mathbf{x}, \mathbf{u}) = g(\mathbf{z}, \mathbf{u})$, or in other words, is independent of the workspace variables $\mathbf{y}$. Therefore, for any two states of the form $\mathbf{x}_1 = (\mathbf{y}_1, \mathbf{z})$ and $\mathbf{x}_2 = (\mathbf{y}_2, \mathbf{z})$, that agree on their non-workspace variables but have different workspace "configurations", the RHS of the ODE are the same $f(\mathbf{x}_1, \mathbf{u}) = f(\mathbf{x}_2, \mathbf{u})$. This generalizes the notion of translational symmetry.

Example 8 Consider the Dubins vehicle with state variables $\mathbf{x} : (x, y, \theta)$ and control input $u \in \mathbb{R}$. The workspace variables are $\mathbf{y} : (x, y)$. $\dot{x} = \cos(\theta), \dot{y} = \sin(\theta), \dot{\theta} = u$. The RHS of the ODE is independent of the position (x, y).

Obstacle Model We assume that there is an obstacle $\mathcal{O}$ whose dynamics are defined in the workspace in the form of constraints $g(\mathbf{y}_o, \dot{\mathbf{y}}_o) \geq 0$, wherein $\mathbf{y}_o \in \mathbb{R}^m$ represents the obstacle position and $\dot{\mathbf{y}}_o \in \mathbb{R}^m$ represents the obstacle velocity in the workspace. We assume without loss of generality that $\mathbf{y}_o(0) = \mathbf{0}$.

Example 9 (Velocity-Bounded Obstacle) Consider a velocity bounded obstacle modeled as a newtonian particle with the constraint $\|\dot{\mathbf{y}}_o\| \leq \alpha$, wherein α is the velocity bound.

Given an obstacle with constraints $g(\mathbf{y}_o, \dot{\mathbf{y}}_o) \geq 0$ over its positions and velocities and time $\tau > 0$, let $\hat{\mathcal{O}}(\tau)$ represent the *reachable set* of all obstacle positions within time τ wherein the obstacle dynamics satisfy $g \geq 0$ at all times:

$$\hat{\mathcal{O}} = \left\{ \mathbf{y}_o(t) \;\middle|\; \begin{array}{c} t \in [0, \tau],\ \mathbf{y}_o(0) = \mathbf{0}, \\ \mathbf{y}_o(t) \text{ differentiable and} \\ \forall t \in [0, \tau]\ g(\mathbf{y}_o(t), \dot{\mathbf{y}}_o(t)) \geq 0 \end{array} \right\}$$

Example 10 Consider the velocity-bounded obstacle model from Ex. 9. The set $\hat{\mathcal{O}} = \{(x, y) \mid x^2 + y^2 \leq \alpha^2\tau^2\}$ represents all reachable positions within time $\tau > 0$.

Measurement Model Let us assume that the obstacle position is measured every $\tau > 0$ time units. For simplicity we will assume that this position measurement is error-free. However, our work readily accommodates erroneous position measurements by redefining the reachable set $\hat{\mathcal{O}}$ to include the possible measurement error in the obstacle.

Periodic Translations In our framework, we will perform a translation over the workspace coordinates to place the obstacle at $\mathbf{y}_o = \mathbf{0}$, whenever a position measurement is made available to us. For a function $f(t)$ and some fixed t, let $f(t^+) = \lim_{s \to t^+} f(s)$ be the "right" limit and $f(t^-) = \lim_{s \to t^-} f(s)$ be the "left" limit. If f is continuous at t, then $f(t^+) = f(t^-) = f(t)$. We say that f is "right-continuous" at t if $f(t^+)$ and $f(t^-)$ exist for all t and $f(t) = f(t^+)$.

The translation happens whenever a measurement $\mathbf{y}_o$ is obtained and "instantaneously" shifts the workspace coordinates of the system to $\mathbf{y}(t^+) = \mathbf{y}(t) = \mathbf{y}(t^-) - \mathbf{y}_o = \mathbf{0}$, creating a discontinuity at time $t = k\tau$. For simplicity, we assume measurements are obtained in the current reference frame: i.e., the measured position is $\mathbf{y}_o(t^-) = \mathbf{y}_o$. The result of considering this conceptual translation makes our system a *hybrid system* with a reset map [31, 57].

Definition 13 (Translational Jump) Let us assume that the system receives an obstacle measurement $\mathbf{y}_o(t^-) = \mathbf{y}_o$ at time $t = k\tau$ for $k \in \mathbb{N}$. The translational jump corresponding to this measurement resets the obstacle position $\mathbf{y}_o(t^+) = \mathbf{y}_o(t) = \mathbf{0}$ and the system state to $\mathbf{x}(t^+) = \mathbf{x}(t) = (\mathbf{y}(t^-) - \mathbf{y}_o, \mathbf{z}(t^-))$ that changes the workspace state while leaving the other state variables unaltered.

Formally, the overall execution of the system starting from initial state $\mathbf{x}(0)$, initial obstacle position $\mathbf{y}_o(0) = \mathbf{0}$ and control input signal $\mathbf{u}(t)$ consists of the system state trajectory $\mathbf{x} : \mathbb{R}_{\geq 0} \to \mathbb{R}^n$ that maps time t to state $\mathbf{x}(t)$, and an obstacle trajectory $\mathbf{y}_o : \mathbb{R}_{\geq 0} \to \mathbb{R}^m$, such that

(1) $\mathbf{x}(s)$ is a smooth function in each open interval $s \in (k\tau, (k+1)\tau)$ with $\dot{\mathbf{x}}(s) = f(\mathbf{x}(s), \mathbf{u}(s))$;

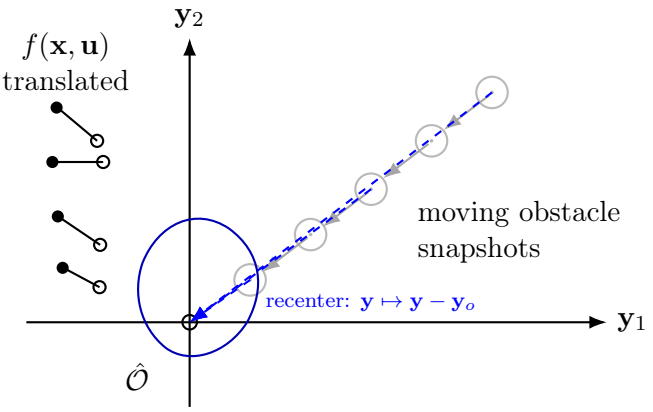


Figure 6.1: Translation invariance and recentering for moving obstacles. Each obstacle snapshot is translated back to the origin via $y \mapsto y - y_o$, so the same local feedback template can be reused in relative coordinates.

(2) $\mathbf{y}_o(s) \in \hat{\mathcal{O}}$ for each $s \in (k\tau, (k+1)\tau)$;

(3) For each $k \in \mathbb{N}_{>0}$, $\mathbf{x}(k\tau)$ is *right-continuous* i.e., the right limit $\lim_{s \to k\tau^+} \mathbf{x}(s)$ exists and equals $\mathbf{x}(t)$.

(4) Likewise, we will assume that the obstacle position is also right-continuous.

(5) For each $k \in \mathbb{N}_{>0}$, $\mathbf{x}(k\tau) = \lim_{t \to k\tau^+} \mathbf{x}(t) = (\mathbf{y}(k\tau) - \mathbf{y}_o(t^-), \mathbf{z}(k\tau))$, wherein $\mathbf{y}_o(t^-) \in \hat{\mathcal{O}}$ is the measurement obtained at time $t = k\tau$.

Safety Enforcement A trajectory collides with the obstacle if $\mathbf{x}(t) \in \hat{\mathcal{O}}$ for any time t. We assume that the following information is provided to us:

(1) The system dynamics $\mathbf{x} = f(\mathbf{x}, \mathbf{u})$ with control inputs drawn from some compact set U, and satisfying translational invariance over the workspace coordinates $\mathbf{y}$.

(2) The measurement time period τ, assuming precise measurement.

(3) The obstacle reachable set $\hat{\mathcal{O}}$ consisting of all obstacle reachable configurations during the time interval $[0, \tau]$.

We wish to synthesize a control barrier function $B : \mathbb{R}^n \to \mathbb{R}$, a smooth function that is: (a) strictly

positive over the unsafe set $\hat{\mathcal{O}}$; (b) the set $\{\mathbf{x} \mid B(\mathbf{x}) \leq 0\}$ can be maintained control invariant; and (c) control invariant over possible translational jumps resulting from periodic measurements.

6.4 Repulsive Barrier Functions

We recall the control barrier function and its forward-invariance guarantee from Section 2.3.2 (Definition 5). Definition 6 of Chapter 2 previewed the repulsive variant informally; we now make it precise and prove the hybrid safety theorem.

Theorem 10 Given a CBF $B(\mathbf{x})$, there exists a feedback law $\kappa : X \to U$ such that for all initial states $\mathbf{x}(0)$ with $B(\mathbf{x}(0)) \leq 0$, the closed loop trajectory of the system $\dot{\mathbf{x}} = f(\mathbf{x}, \kappa(\mathbf{x}))$ satisfies $B(\mathbf{x}(t)) \leq \exp(\lambda t) B(\mathbf{x}(0)) \leq 0$.

The proof can be obtained by adapting the more general case shown in Ames et al. [4]. In our setting, we need to adapt the CBF definition to deal with the expected periodic jumps.

Let $\mathbf{y} \in \mathcal{Y}$ represent all workspace configurations and $\mathbf{z} \in \mathcal{Z}$ represent non-workspace state variables so that the state-space $\mathcal{X} = \mathcal{Y} \times \mathcal{Z}$.

Definition 14 (Repulsive Barrier Function) A smooth function $B(\mathbf{y}, \mathbf{z})$ is said to be a *repulsive* barrier function w.r.t to unsafe set $\hat{\mathcal{O}}$ and periodic translation jumps iff the following conditions hold:

(1) **(Unsafe Set)** $B(\mathbf{y}, \mathbf{z}) \geq \epsilon > 0$ for all $\mathbf{y} \in \hat{\mathcal{O}}$ and $\mathbf{z} \in \mathcal{Z}$, where ϵ is a fixed constant.

(2) **(Decrease)** $\forall \mathbf{y} \in \mathcal{Y}, \forall \mathbf{z} \in \mathcal{Z}, \exists \mathbf{u} \in U,\ \nabla B \cdot f(\mathbf{x}, \mathbf{u}) \leq \lambda B(\mathbf{x})$ for $\lambda > 0$. $\lambda > 0$ is a fixed constant that makes this function a *repulsive* function.

(3) **(Bounded Jump)** There is a fixed constant $K > 0$ s.t.

$$\forall (\mathbf{y}, \mathbf{z}) \in \mathcal{Y} \times \mathcal{Z}, \forall \mathbf{y}_o \in \hat{\mathcal{O}},\ B(\mathbf{y} - \mathbf{y}_o, \mathbf{z}) - B(\mathbf{y}, \mathbf{z}) \leq K\,.$$

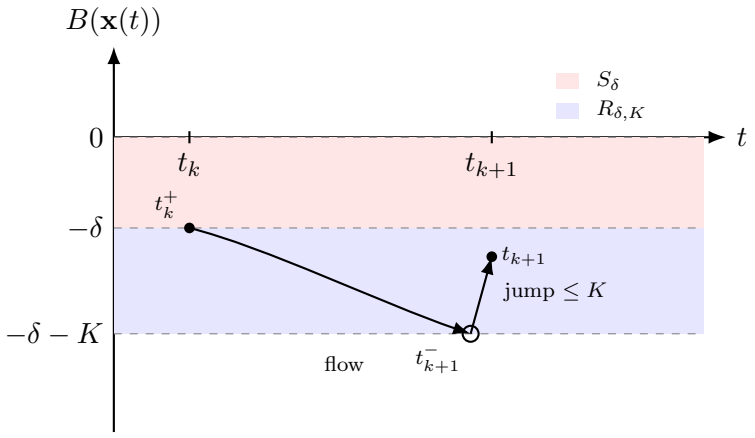


Figure 6.2: Sampled-time repulsion under translational jumps. The red band denotes the safety-margin set $S_\delta = \{\mathbf{x} \mid -\delta < B(\mathbf{x}) \leq 0\}$, and the blue band denotes the repulsion region $R_{\delta,K} = \{\mathbf{x} \mid -\delta - K \leq B(\mathbf{x}) \leq -\delta\}$. Starting from $B(\mathbf{x}(t_k)) \leq -\delta$, the flow drives the barrier value to $B(\mathbf{x}(t_{k+1}^-)) \leq -\delta - K$, and the translational jump increases it by at most K, yielding $B(\mathbf{x}(t_{k+1})) \leq -\delta$.

Let $\mathbf{u} = \kappa(\mathbf{x})$ be the safety enforcing feedback law associated with B that satisfies the results of Theorem 10. Given a repulsive barrier function B, let us choose a constant $\delta > 0$ as our *margin* and let $S_\delta = \{\mathbf{x} \mid -\delta < B(\mathbf{x}) \leq \mathbf{0}\}$ denote a "safety margin". Note by definition $S_\delta \cap \hat{\mathcal{O}} = \emptyset$ and S_δ is part of the control invariant region. However, we will ensure that *the system never reaches a state in* S_δ. To this end, we define the "repulsion region" $R_{\delta,K} = \{\mathbf{x} \mid -\delta - K \leq B(\mathbf{x}) \leq -\delta\}$. See Figure 6.2 for an illustration of these regions.

Given τ, let $t_k = k\tau$ for $k \in \mathbb{N}$. We will now show that if the system is initialized such that $B(\mathbf{x}(0)) \leq -\delta$, there is a control strategy such that $B(\mathbf{x}(t)) \leq -\delta$ for all time.

Lemma 11 Suppose $\lambda \geq \frac{1}{\tau}\log\left(1 + \frac{K}{\delta}\right)$, $B(\mathbf{x}(t_k)) \leq -\delta$ for some time $t_k = k\tau$, applying the feedback $\mathbf{u} = \kappa(\mathbf{x})$ over $(t_k, t_{k+1}]$, ensures that (a) $B(\mathbf{x}(t_{k+1}^-)) \leq -\delta - K$ and (b) $B(s) \leq -\delta$ for $s \in (t_k, t_{k+1}]$.

Proof. Since $\lambda \geq \frac{1}{\tau}\log\left(1 + \frac{K}{\delta}\right)$, we get $\exp(\lambda\tau) \geq 1 + \frac{K}{\delta}$. Since $B(\mathbf{x}(t_k)) \leq -\delta$, from Theorem 10, we get $B(\mathbf{x}(t_k + s)) \leq \exp(\lambda s)B(\mathbf{x}(t_k)) \leq \exp(\lambda s)(-\delta) \leq -\delta$ for $s \in (0, \tau)$, since $\lambda, s > 0$. Note that the function $g(s) = -\delta\exp(\lambda s)$ is continuous and $B(\mathbf{x}(t_k + s)) \leq g(s)$ over $s \in [t_k, t_{k+1})$. Therefore, $\lim_{s\to\tau^-} B(\mathbf{x}(t_k + s)) \leq \lim_{s\to\tau^-} g(s) = -\delta\exp(\lambda\tau) = -\delta\left(1 + \frac{K}{\delta}\right) = -\delta - K$. □

We now prove that using a repulsive barrier function allows us to find a control strategy that establishes the absence of collision with the moving obstacle.

Theorem 11 Suppose B is a repulsive barrier function with $\lambda \geq \frac{1}{\tau}\log\left(1+\frac{K}{\delta}\right)$ and the initial state $\mathbf{x}(0)$ is such that $B(\mathbf{x}(0)) \leq -\delta$. For any sequence of measurements $\mathbf{y}_o(t_k^-) \in \hat{\mathcal{O}}$, for $k \in \mathbb{N}$, there exists a control strategy such that the resulting trajectory will never intersect $\hat{\mathcal{O}}$.

Proof. The trajectory of the system is a hybrid trajectory with a flow over each time interval $[k\tau, (k+1)\tau)$ for each $k \in \mathbb{N}$ and a translational jump at each time $t_k = k\tau$, triggered by a position measurement $\mathbf{y}_o(t_k^-) \in \hat{\mathcal{O}}$.

Formally, we establish by induction that at each time $t_k = k\tau$ for $k \in \mathbb{N}$, the following conditions holds: $B(\mathbf{x}(t_k)) \leq -\delta$, $B(\mathbf{x}(t_k+s)) \leq -\delta$ for all $s \in (0,\tau]$ and $B(\mathbf{x}(t_{k+1}^-)) \leq -K-\delta$. Since $B(\mathbf{x}) > 0$ for $\mathbf{x} \in \hat{\mathcal{O}}$, this will prove safety of the system as well for all times $t \geq 0$.

For the base case, note that $\mathbf{y}_o(0) = \mathbf{0}$ and $B(\mathbf{x}(0)) \leq -\delta$ by assumption. Next, from Lemma 11 we conclude that $B(\mathbf{x}(s)) \leq -\delta$ for all $s \in [0,\tau)$ and $B(\mathbf{x}(\tau^-)) \leq -K-\delta$.

To establish the induction, suppose the properties (A), (B), (C) hold over the interval $s \in [t_k, t_{k+1})$ for some $k \in \mathbb{N}$, we wish to establish it for $s \in [t_{k+1}, t_{k+2})$. We have that $B(\mathbf{x}(t_{k+1}^-)) \leq -K-\delta$. Let $\mathbf{y}_o \in \hat{\mathcal{O}}$ be the measurement obtained at time $t = t_{k+1}$. Thus, we have $B(\mathbf{x}(t_{k+1})) = B(\mathbf{y}(t_{k+1}^-) - \mathbf{y}_o, \mathbf{z}(t_{k+1}^-)) \leq B(\mathbf{y}(t_{k+1}^-), \mathbf{z}(t_{k+1}^-)) + K$ due to the bounded jump condition from Def. 14. Therefore, we have $B(\mathbf{x}(t_{k+1})) \leq -\delta$, showing (A). We obtain (B) and (C) by applying Lemma 11 for $t = t_{k+1}$. □

In the next section, we describe the procedure for synthesizing repulsive barrier functions using SOS programs.

6.5 Synthesis of Repulsive Barrier Functions

In this section we use Sum-of-Squares (SOS) programming to synthesize repulsive barrier functions.

We recall the SOS machinery and Putinar's positivstellensatz from Section 2.5. The three

$$\hat{\mathcal{O}} = \left\{\mathbf{y} \mid p_o^{(j)}(\mathbf{y}) \geq 0, j \in [N_O]\right\} \quad \mathcal{Y} = \left\{\mathbf{y} \mid p_y^{(j)}(\mathbf{y}) \geq 0, j \in [N_Y]\right\}$$
$$\mathcal{Z} = \left\{\mathbf{z} \mid p_z^{(j)}(\mathbf{z}) \geq 0, j \in [N_Z]\right\}$$

$$\begin{cases} B(\mathbf{y},\mathbf{z}) & = \sum_j c_j \phi_j(\mathbf{y},\mathbf{z}) \\ B - \epsilon & = \sigma_0 + \sum_{j=1}^{N_O} \sigma_{j,o} p_o^{(j)}(\mathbf{y}) + \sum_{j=1}^{N_Z} \sigma_{j,z} p_z^{(j)} \\ \lambda B - \nabla B \cdot f(\mathbf{x},\mathbf{u}_r) & = \pi_0 + + \sum_{j=1}^{N_Y} \pi_{j,y} p_y^{(j)}(\mathbf{y}) + \sum_{j=1}^{N_Z} \pi_{j,z} p_z^{(j)} \\ K - B - B(\mathbf{y} - \mathbf{y}_o, \mathbf{z}) & = \xi_0 + + \sum_{j=1}^{N_Y} \xi_{j,y} p_y^{(j)}(\mathbf{y}) + \sum_{j=1}^{N_Z} \xi_{j,z} p_z^{(j)} \\ & \quad + \sum_{j=1}^{N_O} \xi_{j,o} p_o^{(j)}(\mathbf{y}_o) \\ & \sigma_*, \pi_*, \xi_* \text{ Sum of Squares} \end{cases}$$

Figure 6.3: Sum of Squares formulation for repulsive barrier functions. $\phi_1, \ldots, \phi_N$ are basis functions (eg., monomials of degree less than D), $\mathbf{u}_r$ is the user-provided feedback function, $\lambda, \epsilon, K > 0$ are user-provided constants.

conditions of Definition 14; positivity on $\hat{\mathcal{O}}$, the repulsive decrease, and the bounded jump, are each encoded as SOS constraints.

6.5.1 Synthesizing Control Barrier Functions

The problem of synthesizing control barrier functions is well known to be hard. The key issue lies in the $\forall\exists$ structure of the decrease condition in Def. 14. This quantifier alternation requires us to simultaneously search for a feedback law $\mathbf{u} = \kappa(\mathbf{x})$ and the barrier function B at the same time, leading to a non-linear and non-convex optimization problem [72, 81]. In this work, we use ideas developed in our previous work on successive barrier functions to side-step this requirement [92] by synthesizing multiple barrier functions instead of a single function, with the overall control invariant region provided by the union of each individual region. However, each function fixes the control input $\mathbf{u} \in U$ to a fixed value, making each control invariant region small.

Let $U_{\text{fin}} \subseteq U$ be a finite set of control inputs. The set can be chosen by taking points that lie on a finite grid within the set U (assumed to be compact) or in the case that the dynamics are control affine: $f(\mathbf{x},\mathbf{u}) = g(\mathbf{x}) + h(\mathbf{x})\mathbf{u}$ for functions g, h and U is polyhedral, we can choose the vertices of the set U to be our samples [72].

We aim to synthesize multiple control barrier functions $B_1, \ldots, B_m$ by repeatedly solving the SOS problem in Figure 6.3, each associated with a unique reference control input $\mathbf{u}_1, \ldots, \mathbf{u}_m \in U_{\text{fin}}$ that are sampled from the control input set U. The overall procedure is presented in Algorithm 5. Besides fixing $U_{\text{fin}} \subseteq U$, we also fix a test coverage set of state samples $S \subseteq \mathcal{X}$. Using the set S, we are able to select barrier functions that attempt to "cover" each test point $\mathbf{x}_t \in S$ inside the controllable region $B(\mathbf{x}) \leq 0$.

Algorithm 5: Algorithm for multiple repulsive barrier computation

Data: System Σ, unsafe set $\hat{\mathcal{O}}$, finite control set U_{fin}, constants $\epsilon > 0$, $\tau > 0$, $K > 0$ and $\delta > 0$, degree bound D, test set S.
Result: A collection $\mathcal{B}$ of local repulsive barriers and associated controls.

1 $\mathcal{B} = \emptyset$ `/* Initialize to empty set */`
2 $\lambda = \frac{1}{\tau} \log\left(1 + \frac{K}{\delta}\right)$ `/* Fixed to worst case */`
3 **for** $\mathbf{u}_t \in U_{\text{fin}}$ **do**
4 **for** $\mathbf{x}_t \in S$ **do**
5 find B
6 by solving SOS problem in Figure 6.3
7 with $\mathbf{u} = \mathbf{u}_r$ and
8 additional constraint $B(\mathbf{x}_t) \leq 0$.
9 **if** *B found (feasible SOS problem)* **then**
10 $\mathcal{B} = \mathcal{B} \cup \{(B, \mathbf{u}_t)\}$
11 $S = S \setminus \{\mathbf{x} \in S \mid B(\mathbf{x}) \leq 0\}$ `/* remove covered test points */`
12 **return** $\mathcal{B}$

In the algorithm, we begin by assuming a finite set of control inputs $U_{\text{fin}} \subseteq U$ for our system and a set of sampled test states S. The choice U_{fin} is discussed in detail later in the subsequent section. Next, we iterate over the control inputs $\mathbf{u}_t \in U_{\text{fin}}$ and the test points $\mathbf{x}_t \in S$. In each iteration, we search for a barrier candidate based on the SOS problem formulated earlier (lines 5-8). Once we find a successful barrier candidate, we add it to our collection of repulsive barriers (line 10), eliminate all the test points it satisfies (line 11), and move on to the next iteration. Thus, we are able to compute multiple repulsive barrier functions, effectively covering a large part of the workspace for our system.

We conclude the section by partly justifying our choice of a finite set of control inputs U_{fin}. Suppose there were a repulsive barrier B satisfying the conditions in Def. 14 in a "robust manner",

we prove that there is a finite subset $U_{\text{fin}} \subseteq U$ such that at each $\mathbf{x}$, the decrease condition in Def. 14 can be satisfied by selecting $\mathbf{u} \in U_{\text{fin}}$.

Theorem 12 If B is a smooth barrier function and f is a differentiable vector field, satisfying a robust decrease condition for some $\lambda, \varepsilon > 0$:

$$\forall \mathbf{y} \in \mathcal{Y}, \mathbf{z} \in \mathcal{Z}, \exists \mathbf{u} \in U,\ \nabla B \cdot f(\mathbf{x}, \mathbf{u}) \leq \lambda B(\mathbf{x}) - \varepsilon\,,$$

there is a finite set $U_{\text{fin}} \subseteq U$ such that

$$\forall \mathbf{y} \in \mathcal{Y}, \mathbf{z} \in \mathcal{Z}, \exists \underline{\mathbf{u} \in U_{\text{fin}}},\ \nabla B \cdot (f(\mathbf{x}, \mathbf{u})) \leq \lambda B - \varepsilon/2\,.$$

Proof. Given B is at least twice differentiable, for all $\mathbf{x}$ the function $\nabla B \cdot f(\mathbf{x}, \mathbf{u}) - \lambda B(\mathbf{x})$ is differentiable, and therefore Lipschitz. Since U is compact, there exists a Lipschitz constant $\mathcal{L} > 0$ for all $\mathbf{x} \in \mathcal{X}$ and $\mathbf{u}, \hat{\mathbf{u}} \in U$, $\|\nabla B \cdot (f(\mathbf{x}, \mathbf{u})) - \nabla B \cdot (f(\mathbf{x}, \hat{\mathbf{u}})\| \leq \mathcal{L}\|\mathbf{u} - \hat{\mathbf{u}}\|$.

Assuming that we choose U_{fin} as a $\varepsilon/(2\mathcal{L})$-packing of the set U so that for all $\mathbf{u} \in U$, there exists a $\mathbf{u}_f \in U_{\text{fin}}$ such that $\|\mathbf{u} - \mathbf{u}_f\| \leq \varepsilon/(2\mathcal{L})$. Take any $\mathbf{x} \in \mathcal{X}$. Suppose $\mathbf{u} = \kappa(\mathbf{x})$ represents the control input that satisfies the decrease condition. Let $\mathbf{u}_f \in U_{\text{fin}}$ such that $\|\mathbf{u} - \mathbf{u}_f\| \leq \varepsilon/(2\mathcal{L})$.

$$\begin{aligned} \nabla B \cdot (f(\mathbf{x}, \mathbf{u}_f)) - \lambda B &\leq \nabla B \cdot f(\mathbf{x}, \mathbf{u}) - \lambda B + \mathcal{L}\|\mathbf{u} - \mathbf{u}_f\| \\ &\leq -\epsilon + \mathcal{L}\epsilon/(2L) \ \leq -\epsilon/2\,. \end{aligned}$$

Thus, $\mathbf{u}_f \in U_{\text{fin}}$ is able to decrease the barrier function. □

6.6 Safety Filter

Let $\mathcal{B} = \{(B_1, u_1), \ldots, (B_m, u_m)\}$ be a collection of repulsive barriers and associated controls synthesized in Section 6.5 for common parameters τ, δ and K, and define $B_{\min}(\mathbf{x}) = \min(B_1(\mathbf{x}), \ldots, B_m(\mathbf{x}))$. At each measurement time $t_k = k\tau$, the obstacle position is measured and the state is recentered via $\mathbf{x}(t_k^+) = (\mathbf{y}(t_k^-) - \mathbf{y}_o(t_k^-), \mathbf{z}(t_k^-))$. The *event-triggered* safety filter then operates in one of the two modes: nominal vs. override (see Fig. 6.4).

In *nominal mode*, the nominal controller is applied while $B_{\min}(\mathbf{x}(t)) < -\delta - K$. We monitor the state $\mathbf{x}(t)$ and compute $B_{\min}(\mathbf{x}(t))$ at a rapid monitoring period $\tau_m > 0$. If at some time $t_s \in [t_k, t_{k+1})$ we detect $B_{\min}(\mathbf{x}(t_s)) \geq -\delta - K$, the filter immediately switches to *override mode*, selects an index $j \in \arg\min(B_1(\mathbf{x}(t_s)), \ldots, B_m(\mathbf{x}(t_s)))$, and applies the associated control $\mathbf{u}_j$ until the next measurement time t_{k+1}. The monitoring frequency must be small enough that the event $-K - \delta \leq B_{\min}(\mathbf{x}(t)) \leq -K$ is reliably detected and the transition from nominal to override mode is made.

Lemma 12 If we detect the event $B_{\min}(\mathbf{x}(t_s)) = -\delta - K$ at some time $t_s \in [t_k, t_{k+1})$. Let $j \in \arg\min_i B_i(\mathbf{x}(t_s))$. Applying the control input $\mathbf{u} = \mathbf{u}_j$ over the interval $(t_s, t_{k+1}]$ guarantees that $B_j(\mathbf{x}(t_{k+1}^-)) < -\delta - K$, and after the translational jump $B_j(\mathbf{x}(t_{k+1}^+)) < -\delta$.

Proof is a direct consequence of the decrease in the repulsive barrier function from Theorem 10.

Thus, the periodic recentering is used only to update the relative-coordinate frame, whereas, switching from nominal to override is event-triggered. The operation of the filter is illustrated in Fig. 6.4.

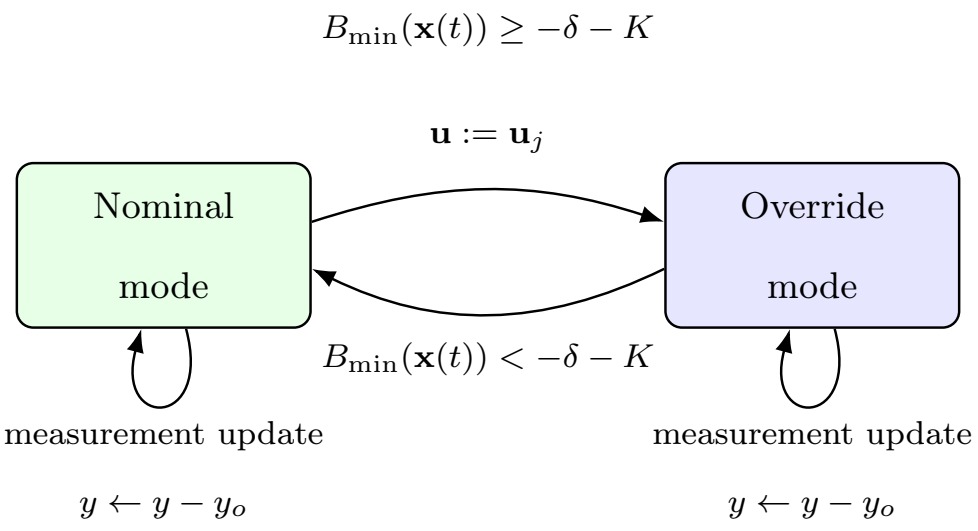


Figure 6.4: Event-triggered safety filter. Measurement updates and recentering occur at times $t_k = k\tau$, while switching from nominal to override is triggered immediately when $B_{\min}(x(t)) \geq -\delta - K$.

Numerical soundness. The repulsive barriers are SOS certificates produced by a floating-point SDP solver; their defining inequalities are validated using the high-precision positivstellensatz residue check developed uniformly in Chapter 4 [75].

6.7 Implementation and Experiments

We evaluate our approach on some numerical examples.[1] Table 6.1 shows the number of state variables, degree of polynomials, the number of barrier functions obtained and total computation time for each example.

6.7.1 3D Dynamics (Dubins Model)

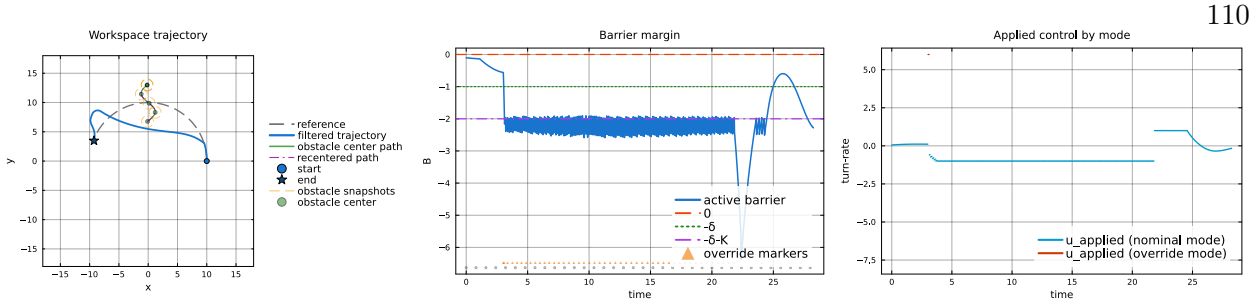
Consider a Dubins vehicle model with 3 state variables and control input $u_1 \in [-10, 10]$.

$$\dot{x}_1 = v\cos x_3,\ \dot{x}_2 = v\sin x_3,\ \dot{x}_3 = u_1.$$

We set $\hat{\mathcal{O}} = \{(x_1, x_2) | x_1^2 + x_2^2 \leq 0.6\}$. The set $U_{\text{fin}} = [-10, -6, -2, 2, 6, 10]$ and we chose 200 randomly selected test states. The repulsive barrier parameters were chosen as $\delta = 1.0$, $K = 1.0$ and $\tau = 0.1$.

Fig. 6.5 summarizes a representative closed-loop run for the Dubins vehicle under a moving obstacle with periodic recentering. Fig. 6.5(a) shows that the safety-filtered trajectory avoids the obstacle. Fig. 6.5(b) plots the active recentered barrier value over time together with the safety threshold $-\delta$, and the jump-robust margin $-\delta - K$. Throughout the trajectory, the barrier remains below zero, confirming that the closed-loop execution never enters the certified unsafe set. Moreover, whenever the barrier approaches $-\delta$, the controller "repels" it downward and re-establishes the stronger margin below $-\delta - K$. Fig. 6.5(c) shows the applied control signals. The nominal control is applied whenever sufficient barrier margin is available, whereas overrides occur only during intervals in which the safety filter is active.

[1] The implementation and resulting barrier functions are available at https://github.com/rameezw/RepulsiveBarriers



(a) Workspace trajectory under moving obstacle and measurement recentering.

(b) The minimum over barriers $B_{\min}(\mathbf{x}(t))$ against time.

(c) Nominal versus safety-filtered feedback.

Figure 6.5: Closed-loop Dubins vehicle experiment with a moving obstacle and periodic recentering. (a) The safety-filtered trajectory avoids the obstacle while remaining close to the nominal motion. (b) The active recentered barrier remains below the unsafe boundary and is periodically driven below the stronger margin $-\delta - K$ to account for future recentering jumps. (c) The applied control differs from the nominal control only when the safety filter is active.

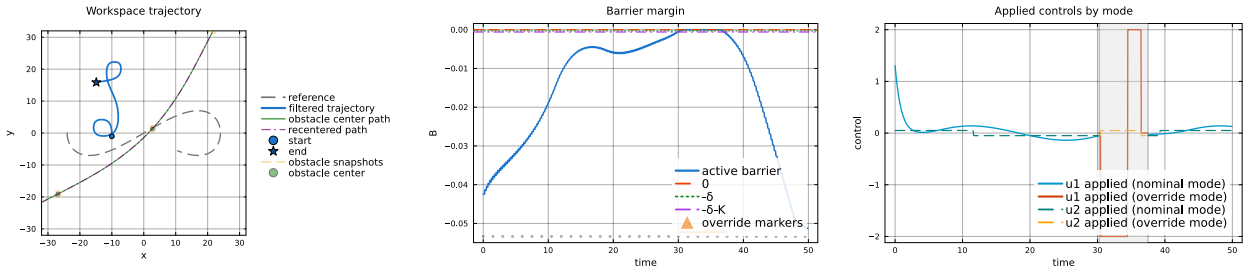
6.7.2 5D Nonlinear Dynamics (Coordinated Turn Model)

We now consider a simplified planar kinematic model for coordinated turn flight with 5 state variables and 2 control inputs $u_1, u_2 \in [-5, 5]^2$.

$$\dot{x}_1 = x_3 \cos x_4, \;\; \dot{x}_2 = x_3 \sin x_4, \;\; \dot{x}_3 = u_1$$

$$\dot{x}_4 = x_5, \;\; \dot{x}_5 = u_2 \, .$$

The unsafe set is taken to be $\hat{\mathcal{O}} = \{(x_1, x_2) | x_1^2 + x_2^2 \leq 0.6\}$. The set U_{fin} was taken to be $[-2.0, -1.0, 0.0, 1.0, 2.0] \times [-0.015, -0.0075, 0.0, 0.0075, 0.015]$ and we chose 500 randomly selected test states. The repulsive barrier parameters were chosen as $\delta = 0.003$, $K = 0.003$ and $\tau = 0.1$. An experimental run for the model is summarized in Fig. 6.6.



(a) Workspace trajectory under moving obstacle and measurement recentering.

(b) The minimum over barriers $B_{\min}(\mathbf{x}(t))$ against time.

(c) Nominal versus safety-filtered feedback.

Figure 6.6: Coordinated turn model experiment with a moving obstacle and periodic recentering.

6.7.3 6D Nonlinear Dynamics (Planar Multirotor)

We now consider a planar multirotor model with 6 state variables and 2 control inputs $u_1, u_2 \in [0, 2]^2$.

$$\dot{x}_1 = x_3, \qquad \dot{x}_4 = (u_1 + u_2)\cos x_5 - 2,$$
$$\dot{x}_2 = x_4, \qquad \dot{x}_5 = x_6,$$
$$\dot{x}_3 = (u_1 + u_2)\sin x_5, \quad \dot{x}_6 = u_1 - u_2,$$

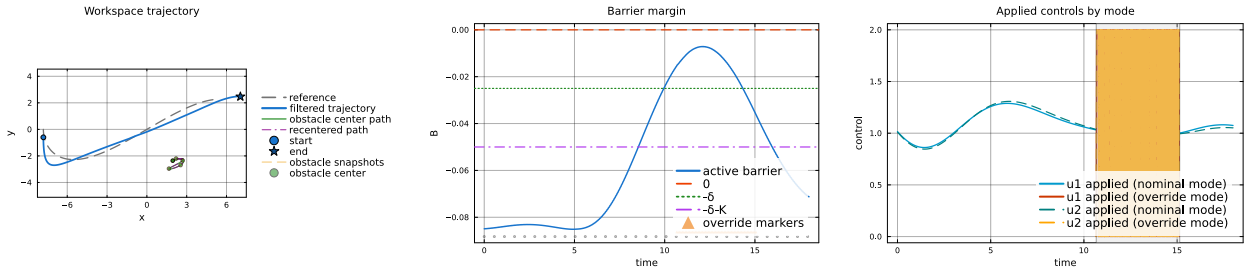
The unsafe set is taken to be $\hat{\mathcal{O}} = \{(x_1, x_2) | x_1^2 + x_2^2 \leq 0.6\}$. The set U_{fin} was set to the vertices of $U : [0, 2]^2$ and we chose 900 randomly selected test states. A representative run for the model is summarized in Fig. 6.7.

6.8 Summary

This chapter reuses the multiple-barrier construction of Chapter 3 in a dynamic, hybrid setting, exploiting translational symmetry to keep the synthesis on a compact relative-coordinate domain. Its limitations include: conservatism of the fixed-input barriers, the restriction to polynomial dynamics, and a single moving obstacle, and the workspace-invariance assumption. These are discussed alongside the other technical chapters in Chapter 7.

Table 6.1: Barrier synthesis summary.

System	# States	# Barriers	Degree	Time (s)
Dubins vehicle	3	9	4	5.5
Coordinated turn	5	25	2	10.3
Planar multirotor	6	4	4	26.5



(a) Workspace trajectory under moving obstacle and measurement recentering.

(b) The minimum over barriers $B_{\min}(\mathbf{x}(t))$ against time.

(c) Nominal versus safety-filtered feedback.

Figure 6.7: Planar multirotor model experiment with a moving obstacle and periodic recentering.

Chapter 7

Conclusion

"We can only see a short distance ahead, but we can see plenty there that needs to be done."
— Alan Turing, *Computing Machinery and Intelligence*

The dissertation shows that one mechanism, convex per-level synthesis under a finite control set, composed into a successive hierarchy, and made sound by exact rational repair, spans invariance, reachability, and moving-obstacle avoidance. This chapter summarizes the contributions, states the shared limitations, and lays out future directions, including reach-while-stay path following via successive funnels, combined safety-stability certificates, and lifting the entire pipeline into a machine-checked setting.

This dissertation set out to answer a single question: how can we make the certified region on which an autonomous system is provably safe and goal-directed *large enough to be useful*, without sacrificing the guarantees that makes certificate-based control attractive in the first place? The recurring obstacle, diagnosed in Chapter 1, is that the three difficulties of certificate synthesis; the bilinearity of the joint search for a certificate and a feedback law, the conservatism of any single monolithic certificate, and the poor scalability, are not independent. Bilinearity forces practitioners toward cheap relaxations, and those relaxations make the certified regions small. The thesis advanced in this work is that the same coupling can be turned to our advantage. A certificate made cheap enough to compute can be computed *many times*, and a hierarchy of cheap certificates, stitched together by a state-dependent switching controller, recovers much of the coverage that a single expensive certificate would have promised but could not deliver.

7.1 Summary of Contributions

The technical chapters employ one mechanism, in three progressively richer specifications, and enforce it with the numerical machinery needed to trust its output.

The mechanism has two ingredients, and both appear in every chapter. First, *control discretization breaks bilinearity*: by fixing the input to a constant value drawn from a finite set $U_{\text{fin}} \subset U$ and searching only for the certificate, the Lie-derivative condition becomes linear in the certificate's coefficients, and synthesis collapses to a convex sum-of-squares feasibility problem. The finite-control-set sufficiency theorem makes it precise: whenever a smooth feedback witnessed by a smooth certificate exists, a sufficiently fine finite control set reproduces the decrease (or invariance) property up to a constant factor, and for control-affine dynamics over a polytopic input set the vertices already suffice. Second, *successive hierarchies expand the certified region*: each level treats the union of all previously certified regions as its new target, so the cumulative region grows monotonically with the number of levels while each level remains convex. Unlike classical multiple-Lyapunov and patchy constructions, the levels need not be nested, and a lower level is reached by construction.

Against this common backbone, the four specifications are as follows.

Invariance (Chapter 3). Successive control barrier functions certify a controlled-invariant set as a union of fixed-input barriers, substantially larger than any single CBF can certify. The accompanying transit-time analysis yields a lower bound on the time a trajectory needs to cross the safety margin, which converts the nonsmooth pointwise-minimum barrier into a chattering-free, sampled-data runtime monitor with an infinite-horizon safety guarantee. The construction scales to benchmarks from two to seven state variables and certifies larger controllable sets than neural barriers verified with FOSSIL.

Numerical soundness (Chapter 4). Because every certificate in the dissertation is produced by a floating-point semidefinite solver, an apparently feasible solution may violate its own defining inequalities. Chapter 4 closes this gap with a post-hoc soundness layer that rationalizes the solver output, measures the failure of the exact identity through a residual polynomial, and repairs the Gram matrices by refitting them inside the diagonally dominant cone. Diagonal dominance both implies positive semi-definiteness without square roots and yields an explicit rational sum-of-squares decomposition. The repair is an exact rational linear program, the resulting certificate is sound, and it is checkable by a short standalone program or, for the highest level of assurance, by the `Lean 4` theorem prover. This chapter is what entitles the rest of the dissertation to call its certificates *proofs*.

Reachability (Chapter 5). The successive paradigm carries from safety to reachability. A hierarchy of control Lyapunov-like functions progressively enlarges a region of attraction by decomposing the temporal-logic property "Ω^* until $\mathcal{X}_0$" into a chain of local steps "Ω_i until Ω_{i-1}," each certified by a fixed-input CLF. A control-quantization argument establishes using finitely many inputs, switched finitely often, and the regions of attraction obtained compare favorably with neural CLF baselines on benchmarks ranging from Zubov's example to the Caltech ducted fan.

Moving obstacles (Chapter 6). Finally, for the broad class of systems whose dynamics are invariant under workspace translation, periodically recentering the state so that a measured obstacle sits at the origin turns the moving-obstacle problem into a hybrid system with measurement-induced translational jumps. Repulsive barrier functions combine a continuous-time decrease condition with robustness to the worst-case jump, yielding an infinite-horizon safety guarantee under

nondeterministic obstacle motion. They are implemented in an event-triggered safety filter that takes over from the nominal controller only when the jump-robust margin is at risk.

Taken together, the chapters show that a single idea, convex per-level synthesis under discretized control, assembled into a switching hierarchy and certified to be numerically sound, is broad enough to span invariance, reachability, and moving-obstacle avoidance, while scaling to systems beyond the comfortable reach of grid-based methods.

7.2 A Unifying Perspective

The conventional reading of certificate synthesis treats each certificate as a monolithic object whose quality is measured by the size of the single region it certifies, and treats the bilinearity of control synthesis as a wall to be scaled by ever more elaborate nonconvex machinery. This work proposes a different perspective. A certificate need not be large; it needs only to be *cheap, sound, and composable*. The finite-control-set theorem makes it cheap, by trading the existential quantifier over inputs for a finite disjunction; the soundness layer of Chapter 4 makes it trustworthy, by replacing a tolerance-bounded numerical check with an exact, auditable proof; and the successive construction makes it composable, by giving each level a target it is guaranteed to reach. Coverage then becomes a resource to be accumulated rather than a property to be established from a single optimization.

This perspective also clarifies where the approach sits relative to its neighbors. It is complementary to the online optimization viewpoint of the CBF-QP and predictive safety filters: rather than redesigning the online rule that decides, at each instant, whether to trust the nominal controller, the dissertation *enlarges the offline-certified region* on which any such rule is permitted to defer to the nominal controller, so that the filter intervenes sparingly. It is complementary to Hamilton-Jacobi reachability and to learning-based certificates as well: where the former computes the maximal set but succumbs to the curse of dimensionality, and the latter trains quickly but its verification scales poorly in dimension. The successive construction occupies the middle ground, recovering much of the coverage of the exact methods at a cost that grows with the number of levels rather than with the resolution of a grid.

7.3 Limitations

Several limitations are discussed below which are common to the technical chapters.

Conservatism is reduced, not eliminated. Each successive level reclaims states that the previous levels marked uncontrollable, but the cumulative region remains an inner approximation of the true controllable set. The rate at which it converges, and whether it converges at all for a given system, depend on the richness of the finite control set, the polynomial degree of the templates, and the geometry of the feasible set. The construction offers no a priori guarantee that the cumulative region approaches the maximal one.

The polynomial restriction. Sum-of-squares synthesis applies to polynomial dynamics and basic semi-algebraic sets, and several benchmarks are made polynomial only through Taylor or piecewise approximation of trigonometric terms, with the approximation error modeled as a bounded disturbance. The certificates are exact for the polynomial model; their transfer to the original system is only as good as that modeling step, a point Chapter 4 is careful to isolate as a modeling assumption.

Scalability. Although the per-level problem is convex and the method reaches systems where grid-based reachability is intractable, the underlying semidefinite programs still grow quickly with state dimension and certificate degree. The cone restrictions used for the exact repair (DD and SDD) are strictly smaller than the full SOS cone, so the soundness layer is itself a source of conservatism on harder problems.

The dynamic setting is narrow. The moving-obstacle results of Chapter 6 assume workspace-translation invariance, a single obstacle, and error-free periodic measurement. Each of these can be relaxed in principle, measurement error feeds into the obstacle reachable set, and other symmetries admit analogous reset maps. But the construction as presented does not yet treat multiple interacting obstacles or richer symmetry groups.

7.4 Future Directions

The unifying mechanism suggests several directions, some immediate and some more speculative.

Reach-while-stay path following via successive funnels. The most immediate extension of the successive paradigm treats the rate of progress along a reference path as a control degree of freedom rather than a fixed time schedule. Augmenting the state with a virtual progress variable $\theta \in [0, \Theta]$ that advances under a co-designed timing law $\dot{\theta} = u_0 \geq u_0^- > 0$, and measuring the deviation $\mathbf{x}_d = \mathbf{x} - \mathbf{x}_r(\theta)$ from the path, turns path following into a reach-while-stay specification: remain inside a safe tube around the path while progressing to the goal. The certificate is a *control funnel function* whose sublevel set is such a tube, and the successive construction of this dissertation applies. A hierarchy of local funnels, each driving trajectories into the union of those already certified, would enlarge the certified path-following region well beyond what a single thin funnel can cover, under one state-dependent switching controller. Two features distinguish this setting from the invariance and convergence hierarchies of the preceding chapters. First, the time-to-goal is bounded by Θ/u_0^- directly from the timing law, independently of any Lyapunov decrease rate, because progress is itself a state with a positive lower rate; the same bound holds for the switched controller, since the progress coordinate traverses $[0, \Theta]$ only once across all handoffs. Second, well-posedness of the switching controller calls for a closed-sublevel-set formulation, so that the active-level rule remains defined on the funnel boundaries. Realizing this direction would mean establishing the cumulative-coverage and Zeno-freeness guarantees with the invariance and stability hierarchies, treating the goal-absorbing condition at the path endpoint carefully, and validating the construction empirically across path-following benchmarks.

Combined safety-reachability certificates. A related direction is to fuse the invariance guarantee of the barrier construction (Chapters 3 and 6) with the convergence guarantee of the Lyapunov construction (Chapter 5) into a single certificate that simultaneously keeps a system inside a safe set and drives it toward a goal, even as the unsafe set moves. The successive paradigm

is common to both, and the recentered coordinates that handle moving obstacles are compatible with the region-of-attraction expansion of the reachability hierarchy, so a successive hierarchy over the joint specification would certify safe, goal-directed motion in dynamic environments under one switching controller.

Scalable convex relaxations. The diagonally dominant and scaled diagonally dominant relaxations introduced for the soundness layer of Chapter 4 are also synthesis tools. Replacing the per-level SOS program with a DSOS linear program or an SDSOS second-order cone program would trade some coverage for a substantial gain in scalability, and the successive hierarchy is well suited to absorb that trade: a cheaper, more conservative per-level certificate can simply be compensated by more levels. Quantifying this coverage-for-speed exchange, and choosing the relaxation adaptively per level, is an attractive line of work.

Integration with learning and sampling-based planners. The certificates developed here are designed precisely to wrap untrusted controllers, which makes them a natural shield for learned policies during both training and deployment. A complementary integration is with sampling-based motion planners. The cumulative certified region of a successive hierarchy is exactly the set of states from which a planner may safely commit to the nominal plan, so the hierarchy can serve as a fast, offline-certified feasibility oracle for online planning. Closing the loop, using the planner to suggest where coverage is lacking and the synthesis to certify the suggested region, would let the two methods improve one another.

Robustness to disturbances and richer symmetries. Replacing the strict decrease $\dot{V} \leq -\xi$ with a robust version that accounts for a bounded disturbance term would extend every chapter to systems with process noise and modeling error, at the cost of a more conservative decrease rate. On the symmetry side, the recentering idea of Chapter 6 rests on translation invariance, but rotational and scaling symmetries admit analogous reset maps, and exploiting them at the level of certificate synthesis, rather than, as in prior work, at the level of reachability computation, would broaden the class of dynamic environments the method can handle.

From checked certificates to a checked pipeline. Chapter 4 makes each certificate an auditable proof object. A further step is to lift the entire successive construction into the same machine-checked setting, so that not only the individual inequalities but also the infinite-horizon safety and finite-time reachability theorems are checked by a theorem prover. The result would be an end-to-end certified safety architecture, from polynomial model to runtime monitor.

7.5 Closing Remarks

The arc of this dissertation is from a single certificate to a hierarchy, from a nonconvex search to a sequence of convex ones, and from a numerical check to an exact proof. The thread that runs through it is a simple building block, one fixed-input certificate, synthesized convexly and certified soundly, together with the discovery that simplicity, repeated and composed, is enough. Safety and progress for autonomous systems need not wait on a single heroic certificate. They can be assembled, level by level, from cheap and trustworthy pieces, and the assembly can be made to grow exactly where the system needs room to maneuver.

If the safety-critical systems have to earn our trust, it will be through guarantees that are at once large enough to be useful, cheap enough to compute, and sound enough to check. This dissertation offers one route to all three.

References

[1] A. A. Ahmadi and A. Majumdar, "DSOS and SDSOS optimization: more tractable alternatives to sum of squares and semidefinite optimization", SIAM Journal on Applied Algebra and Geometry **3**, 193 (2019) (Cited on pp. 23, 59).

[2] M. Ahmadi, X. Xiong, and A. D. Ames, "Risk-averse control via cvar barrier functions: application to bipedal robot locomotion", IEEE Control Systems Letters **6**, 878 (2021) (Cited on p. 97).

[3] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding", in Aaai conference on artificial intelligence, Vol. 32 (2018), pp. 2669–2678 (Cited on p. 5).

[4] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: theory and applications", in 2019 18th european control conference (ecc) (2019), pp. 3420–3431 (Cited on pp. 3, 5, 76, 96, 97, 102).

[5] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems", IEEE Transactions on Automatic Control **62**, 3861 (2017) (Cited on pp. 5, 96, 97).

[6] D. L. Applegate, W. Cook, S. Dash, and D. G. Espinoza, "Exact solutions to linear programming problems", Operations Research Letters **35**, 693 (2007) (Cited on p. 64).

[7] Z. Artstein, "Stabilization with relaxed controls", Nonlinear Analysis: Theory, Methods & Applications **7**, 1163 (1983) (Cited on pp. 3, 20, 75).

[8] J.-P. Aubin, *Viability theory*, Systems & Control: Foundations & Applications (Birkhäuser, Boston, 1991) (Cited on p. 17).

[9] S. Bansal, M. Chen, S. Herbert, and C. J. Tomlin, "Hamilton-Jacobi reachability: a brief overview and recent advances", in Ieee conference on decision and control (cdc) (2017) (Cited on p. 7).

[10] K. E. Bekris, "Avoiding inevitable collision states: safety and computational efficiency in replanning with sampling-based algorithms", in Workshop on guaranteeing safe navigation in dynamic environments. in: international conference on robotics and automation (icra-10) (2010) (Cited on p. 28).

[11] K. E. Bekris, K. I. Tsianos, and L. E. Kavraki, "Safe and distributed kinodynamic replanning for vehicular networks", Mobile Networks and Applications **14**, 292 (2009) (Cited on p. 28).

[12] F. Blanchini, "Set invariance in control", Automatica **35**, 1747 (1999) (Cited on p. 17).

[13] G. Blekherman, P. A. Parrilo, and R. R. Thomas, *Semidefinite optimization and convex algebraic geometry* (SIAM, 2012) (Cited on p. 70).

[14] R. Bloem, B. Könighofer, R. Könighofer, and C. Wang, "Shield synthesis: runtime enforcement for reactive systems", in Tools and algorithms for the construction and analysis of systems (tacas) (2015), pp. 533–548 (Cited on p. 5).

[15] M. S. Branicky, "Multiple Lyapunov functions and other analysis tools for switched and hybrid systems", IEEE Transactions on Automatic Control **43**, 475 (1998) (Cited on pp. 8, 76).

[16] J. Breeden and D. Panagou, "Compositions of multiple control barrier functions under input constraints", in 2023 american control conference (acc) (2023), pp. 3688–3695 (Cited on p. 27).

[17] J. Breeden and D. Panagou, "Safety-critical control for systems with impulsive actuators and dwell time constraints", IEEE Control Systems Letters **7**, 2119 (2023) (Cited on pp. 27, 97).

[18] J. Breeden, L. Zaccarian, and D. Panagou, "Robust safety-critical control for systems with sporadic measurements and dwell time constraints", IEEE Control Systems Letters (2024) (Cited on p. 27).

[19] Y.-C. Chang, N. Roohi, and S. Gao, "Neural Lyapunov control", in Advances in neural information processing systems (neurips) (2019) (Cited on pp. 7, 92, 93).

[20] M. Chen, S. L. Herbert, H. Hu, Y. Pu, J. F. Fisac, S. Bansal, S. Han, and C. J. Tomlin, "Fastrack: a modular framework for real-time motion planning and guaranteed safe tracking", IEEE Transactions on Automatic Control **66**, 5861 (2021) (Cited on p. 97).

[21] Y. Chou, H. Yoon, and S. Sankaranarayanan, "Predictive runtime monitoring of vehicle models using bayesian estimation and reachability analysis", in Intl. conference on intelligent robots and systems (iros) (2020), pp. 2111–2118 (Cited on p. 98).

[22] F. H. Clarke, *Optimization and nonsmooth analysis*, Canadian Mathematical Society series of monographs and advanced texts (Wiley-Interscience, New York, 1983) (Cited on p. 31).

[23] A. Edwards, A. Peruffo, and A. Abate, "FOSSIL 2.0: formal certificate synthesis for the verification and control of dynamical models", in Acm international conference on hybrid systems: computation and control (hscc) (2024), pp. 1–10 (Cited on pp. 7, 10, 26, 47, 92).

[24] P. Fiorini and Z. Shiller, "Motion planning in dynamic environments using velocity obstacles", The international journal of robotics research **17**, 760 (1998) (Cited on p. 97).

[25] J. F. Fisac, A. K. Akametalu, M. N. Zeilinger, S. Kaynama, J. Gillula, and C. J. Tomlin, "A general safety framework for learning-based control in uncertain robotic systems", IEEE Transactions on Automatic Control **64**, 2737 (2019) (Cited on pp. 4, 5).

[26] T. Fraichard and H. Asama, "Inevitable collision states—a step towards safer robots?", Advanced Robotics **18**, 1001 (2004) (Cited on pp. 20, 28).

[27] K. Fukuda, "Cdd/cdd+ reference manual", Institute for Operations Research, ETH-Zentrum, 91 (1997) (Cited on p. 64).

[28] P. Glotfelter, J. Cortés, and M. Egerstedt, "Nonsmooth barrier functions with applications to multi-robot systems", IEEE control systems letters **1**, 310 (2017) (Cited on pp. 19, 31).

[29] P. Glotfelter, J. Cortés, and M. Egerstedt, "Boolean composability of constraints and control synthesis for multi-robot systems via nonsmooth control barrier functions", in 2018 ieee conference on control technology and applications (ccta) (2018), pp. 897–902 (Cited on pp. 27, 31).

[30] R. Goebel, C. Prieur, and A. R. Teel, "Smooth patchy control lyapunov functions", Automatica **45**, 675 (2009) (Cited on pp. 8, 76).

[31] R. Goebel, R. G. Sanfelice, and A. R. Teel, *Hybrid dynamical systems: modeling, stability, and robustness* (Princeton University Press, 2012) (Cited on pp. 18, 95, 100).

[32] J. Harrison, "Verifying nonlinear real formulas via sums of squares", in International conference on theorem proving in higher order logics (Springer, 2007), pp. 102–118 (Cited on p. 56).

[33] S. L. Herbert, M. Chen, S. Han, S. Bansal, J. F. Fisac, and C. J. Tomlin, "FaSTrack: a modular framework for fast and guaranteed safe motion planning", in Ieee conference on decision and control (cdc) (2017), pp. 1517–1522 (Cited on p. 97).

[34] R. A. Horn and C. R. Johnson, *Matrix analysis* (Cambridge university press, 2012) (Cited on p. 60).

[35] A. Jadbabaie and J. Hauser, "Control of a thrust-vectored flying wing: a receding horizon—lpv approach", International Journal of Robust and Nonlinear Control: IFAC-Affiliated Journal **12**, 869 (2002) (Cited on p. 91).

[36] Z. Jarvis-Wloszek, R. Feeley, W. Tan, K. Sun, and A. Packard, "Control applications of sum of squares programming", in *Positive polynomials in control* (Springer, 2005), pp. 3–22 (Cited on p. 75).

[37] H. Kong, F. He, X. Song, W. N. Hung, and M. Gu, "Exponential-condition-based barrier certificate generation for safety verification of hybrid systems", in International conference on computer aided verification (2013), pp. 242–257 (Cited on p. 96).

[38] B. Könighofer, M. Alshiekh, R. Bloem, L. Humphrey, R. Könighofer, U. Topcu, and C. Wang, "Shield synthesis", Formal Methods in System Design **51**, 332 (2017) (Cited on p. 5).

[39] J. B. Lasserre, "Global optimization with polynomials and the problem of moments", SIAM Journal on Optimization **11**, 796 (2001) (Cited on pp. 4, 21).

[40] B. Legat, C. Coey, R. Deits, J. Huchette, and A. Perry, "Sum-of-squares optimization in Julia", in The first annual jump-dev workshop (2017) (Cited on pp. 23, 30, 51, 59, 96).

[41] D. Liberzon, *Switching in systems and control*, Vol. 190 (Springer, 2003) (Cited on pp. 8, 76).

[42] J. Lim, F. Achermann, R. Girod, N. Lawrance, and R. Siegwart, "Safe low-altitude navigation in steep terrain with fixed-wing aerial vehicles", IEEE Robotics and Automation Letters (2024) (Cited on p. 28).

[43] L. Lindemann and D. V. Dimarogonas, "Control barrier functions for signal temporal logic tasks", IEEE control systems letters **3**, 96 (2018) (Cited on p. 27).

[44] J. Lofberg and P. A. Parrilo, "From coefficients to samples: a new approach to sos optimization", in 2004 43rd ieee conference on decision and control (cdc)(ieee cat. no. 04ch37601), Vol. 3 (IEEE, 2004), pp. 3154–3159 (Cited on p. 22).

[45] A. Loría, E. Panteley, D. Popovic, and A. R. Teel, "A nested matrosov theorem and persistency of excitation for uniform convergence in stable nonautonomous systems", IEEE Transactions on automatic control **50**, 183 (2005) (Cited on p. 76).

[46] M. Maaz, *Proving polynomial inequalities with sum-of-squares certificates*, (2026) https://www.mmaaz.ca/writings/sostactic.html (visited on 06/18/2026) (Cited on p. 66).

[47] M. Maaz, *Sostactic*, https://github.com/mmaaz-git/sostactic, 2026 (Cited on p. 66).

[48] M. Maghenem and R. G. Sanfelice, "Sufficient conditions for forward invariance and contractivity in hybrid inclusions using barrier functions", Automatica **124**, 109328 (2021) (Cited on p. 98).

[49] J. N. Maidens and M. Arcak, "Exploiting symmetry for discrete-time reachability computations", IEEE Control Systems Letters **2**, 213 (2018) (Cited on p. 98).

[50] A. Majumdar, A. A. Ahmadi, and R. Tedrake, "Control design along trajectories with sums of squares programming", in 2013 ieee international conference on robotics and automation (IEEE, 2013), pp. 4054–4061 (Cited on p. 75).

[51] O. Maler and D. Nickovic, "Monitoring temporal properties of continuous signals", in International symposium on formal techniques in real-time and fault-tolerant systems (Springer, 2004), pp. 152–166 (Cited on p. 17).

[52] M. Marshall, *Positive polynomials and sums of squares*, 146 (American Mathematical Soc., 2008) (Cited on p. 21).

[53] L. Martinez-Gomez, "Safe navigation for autonomous vehicles in dynamic environments: an inevitable collision state (ics) perspective", PhD thesis (Université de Grenoble, 2010) (Cited on pp. 21, 28).

[54] L. Martinez-Gomez and T. Fraichard, "An efficient and generic 2d inevitable collision state-checker", in 2008 ieee/rsj international conference on intelligent robots and systems (2008), pp. 234–241 (Cited on p. 28).

[55] L. Martinez-Gomez and T. Fraichard, "Collision avoidance in dynamic environments: an ics-based solution and its comparative evaluation", in 2009 ieee international conference on robotics and automation (2009), pp. 100–105 (Cited on p. 28).

[56] I. M. Mitchell, A. M. Bayen, and C. J. Tomlin, "A time-dependent Hamilton-Jacobi formulation of reachable sets for continuous dynamic games", IEEE Transactions on Automatic Control **50**, 947 (2005) (Cited on pp. 7, 76).

[57] S. Mitra, *Verifying cyber-physical systems: a path to safe autonomy* (MIT Press, 2021), 312 pp. (Cited on p. 100).

[58] T. S. Motzkin, "The arithmetic-geometric inequality", Inequalities (Proc. Sympos. Wright-Patterson Air Force Base, Ohio, 1965) **205**, 54 (1967) (Cited on p. 70).

[59] L. d. Moura and S. Ullrich, "The lean 4 theorem prover and programming language", in International conference on automated deduction (Springer, 2021), pp. 625–635 (Cited on p. 65).

[60] Q. Nguyen and K. Sreenath, "Exponential control barrier functions for enforcing high relative-degree safety-critical constraints", in 2016 american control conference (acc) (2016), pp. 322–328 (Cited on pp. 27, 97).

[61] P. S. Oruganti, P. Naghizadeh, and Q. Ahmed, "Robust control barrier functions for sampled-data systems", IEEE Control Systems Letters **8**, 103 (2023) (Cited on p. 98).

[62] A. Papachristodoulou, J. Anderson, G. Valmorbida, S. Prajna, P. Seiler, P. Parrilo, M. M. Peet, and D. Jagt, "Sostools version 4.00 sum of squares optimization toolbox for matlab", arXiv preprint arXiv:1310.4716 (2013) (Cited on p. 96).

[63] H. K. Park, T. Kim, and D. Panagou, "Beyond collision cones: dynamic obstacle avoidance for nonholonomic robots via dynamic parabolic control barrier functions", arXiv preprint arXiv:2510.01402 (2025) (Cited on p. 98).

[64] P. A. Parrilo, "Semidefinite programming relaxations for semialgebraic problems", Mathematical Programming **96**, 293 (2003) (Cited on pp. 4, 21, 75).

[65] P. A. Parrilo, "Polynomial optimization, sums of squares, and applications", in *Semidefinite optimization and convex algebraic geometry* (SIAM, 2012) Chap. 3, pp. 47–157 (Cited on p. 21).

[66] C. Pek and M. Althoff, "Efficient computation of invariably safe states for motion planning of self-driving vehicles", in 2018 ieee/rsj international conference on intelligent robots and systems (iros) (2018), pp. 3523–3530 (Cited on p. 28).

[67] H. Peyrl and P. A. Parrilo, "Computing sum of squares decompositions with rational coefficients", Theoretical Computer Science **409**, 269 (2008) (Cited on pp. 55, 56).

[68] S. Prajna and A. Jadbabaie, "Safety verification of hybrid systems using barrier certificates", in Hybrid systems: computation and control, edited by R. Alur and G. J. Pappas (2004), pp. 477–492 (Cited on pp. 3, 19, 76, 96).

[69] S. Prajna, P. A. Parrilo, and A. Rantzer, "Nonlinear control synthesis by convex optimization", in, Vol. 49, 2 (2004), pp. 310–314 (Cited on pp. 6, 75, 96).

[70] M. Putinar, "Positive polynomials on compact semi-algebraic sets", Indiana University Mathematics Journal **42**, 969 (1993) (Cited on pp. 21, 70).

[71] M. Rauscher, M. Kimmel, and S. Hirche, "Constrained robot control using control barrier functions", in 2016 ieee/rsj international conference on intelligent robots and systems (iros) (2016), pp. 279–285 (Cited on p. 27).

[72] H. Ravanbakhsh and S. Sankaranarayanan, "Learning control lyapunov functions from counterexamples and demonstrations", Autonomous Robots **43**, 275 (2018) (Cited on p. 105).

[73] B. Reznick, "Uniform denominators in hilbert's seventeenth problem", Mathematische Zeitschrift **220**, 75 (1995) (Cited on p. 70).

[74] B. Reznick, "Some concrete aspects of hilbert's 17th problem", Contemporary mathematics **253** (2000) (Cited on p. 70).

[75] P. Roux, Y.-L. Voronin, and S. Sankaranarayanan, "Validating numerical semidefinite programming solvers for polynomial invariants", Formal Methods in System Design **53**, 286 (2018) (Cited on pp. 10, 22, 47, 51, 54, 57, 59, 68–70, 109).

[76] R. G. Sanfelice and A. R. Teel, "Asymptotic stability in hybrid systems via nested matrosov functions", IEEE Transactions on Automatic Control **54**, 1569 (2009) (Cited on p. 76).

[77] K. Schmüdgen, "The K-moment problem for compact semi-algebraic sets", Mathematische Annalen **289**, 203 (1991) (Cited on p. 52).

[78] H. Sibai, N. Mokhlesi, and S. Mitra, "Using symmetry transformations in equivariant dynamical systems for their safety verification", in Automated technology for verification and analysis (atva), Lecture Notes in Computer Science (2019), pp. 98–114 (Cited on p. 98).

[79] J. Snape, J. Van Den Berg, S. J. Guy, and D. Manocha, "The hybrid reciprocal velocity obstacle", IEEE Transactions on Robotics **27**, 696 (2011) (Cited on p. 97).

[80] E. D. Sontag, "A 'universal'construction of artstein's theorem on nonlinear stabilization", Systems & control letters **13**, 117 (1989) (Cited on pp. 3, 20, 75).

[81] W. Tan and A. Packard, "Stability region analysis using sum of squares programming", in Proc. ACC (2007) (Cited on pp. 6, 22, 105).

[82] X. Tan, W. S. Cortez, and D. V. Dimarogonas, "High-order barrier functions: robustness, safety, and performance-critical control", IEEE Transactions on Automatic Control **67**, 3021 (2021) (Cited on p. 27).

[83] M. Tayal, B. G. Goswami, K. Rajgopal, R. Singh, M. T. Rao, J. Keshavan, P. Jagtap, and S. N. Yadukumar, "A collision cone approach for control barrier functions", IEEE Transactions on Control Systems Technology (2026) (Cited on p. 98).

[84] A. J. Taylor, V. D. Dorobantu, R. K. Cosner, Y. Yue, and A. D. Ames, "Safety of sampled-data systems with control barrier functions via approximate discrete time models", in 2022 ieee 61st conference on decision and control (cdc) (IEEE, 2022), pp. 7127–7134 (Cited on p. 98).

[85] R. Tedrake, I. R. Manchester, M. Tobenkin, and J. W. Roberts, "Lqr-trees: feedback motion planning via sums-of-squares verification", The International Journal of Robotics Research **29**, 1038 (2010) (Cited on p. 75).

[86] The mathlib Community, "The Lean Mathematical Library", in Proceedings of the 9th ACM SIGPLAN international conference on certified programs and proofs, CPP 2020 (2020) (Cited on p. 65).

[87] U. Topcu, A. Packard, and P. Seiler, "Local stability analysis using simulations and sum-of-squares programming", Automatica **44**, 2669 (2008) (Cited on pp. 6, 22).

[88] J. Van Den Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal n-body collision avoidance", in Robotics research: the 14th international symposium isrr (Springer, 2011), pp. 3–19 (Cited on p. 97).

[89] J. Van den Berg, M. Lin, and D. Manocha, "Reciprocal velocity obstacles for real-time multi-agent navigation", in 2008 ieee international conference on robotics and automation (Ieee, 2008), pp. 1928–1935 (Cited on p. 97).

[90] K. P. Wabersich, A. J. Taylor, J. J. Choi, K. Sreenath, C. J. Tomlin, A. D. Ames, and M. N. Zeilinger, "Data-driven safety filters: Hamilton-Jacobi reachability, control barrier functions, and predictive methods for uncertain systems", IEEE Control Systems Magazine **43**, 137 (2023) (Cited on p. 5).

[91] K. P. Wabersich and M. N. Zeilinger, "A predictive safety filter for learning-based control of constrained nonlinear dynamical systems", Automatica **129**, 109597 (2021) (Cited on p. 5).

[92] R. Wajid and S. Sankaranarayanan, "Successive control barrier functions for nonlinear systems", in Proceedings of the 28th acm international conference on hybrid systems: computation and control (2025), pp. 1–11 (Cited on pp. 7, 10, 74, 76, 87, 97, 105).

[93] X. Wang, T. Kim, B. Hoxha, G. Fainekos, and D. Panagou, "Safe navigation in uncertain crowded environments using risk adaptive cvar barrier functions", in 2025 ieee/rsj international conference on intelligent robots and systems (iros) (IEEE, 2025), pp. 7669–7676 (Cited on p. 97).

[94] T. Weisser, B. Legat, C. Coey, L. Kapelevich, and J. P. Vielma, "Polynomial and moment optimization in julia and jump", in Juliacon (2019) (Cited on p. 30).

[95] P. Wieland and F. Allgöwer, "Constructive safety using control barrier functions", IFAC Proceedings Volumes **40**, 462 (2007) (Cited on pp. 4, 96).

[96] W. Xiao and C. Belta, "High-order control barrier functions", IEEE Transactions on Automatic Control **67**, 3655 (2021) (Cited on p. 27).

[97] X. Xu, "Constrained control of input–output linearizable systems using control sharing barrier functions", Automatica **87**, 195 (2018) (Cited on pp. 27, 96).

[98] X. Xu, P. Tabuada, J. W. Grizzle, and A. D. Ames, "Robustness of control barrier functions for safety critical control", IFAC-PapersOnLine **48**, 54 (2015) (Cited on p. 97).